

Latakia University

Faculty of Informatics Engineering

Dept. of Software and Information Systems

Fifth Year



جامعة اللاذقية

كلية الهندسة المعلوماتية

قسم البرمجيات ونظم المعلومات

السنة الخامسة

نظام للخدمات المصرفية عبر الإنترنت

Online Banking System

(مشروع تخرج)

إعداد:

عقبه محمد سلمان

عبد الرحمن خالد حداد

خلدون عبد العزيز دهان

الإشراف:

د. رائد جابري

العام الدراسي

2025-2026

مقدمة

شهد القطاع المصرفي خلال السنوات الأخيرة تحولاً ملحوظاً نتيجة التطور السريع في تقنيات المعلومات والاتصالات، الأمر الذي أدى إلى ظهور نماذج جديدة للخدمات المالية تعتمد بشكل أساسي على الحلول الرقمية. ومن أبرز هذه النماذج البنوك الرقمية الحديثة (Neobanks)، التي تقدم خدماتها المصرفية بالكامل عبر المنصات الإلكترونية دون الحاجة إلى فروع تقليدية، مما يتيح تجربة مصرفية أكثر سرعة وكفاءة ويعزز إمكانية الوصول إلى الخدمات المالية.

ومع ازدياد الاعتماد على الأنظمة المصرفية الرقمية، أصبحت مسألة تطوير أنظمة مالية موثوقة وآمنة من التحديات التقنية المهمة. إذ تتطلب هذه الأنظمة القدرة على معالجة المعاملات المالية بكفاءة مع ضمان سلامة البيانات ومنع التلاعب بها، بالإضافة إلى الالتزام بالمعايير التنظيمية المعتمدة في القطاع المالي مثل إجراءات التحقق من هوية العملاء (KYC) ومكافحة غسل الأموال (AML).

يهدف هذا المشروع إلى تصميم وتطوير نموذج لنظام مصرفي إلكتروني مستوحى من الأنظمة المصرفية الرقمية المعتمدة في الاتحاد الأوروبي، وذلك في إطار أكاديمي يركز على تطبيق المفاهيم الحديثة في هندسة البرمجيات والأنظمة الموزعة. يوفر النظام مجموعة من الخدمات المصرفية الأساسية مثل إدارة حسابات المستخدمين، وتنفيذ التحويلات المالية، وتتبع سجل المعاملات، إضافة إلى توفير بوابة إدارية لإدارة العملاء ومراجعة عمليات التحقق من الهوية.

ومن الجوانب التقنية المميزة في هذا المشروع اعتماد سجل معاملات مستوحى من تقنية Block Chain، حيث يتم تسجيل جميع العمليات المالية ضمن سجل مترابط يعتمد على آلية ربط تجزئي (Hash Chaining)، بحيث تحتوي كل عملية على تجزئة مرتبطة بالعملية السابقة. يساهم هذا الأسلوب في إنشاء سجل معاملات قابل للتدقيق ومقاوم للتلاعب، مما يعزز من موثوقية النظام وسلامة بياناته.

كما يعتمد النظام على بنية معمارية قائمة على الخدمات المصغرة (Microservices Architecture)، والتي تسمح بتقسيم النظام إلى خدمات مستقلة يمكن تطويرها ونشرها بشكل منفصل، مما يعزز قابلية التوسع والمرونة في إدارة النظام.

ويعتمد تصميم كل خدمة من الخدمات المصغرة على نمط Hexagonal Architecture، حيث يتم فصل منطق الأعمال عن المكونات الخارجية مثل قواعد البيانات وواجهات البرمجة (APIs). يقوم هذا النمط على تنظيم التفاعل مع النظام عبر واجهات محددة (Ports)، في حين يتم تنفيذ التكامل مع الأنظمة الأخرى من خلال مكونات وسيطة (Adapters). ويسهم ذلك في تقليل الترابط بين مكونات النظام، وتحسين قابلية الاختبار، وتسهيل تطوير وصيانة الخدمات بشكل مستقل.

إضافة إلى ذلك، تم تضمين آليات لمراقبة النظام وقابلية الملاحظة (Monitoring and Observability) بهدف تتبع أداء الخدمات المختلفة وتحليل سلوك النظام أثناء التشغيل، الأمر الذي يساعد في اكتشاف الأعطال المحتملة وتحسين استقرار النظام.

وبناءً على ذلك، يسعى هذا المشروع إلى تقديم نموذج تطبيقي لنظام مصرفي رقمي يجمع بين مفاهيم هندسة البرمجيات الحديثة وتقنيات ضمان سلامة البيانات، مع التركيز على بناء نظام يتمتع بدرجة عالية من الأمان والموثوقية وقابلية التوسع.

الفهرس

5	الفصل الأول: توصيف النظام وتحليله.....
5	1-1 تمهيد.....
6	2-1 المتطلبات الوظيفية.....
8	3-1 المتطلبات غير الوظيفية.....
10	4-1 المخططات.....
14	5-1 توصيف حالات الاستخدام.....
21	الفصل الثاني: الأدوات المستخدمة لتحقيق النظام.....
21	1-2 تمهيد.....
22	2-2 البنية المعمارية للنظام.....
57	3-2 أدوات البنية التحتية.....
62	الفصل الثالث: آليات الأمان والخوارزميات المستخدمة في النظام.....
62	1-3 تمهيد.....
63	2-3 خوارزمية ربط التجزئة.....
65	3-3 خوارزمية SHA-256.....
68	4-3 خوارزمية BCrypt.....
69	5-3 توقيع JWT باستخدام RS256.....
71	6-3 قواعد مكافحة غسل الأموال (AML).....

73	7-3 آلية اعرف عميلك (KYC)
74	8-3 المعاملات الذرية.....
75	9-3 منع تكرار العمليات.....
77	الفصل الرابع: تصميم وتنفيذ الواجهة الأمامية لنظام المصرف الإلكتروني.....
77	1-4 تمهيد.....
78	2-4 التقنيات والأدوات المستخدمة في تطوير الواجهة الأمامية.....
81	3-4 البنية التنظيمية لمشروع الواجهة الأمامية.....
88	4-4 نقطة تشغيل التطبيق وبنية الرئيسية.....
89	5-4 نظام المصادقة وإدارة جلسة المستخدم.....
94	6-4 الاتصال بالواجهة الخلفية.....
96	7-4 الخدمات البرمجية في الواجهة الأمامية.....
101	8-4 صفحات الواجهة الأمامية.....
106	9-4 المكونات القابلة لإعادة الاستخدام.....
109	10-4 إدارة الحالة والوظائف المساعدة.....
112	11-4 آلية التحكم في الوصول إلى الصفحات.....
113	12-4 التعامل مع العمليات الحساسة.....
114	13-4 تصميم الواجهة وتجربة المستخدم.....
115	14-4 خلاصة تنفيذ الواجهة الأمامية.....
117	المراجع.....

الفصل الأول

(توصيف النظام وتحليله)

1-1 تمهيد:

يتناول هذا الفصل تحليل النظام المصرفي الإلكتروني المقترح وتحديد متطلباته بشكل شامل، حيث يتم في البداية عرض المتطلبات الوظيفية التي توضّح الخدمات والعمليات التي يجب أن يقدمها النظام للمستخدمين. وتشمل هذه الخدمات إنشاء الحسابات وإدارتها، تنفيذ التحويلات المالية بين الحسابات، إدارة البطاقات الافتراضية، عرض سجل المعاملات، إضافة إلى العمليات المرتبطة بالتحقق من هوية المستخدمين (KYC) وإدارة الحسابات المشتركة والإجراءات الإدارية الخاصة بالنظام. بعد ذلك يتم استعراض المتطلبات غير الوظيفية التي تهتم بجودة النظام وخصائصه التقنية، مثل مستوى الأمان وحماية البيانات المالية، سرعة الاستجابة، قابلية التوسع، الموثوقية، وإمكانيات الصيانة والتطوير المستقبلي. كما يتم أخذ بنية النظام المعتمدة على الخدمات المصغرة بعين الاعتبار، لما توفره من مرونة في إدارة مكونات النظام وتشغيلها بشكل مستقل.

يلي ذلك تقديم مخطط حالات الاستخدام (Use Case Diagram) لتوضيح كيفية تفاعل المستخدمين مع النظام في مجموعة من السيناريوهات المختلفة، سواء من طرف العملاء أو المسؤولين عن إدارة النظام، مع تقديم توصيف تفصيلي لأهم حالات الاستخدام [1] التي تعبّر عن العمليات الأساسية داخل النظام.

كما يقدم الفصل مخطط الصفوف (Class Diagram) الذي يوضح البنية الداخلية للنظام والعلاقات بين الكيانات الرئيسية مثل المستخدمين والحسابات والمعاملات والبطاقات. بالإضافة إلى ذلك يتم عرض مخطط العمليات (BPMN) الذي يبيّن تسلسل الإجراءات وتدفق العمليات بين الخدمات المختلفة داخل النظام، خاصة فيما يتعلق بمعالجة المعاملات المالية وتسجيلها في سجل المعاملات المعتمد على آلية الربط التجزئي.

يؤرّ هذا الفصل رؤية تحليلية متكاملة للبنية العامة للنظام ووظائفه الأساسية، ويقدم إطاراً واضحاً لفهم آليات عمل المكونات المختلفة وتفاعلها مع بعضها البعض، مما يساعد على تصميم الخدمات وتنفيذ العمليات بطريقة منسّقة ومتّسقة تضمن تحقيق أهداف النظام ومتطلباته بكفاءة وموثوقية.

2-1 المتطلبات الوظيفية:

1-2-1 متطلبات إدارة الحسابات والمصادقة:

- يجب أن يتمكن المستخدم غير المسجل بالنظام من:
إنشاء حساب جديد - التحقق من البريد الإلكتروني.
- يجب أن يتمكن المستخدم المسجل بالنظام من:
تسجيل الدخول إلى حسابه - تسجيل الخروج من حسابه - إعادة تعيين كلمة المرور في حال نسيانها.
- يجب أن يتمكن المسؤول من:
تسجيل الدخول إلى لوحة التحكم - تسجيل الخروج - البحث عن المستخدمين - عرض معلومات العملاء - حظر مستخدم أو إعادة تفعيل حسابه.

2-2-1 متطلبات التحقق من هوية المستخدمين (KYC):

- يجب أن يتمكن المستخدم من:
إدخال بياناته الشخصية المطلوبة للتحقق من الهوية - رفع الوثائق المطلوبة (صورة الهوية وصورة شخصية) - متابعة حالة طلب التحقق من الهوية (قيد المراجعة، مقبول، مرفوض).
- يجب أن يتمكن المسؤول من:
مراجعة طلبات التحقق من الهوية - قبول الطلب أو رفضه - إضافة ملاحظات في حال الرفض.
- يجب أن يتحقق النظام من:
عدم السماح بتنفيذ العمليات المصرفية الأساسية قبل إتمام عملية التحقق من الهوية بنجاح.

3-2-1 متطلبات إدارة الحسابات البنكية:

- يجب أن يتمكن المستخدم من:
إنشاء حساب جاري - إنشاء حساب توفير - إنشاء حساب مشترك مع مستخدم آخر - قبول أو رفض دعوة حساب مشترك - عرض رصيد الحساب - عرض رقم الحساب البنكي (IBAN).
- يجب أن يتمكن المسؤول من:

تجميد حساب بنكي - إعادة تفعيل الحساب.

1-2-4 متطلبات إدارة البطاقات البنكية الافتراضية:

- يجب أن يتمكن المستخدم من:
إنشاء بطاقة دفع افتراضية مرتبطة بالحساب - عرض تفاصيل البطاقة - قفل البطاقة مؤقتاً - إعادة تفعيل البطاقة.
- يجب أن يقوم النظام ب :
تسجيل عمليات الدفع باستخدام البطاقة كمعاملات مالية داخل النظام.

1-2-5 متطلبات التحويلات والمعاملات المالية:

- يجب أن يتمكن المستخدم من:
تحويل الأموال بين حساباته الخاصة - تحويل الأموال إلى مستخدم آخر داخل النظام - عرض سجل المعاملات المالية - عرض تفاصيل كل معاملة.
- يجب أن يتحقق النظام من:
توفر الرصيد الكافي قبل تنفيذ التحويل - صلاحية الحساب - إتمام التحقق من الهوية - منع تكرار تنفيذ المعاملة في نفس الوقت.
- يجب أن يتمكن المسؤول من:
عرض جميع المعاملات في النظام - البحث عن معاملة معينة - محاكاة عمليات تحويل واردة (مثل الراتب) لأغراض الاختبار.

1-2-6 متطلبات الفواتير والاشتراكات:

- يجب أن يتمكن المستخدم من:
عرض الفواتير الصادرة له - دفع الفواتير من رصيد الحساب - إنشاء اشتراكات دورية - إلغاء الاشتراك - عرض حالة وتاريخ الاشتراكات.
- يجب أن يتمكن المسؤول من:
إصدار فواتير للمستخدمين لأغراض المحاكاة.

7-2-1 متطلبات سجل المعاملات ومراقبة العمليات:

- يجب أن يقوم النظام ب :
تسجيل جميع الحركات المالية في سجل معاملات مترابط يعتمد على آلية الربط التجزئي (Hash Chaining) لضمان عدم التلاعب بالبيانات.
- يجب أن يتمكن المسؤول من :
عرض سجل المعاملات - التحقق من سلامة السجل - مراجعة العمليات المالية المشبوهة الناتجة عن قواعد مكافحة غسل الأموال (AML).

3-1 المتطلبات غير الوظيفية:

1-3-1 التوافر (Availability):

- يجب أن يدعم النظام بنية توزيع الخدمات (Microservices) بحيث يتم تشغيل مكونات النظام على شكل خدمات مستقلة.
- يجب أن يستمر النظام في العمل حتى في حال تعطل إحدى الخدمات، بحيث لا يؤثر فشل خدمة معينة على تشغيل باقي الخدمات.
- يجب أن يضمن النظام استمرارية تقديم الخدمات الأساسية للمستخدمين قدر الإمكان.

2-3-1 الأمان (Security):

- يجب أن يطبق النظام آليات المصادقة والتحكم في الصلاحيات (Authentication & Authorization) اعتمادًا على الأدوار المختلفة للمستخدمين مثل العميل والمسؤول.
- يجب أن يتم تخزين كلمات المرور بطريقة آمنة باستخدام تقنيات التشفير غير القابل للعكس (Hashing).

- يجب أن يقوم النظام بحماية البيانات الحساسة مثل وثائق التحقق من الهوية (KYC) وبيانات البطاقات البنكية من خلال تقييد الوصول إليها وتخزينها بطريقة آمنة.
- يجب أن يضمن النظام سرية المعلومات المالية للمستخدمين ومنع الوصول غير المصرح به إليها.

1-3-3 قابلية التوسع (Scalability):

- يجب أن يكون النظام قابلاً للتوسع الأفقي (Horizontal Scaling) بحيث يمكن زيادة عدد نسخ الخدمات عند زيادة عدد المستخدمين أو حجم الطلبات.
- يجب أن يسمح النظام بتوسيع كل خدمة بشكل مستقل حسب الحاجة دون التأثير على باقي الخدمات.
- يجب أن يدعم النظام إمكانية إضافة خدمات جديدة مستقبلاً دون التأثير على بنية النظام الأساسية.

1-3-4 قابلية الصيانة (Maintainability):

- يجب أن يتم تصميم النظام على شكل خدمات مستقلة يمكن تطويرها واختبارها ونشرها بشكل منفصل.
- يجب أن يسمح النظام بتحديث أو إصلاح إحدى الخدمات دون التأثير بشكل كبير على باقي الخدمات.
- يجب أن يعتمد النظام على واجهات واضحة ومستقرة (APIs) لضمان سهولة التواصل بين الخدمات المختلفة.

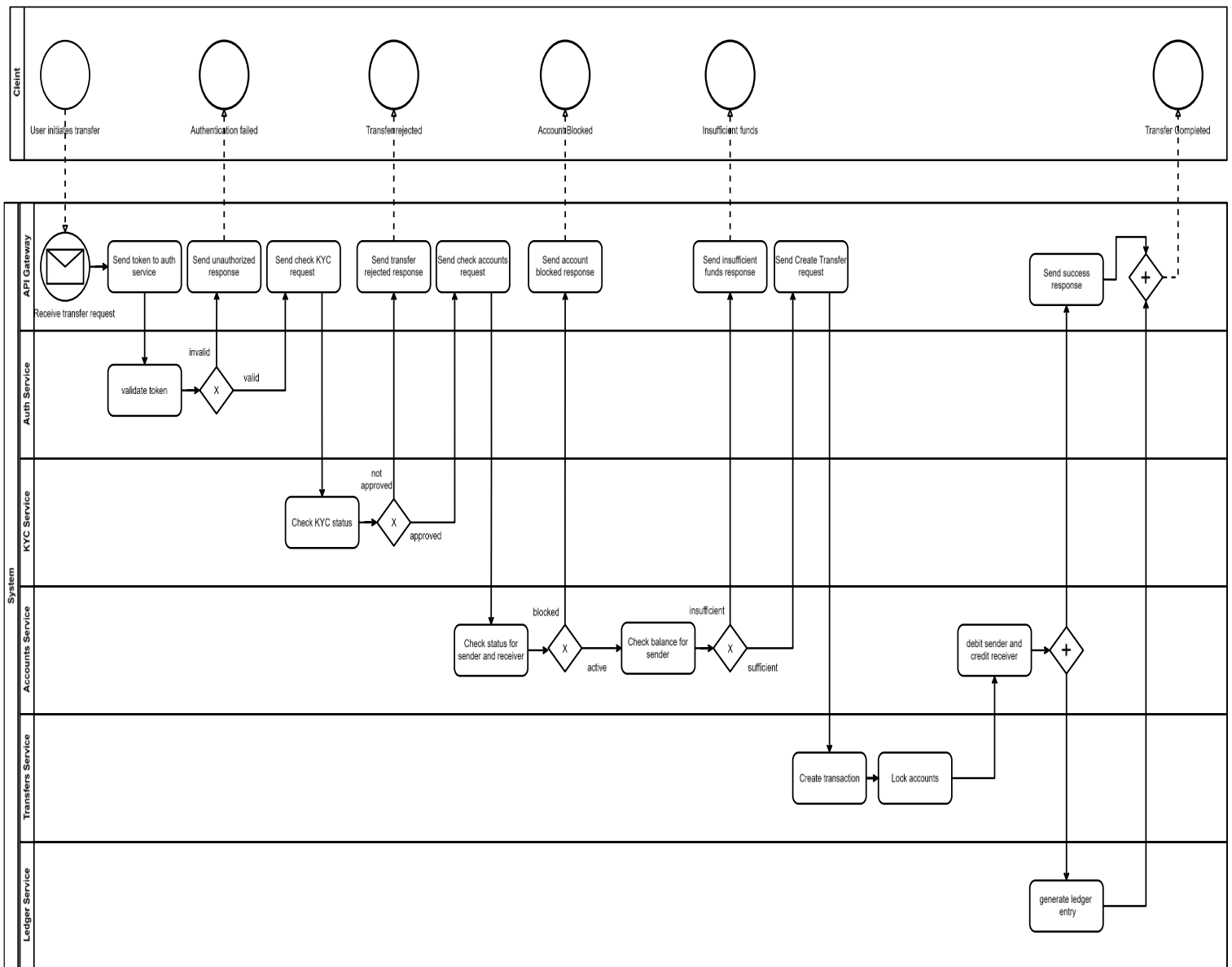
1-3-5 الموثوقية وسلامة البيانات (Reliability & Data Consistency):

- يجب أن يضمن النظام تنفيذ المعاملات المالية بشكل ذري (Atomic Transactions) بحيث يتم تنفيذ العملية بالكامل أو إلغاؤها بالكامل في حال حدوث خطأ.
- يجب أن يمنع النظام حدوث أرصدة سالبة في الحسابات البنكية.
- يجب أن يعتمد النظام أسلوب قاعدة بيانات لكل خدمة (Database per Service) لضمان استقلالية البيانات الخاصة بكل خدمة.
- يجب أن يقوم النظام بتسجيل جميع المعاملات المالية في سجل معاملات يعتمد على آلية الربط التجزئي (Hash Chaining) لضمان سلامة البيانات وإمكانية التدقيق فيها ومنع التلاعب بها.

4-1 المخططات:

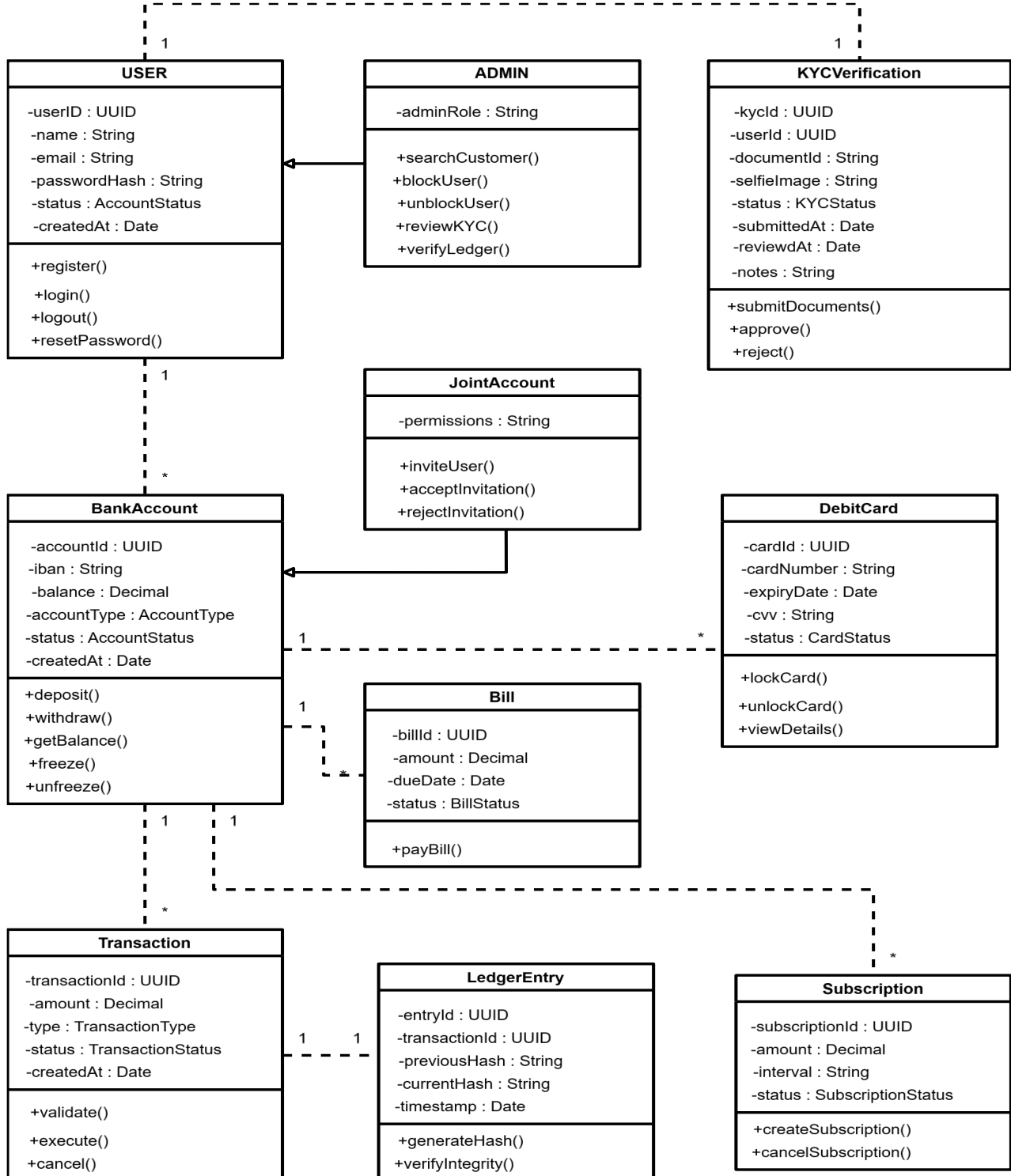
1-4-1 مخطط BPMN لحالة Transfer Money:

يوضح هذا المخطط خطوات عملية Transfer Money بشكل تسلسلي، مع تبيان الأحداث، القرارات، القرارات، والتفاعلات بين المستخدم والخدمات الخلفية [1].



2-4-1 مخطط الصفوف Class Diagram

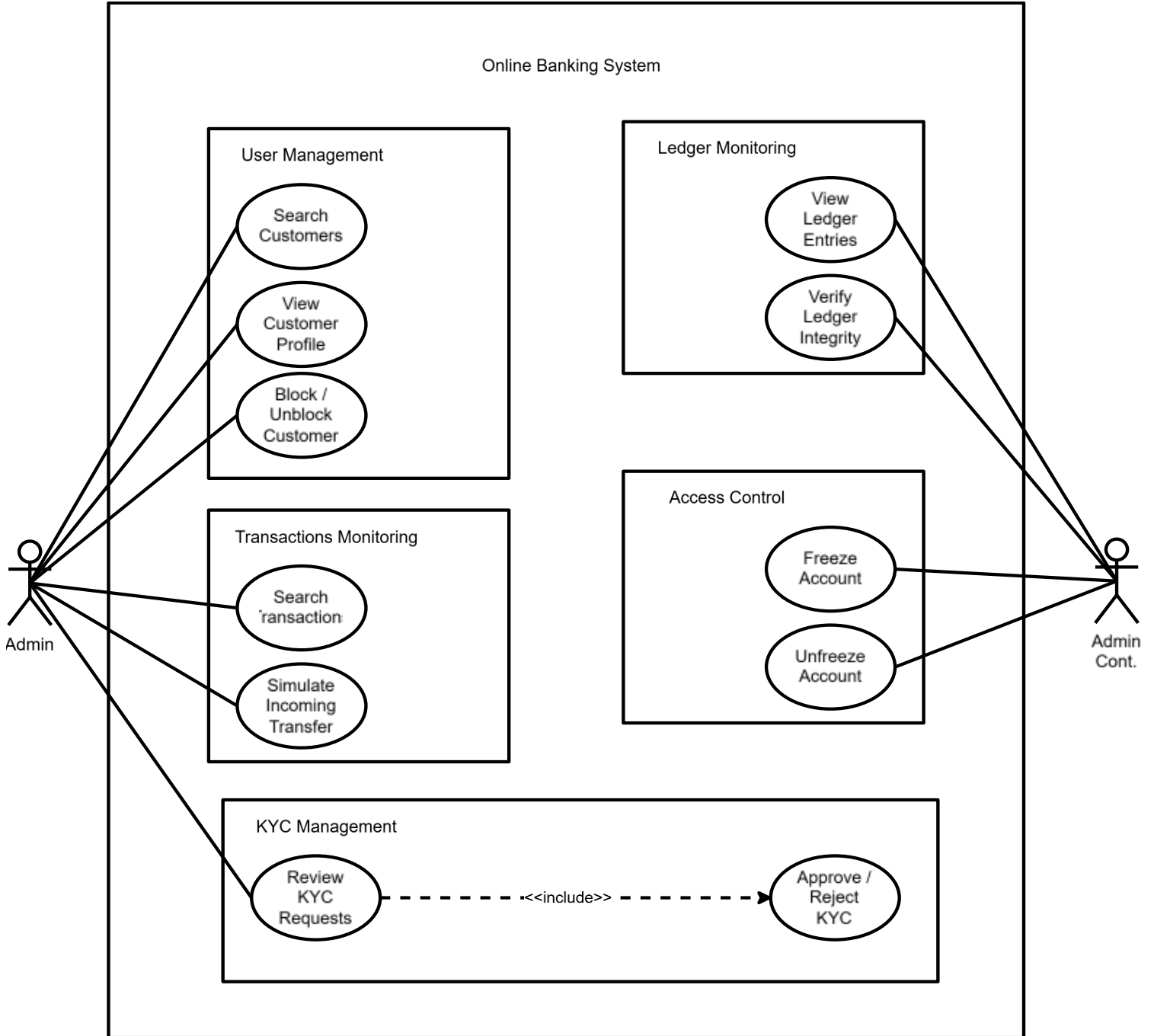
يوضح مخطط الفئات (Class Diagram) البنية الهيكلية للنظام من خلال عرض الفئات الرئيسية وخصائصها والعلاقات بينها [1].



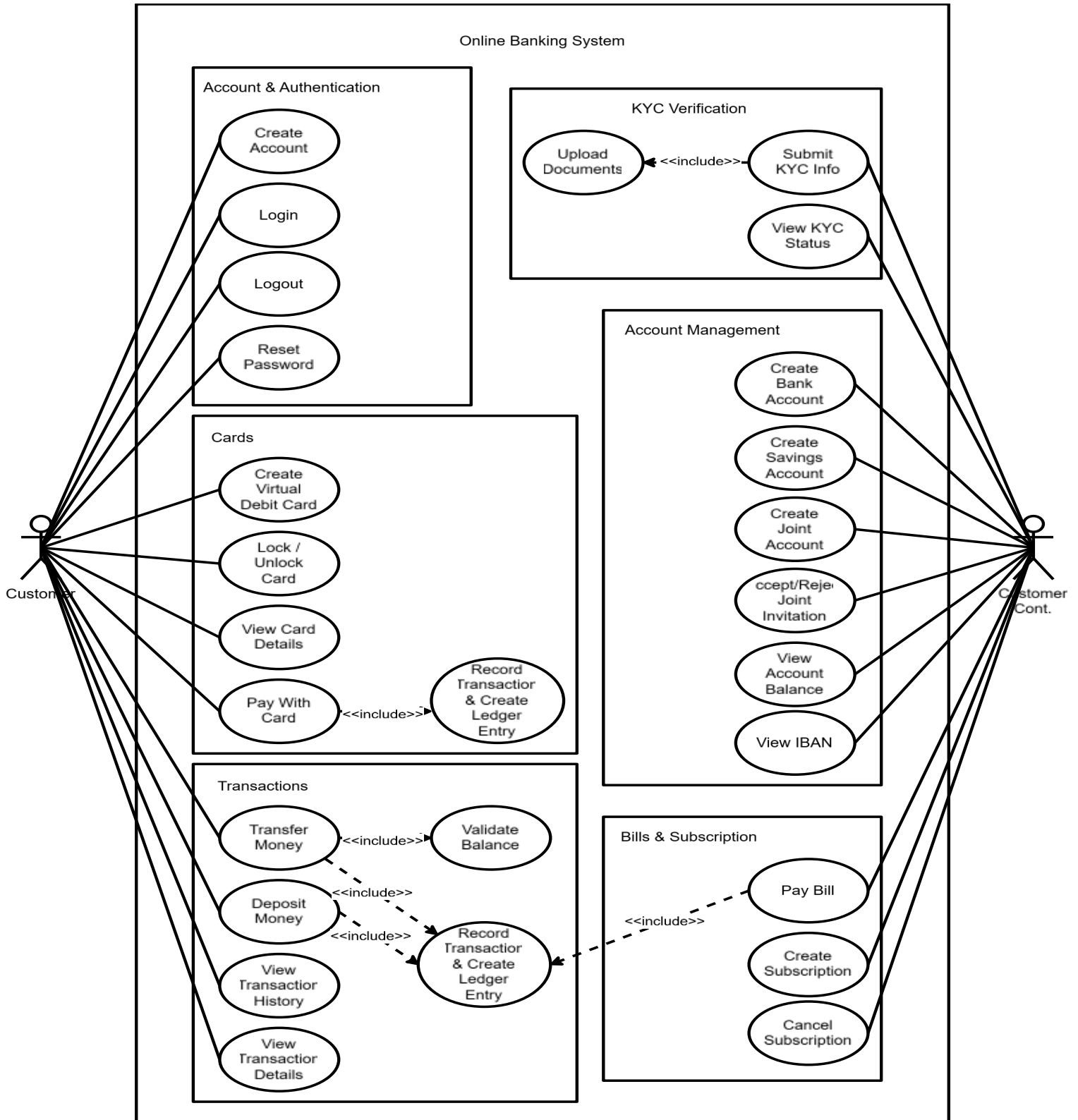
3-4-1 مخطط حالات الاستخدام :Use Case Diagram

يعرض جميع السيناريوهات المحتملة لتفاعل المستخدمين مع النظام، موضحةً الأدوار المختلفة والوظائف التي يمكن لكل مستخدم القيام بها [1].

• مخطط حالات الاستخدام الخاص بالمسؤول:



• مخطط حالات الاستخدام الخاص بالزبون:



5-1 توصيف حالات الاستخدام:

1-5-1 توصيف حالة إنشاء حساب جديد

الاسم	إنشاء حساب جديد
وصف موجز	تتعلق هذه الحالة بقيام مستخدم جديد بالتسجيل في النظام من خلال إدخال بياناته الأساسية مثل الاسم والبريد الإلكتروني وكلمة المرور، وذلك بهدف إنشاء حساب شخصي يمكن من خلاله استخدام خدمات النظام المالية.
الشروط السابقة	• عدم وجود حساب مسجل مسبقاً باستخدام نفس البريد الإلكتروني. • توفر اتصال فعال بالنظام.
الشروط اللاحقة	• إنشاء حساب جديد للمستخدم في قاعدة البيانات. • تخزين بيانات المستخدم بطريقة آمنة. • إمكانية استخدام الحساب لاحقاً لتسجيل الدخول إلى النظام.
حالات الخطأ	• إدخال بيانات ناقصة أو غير صحيحة. • استخدام بريد إلكتروني مسجل مسبقاً.
حالة النظام في حدوث الخطأ	يقوم النظام بإيقاف عملية التسجيل وعرض رسالة توضح سبب الخطأ مع طلب تصحيح البيانات المدخلة.
الجهات الفاعلة	المستخدم.
المحفز	اختيار المستخدم خيار إنشاء حساب جديد من واجهة النظام.

1. يقوم المستخدم باختيار خيار إنشاء حساب. 2. يعرض النظام نموذج إدخال البيانات. 3. يقوم المستخدم بإدخال البيانات المطلوبة. 4. يتحقق النظام من صحة البيانات المدخلة. 5. يتم إنشاء الحساب وتخزين البيانات في قاعدة البيانات. 6. يعرض النظام رسالة تؤكد نجاح عملية التسجيل.	العمليات القياسية
في حال اكتشاف أن البريد الإلكتروني مستخدم مسبقاً، يقوم النظام بإظهار رسالة تنبيه للمستخدم ويطلب إدخال بريد إلكتروني آخر.	الإجراءات البديلة

1-5-2 توصيف حالة تسجيل الدخول إلى النظام:

الاسم	تسجيل الدخول إلى النظام
وصف موجز	تصف هذه الحالة عملية دخول المستخدم المسجل مسبقاً إلى حسابه في النظام باستخدام بيانات الاعتماد الخاصة به للوصول إلى الخدمات المتاحة.
الشروط السابقة	وجود حساب مسجل في النظام.
الشروط اللاحقة	• إنشاء جلسة دخول للمستخدم داخل النظام. • السماح بالوصول إلى وظائف النظام المختلفة.
حالات الخطأ	إدخال بريد إلكتروني أو كلمة مرور غير صحيحة.
حالة النظام في حدوث الخطأ	يقوم النظام برفض عملية تسجيل الدخول وإظهار رسالة تفيد بعدم صحة بيانات الدخول.
الجهات الفاعلة	المستخدم.
المحفز	قيام المستخدم بإدخال بيانات تسجيل الدخول.
العمليات القياسية	1. يقوم المستخدم بإدخال البريد الإلكتروني وكلمة المرور. 2. يتحقق النظام من صحة بيانات الاعتماد. 3. في حال كانت البيانات صحيحة يتم إنشاء جلسة دخول. 4. يتم توجيه المستخدم إلى الصفحة الرئيسية للنظام.
الإجراءات البديلة	في حال كانت البيانات غير صحيحة يسمح النظام للمستخدم بإعادة المحاولة.

1-5-3 توصيف حالة التحقق من هوية المستخدم (KYC):

الاسم	التحقق من هوية المستخدم (KYC)
وصف موجز	تتضمن هذه الحالة قيام المستخدم بإرسال وثائق إثبات الهوية إلى النظام من أجل التحقق من صحة بياناته وتفعيل الحساب للسماح بإجراء العمليات المالية.
الشروط السابقة	- وجود حساب مسجل في النظام. - تسجيل الدخول إلى الحساب.
الشروط اللاحقة	- اعتماد هوية المستخدم وتفعيل الحساب. - السماح باستخدام الخدمات المالية للنظام.
حالات الخطأ	- رفع وثائق غير واضحة أو غير صحيحة. - عدم تطابق البيانات مع الوثائق المقدمة.
حالة النظام في حدوث الخطأ	يقوم النظام برفض عملية التحقق ويطلب من المستخدم إعادة إرسال الوثائق المطلوبة.
الجهات الفاعلة	المستخدم – المسؤول.
المحفز	قيام المستخدم بإرسال وثائق التحقق من الهوية.
العمليات القياسية	- يقوم المستخدم باختيار خيار توثيق الحساب. - يقوم برفع الوثائق المطلوبة. - يقوم النظام بتخزين الوثائق بشكل آمن. - يتم مراجعة الوثائق والتحقق من صحتها. - يتم اعتماد الحساب بعد إتمام عملية التحقق.
الإجراءات البديلة	في حال رفض الوثائق يتم إعلام المستخدم بضرورة إعادة إرسال وثائق صحيحة.

1-5-4 توصيف إيداع الأموال بالمحفظة:

الاسم	إيداع الأموال في المحفظة
وصف موجز	تصف هذه الحالة عملية إضافة مبلغ مالي إلى محفظة المستخدم من خلال وسيلة دفع معتمدة داخل النظام.
الشروط السابقة	• تسجيل دخول المستخدم. • تفعيل الحساب.
الشروط اللاحقة	• زيادة رصيد المحفظة بالمبلغ المودع. • تسجيل العملية في سجل المعاملات.
حالات الخطأ	• فشل عملية الدفع. • إدخال بيانات دفع غير صحيحة.
حالة النظام في حدوث الخطأ	يقوم النظام بإلغاء العملية دون تعديل الرصيد وإظهار رسالة توضح سبب الفشل.
الجهات الفاعلة	المستخدم.
المحفز	قيام المستخدم بطلب إيداع مبلغ مالي.
العمليات القياسية	1. يختار المستخدم خيار إيداع الأموال. 2. يقوم بإدخال المبلغ المطلوب. 3. يقوم النظام بالتحقق من بيانات الدفع. 4. يتم تنفيذ عملية الإيداع. 5. يتم تحديث رصيد المستخدم. 6. يتم تسجيل العملية في سجل المعاملات.
الإجراءات البديلة	في حال فشل الدفع يتم إلغاء العملية وإعلام المستخدم بالخطأ.

5-5-1 توصيف حالة تحويل الأموال بين المستخدمين:

الاسم	تحويل الأموال بين المستخدمين
وصف موجز	تصف هذه الحالة عملية إرسال مبلغ مالي من حساب مستخدم إلى حساب مستخدم آخر داخل النظام.
الشروط السابقة	• تسجيل دخول المستخدم. • توفر رصيد كافٍ في المحفظة.
الشروط اللاحقة	• خصم المبلغ من حساب المرسل. • إضافة المبلغ إلى حساب المستلم. • تسجيل العملية في سجل المعاملات.
حالات الخطأ	• عدم كفاية الرصيد. • إدخال حساب مستلم غير موجود.
حالة النظام في حدوث الخطأ	يتم إلغاء العملية بالكامل دون إجراء أي تعديل على الأرصدة.
الجهات الفاعلة	المستخدم.
المحفز	طلب المستخدم تنفيذ عملية تحويل.
العمليات القياسية	1. يختار المستخدم خيار تحويل الأموال. 2. يقوم بإدخال حساب المستلم والمبلغ المطلوب. 3. يتحقق النظام من توفر الرصيد الكافي. 4. يتم تنفيذ عملية التحويل. 5. يتم تحديث أرصدة الحسابات. 6. يتم تسجيل العملية في سجل المعاملات.
الإجراءات البديلة	في حال عدم توفر رصيد كافٍ يتم إيقاف العملية وإظهار رسالة مناسبة.

1-5-6 توصيف حالة عرض سجل المعاملات:

الاسم	عرض سجل المعاملات
وصف موجز	تتعلق هذه الحالة بعرض قائمة العمليات المالية المرتبطة بحساب المستخدم مثل الإيداعات والتحويلات والسحوبات.
الشروط السابقة	تسجيل دخول المستخدم.
الشروط اللاحقة	عرض سجل العمليات المالية المرتبطة بالحساب.
حالات الخطأ	فشل الاتصال بقاعدة البيانات.
حالة النظام في حدوث الخطأ	يقوم النظام بعرض رسالة تفيد بعدم إمكانية جلب البيانات في الوقت الحالي.
الجهات الفاعلة	المستخدم.
المحفز	قيام المستخدم بطلب عرض سجل المعاملات.
العمليات القياسية	1. يختار المستخدم خيار عرض المعاملات. 2. يقوم النظام بجلب بيانات العمليات من قاعدة البيانات. 3. يتم ترتيب العمليات زمنياً. 4. يتم عرضها للمستخدم.
الإجراءات البديلة	في حال عدم وجود معاملات يتم عرض رسالة تفيد بعدم وجود عمليات مسجلة.

الفصل الثاني

(الأدوات المستخدمة لتحقيق النظام)

1-2 تمهيد:

يستعرض هذا الفصل الأدوات والتقنيات التي تم الاعتماد عليها في تصميم وتطوير النظام المصرفي الإلكتروني، مع توضيح الدور الذي تؤديه كل منها في تحقيق متطلبات النظام الوظيفية وغير الوظيفية. فقد تم بناء النظام بالاعتماد على مجموعة من التقنيات الحديثة في مجال هندسة البرمجيات والأنظمة الموزعة، بهدف توفير نظام يتمتع بدرجة عالية من الأمان والموثوقية وقابلية التوسع وسهولة الصيانة.

يبدأ الفصل بعرض البنية المعمارية المعتمدة للنظام، والأسس التي بُني عليها تصميم الخدمات وآلية التواصل فيما بينها، ثم ينتقل إلى استعراض تقنيات التطوير المستخدمة في بناء مكونات النظام، يلي ذلك عرض أدوات البنية التحتية التي وفرت بيئة مناسبة لتشغيل الخدمات وإدارة البيانات ومراقبة أداء النظام.

كما يتناول الفصل وصف الخدمات الأساسية التي يتكون منها النظام، مع بيان مسؤولية كل خدمة ودورها ضمن البنية العامة، وكيفية تكاملها مع الخدمات الأخرى لتقديم الوظائف المصرفية المختلفة. أما الواجهة الأمامية فيشرح التقنيات المستخدمة في تطويرها وآلية تفاعلها مع بقية مكونات النظام.

ويهدف هذا الفصل إلى تقديم فهم واضح للبنية التقنية للنظام قبل الانتقال إلى تفاصيل التنفيذ، مما يساعد على توضيح الأسباب التي دفعت إلى اختيار هذه التقنيات، وكيف ساهمت في بناء نظام مصرفي رقمي قادر على تلبية متطلبات الأداء والأمان وقابلية التوسع وفق مبادئ هندسة البرمجيات الحديثة.

2-2 البنية المعمارية للنظام (System Architecture):

1-2-2 نظرة عامة على البنية المعمارية للنظام (System Architecture Overview):

تم تصميم النظام المصرفي الإلكتروني المقترح بالاعتماد على بنية معمارية حديثة تهدف إلى تحقيق مجموعة من المتطلبات الأساسية في الأنظمة المالية الرقمية، مثل الاعتمادية (Reliability)، الأمان (Security)، قابلية التوسع (Scalability)، وسهولة الصيانة والتطوير (Maintainability). ونظراً للطبيعة الحساسة للأنظمة المصرفية التي تتعامل مع بيانات مالية ومعلومات شخصية للمستخدمين، فقد تم اعتماد تصميم هندسي يركز على فصل المسؤوليات وتقليل الترابط بين مكونات النظام المختلفة.

يعتمد النظام على نموذج الخدمات المصغرة (Microservices Architecture) [2] ، حيث تم تقسيم النظام إلى مجموعة من الخدمات المستقلة، بحيث تكون كل خدمة مسؤولة عن مجال وظيفي محدد داخل النظام. هذا الأسلوب يسمح بتطوير كل مكون وتشغيله وتحديثه بشكل مستقل، كما يقلل من تأثير حدوث أي مشكلة داخل خدمة معينة على باقي الخدمات، مما يزيد من استقرار النظام وقدرته على التعامل مع التوسع المستقبلي. يتكون النظام من مجموعة من الخدمات الأساسية التي تمثل الوظائف الرئيسية للنظام المصرفي، وتشمل خدمة إدارة المستخدمين والمصادقة (Authentication Service)، خدمة إدارة الحسابات البنكية (Accounts Service)، خدمة تنفيذ التحويلات المالية (Transfers Service)، خدمة سجل المعاملات المالية (Ledger Service)، خدمة التحقق من الهوية (KYC Service)، خدمة الفواتير والاشتراكات (Billing Service)، خدمة البطاقات الافتراضية (Cards Service)، وخدمة الإشعارات (Notifications Service). بالإضافة إلى ذلك، تم إنشاء خدمة بوابة النظام (Gateway Service) التي تعمل كنقطة دخول موحدة للتطبيقات الخارجية وتقوم بإدارة عملية التواصل بين المستخدمين والخدمات الداخلية.

يعتمد التواصل بين الخدمات الداخلية على بروتوكول الاتصال عالي الأداء (gRPC)، حيث يتم تعريف واجهات الاتصال باستخدام ملفات (Protocol Buffers)، مما يسمح بإنشاء عقود اتصال واضحة بين الخدمات المختلفة ويضمن سرعة وكفاءة تبادل البيانات. أما بالنسبة للتواصل مع المستخدمين أو أي تطبيق

خارجي، فيتم من خلال واجهة برمجية موحدة (API Gateway) تعتمد على بروتوكول (HTTP)، حيث تقوم البوابة باستقبال الطلبات والتحقق من هوية المستخدم ثم توجيه الطلب إلى الخدمة المناسبة.

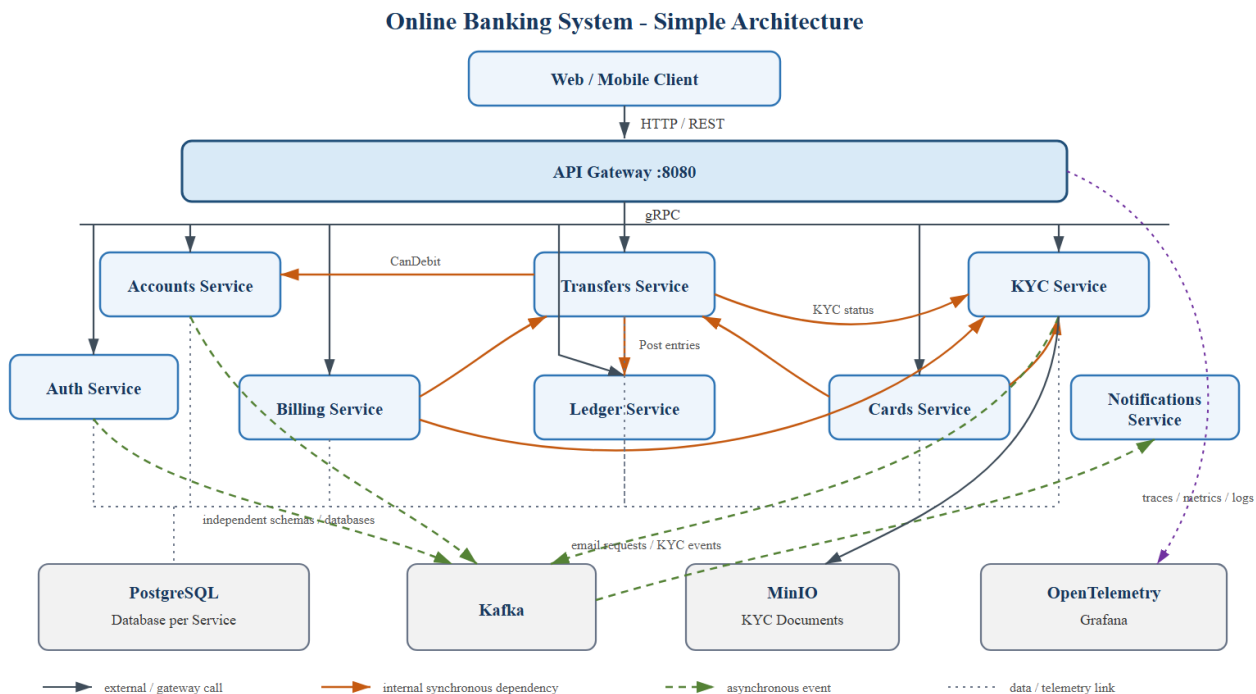
تم اعتماد مبدأ استقلالية البيانات من خلال استخدام نموذج قاعدة بيانات خاصة لكل خدمة (Database per Service)، حيث تمتلك كل خدمة قاعدة بيانات مستقلة مسؤولة عن تخزين وإدارة البيانات الخاصة بها فقط. يساهم هذا التصميم في تقليل الاعتماد المباشر بين الخدمات، ويحافظ على حدود واضحة بين مجالات النظام المختلفة، كما يسمح بإمكانية تطوير أو تغيير طريقة تخزين البيانات في خدمة معينة دون التأثير على بقية الخدمات.

بالإضافة إلى التصميم المعماري للخدمات، تم اعتماد نمط البنية السداسية (Hexagonal Architecture) داخل كل خدمة، والذي يعتمد على فصل منطق النظام الأساسي (Core Business Logic) [3] عن المكونات الخارجية مثل قواعد البيانات، واجهات الاتصال، وخدمات الطرف الثالث. يساعد هذا الأسلوب على تحسين قابلية اختبار النظام، وتقليل الترابط بين المكونات، وتسهيل عملية إضافة أو تغيير أي تقنية خارجية مستقبلاً.

كما تم تصميم النظام مع مراعاة الجوانب المتعلقة بالأمان، حيث يتم تطبيق نظام مصادقة مركزي يعتمد على الرموز الأمنية (JWT)، ويتم تمرير معلومات هوية المستخدم وصلاحياته بين الخدمات باستخدام بيانات الاتصال (gRPC Metadata). بالإضافة إلى ذلك، تم تضمين آليات لحماية البيانات الحساسة، والتحقق من العمليات المالية، ومنع تنفيذ العمليات غير المصرح بها.

من ناحية البنية التشغيلية، يعتمد النظام على تقنية الحاويات (Containerization) باستخدام (Docker)، حيث يتم تشغيل كل خدمة ضمن حاوية مستقلة، مما يوفر بيئة تشغيل موحدة ويسهل عملية النشر (Deployment) والإدارة. كما يتم تشغيل المكونات الداعمة مثل قواعد البيانات، نظام الرسائل (Kafka)، ونظام تخزين الملفات (MinIO) ضمن بيئة تشغيل موحدة باستخدام (Docker Compose).

يوضح الشكل التالي البنية العامة للنظام والعلاقة بين مكوناته الرئيسية:



ومن خلال هذه البنية المعمارية، يحقق النظام توازناً بين الأداء، الأمان، وقابلية التطوير، مما يجعله نموذجاً مناسباً لتطبيقات الخدمات المصرفية الرقمية الحديثة التي تتطلب مستوى عالياً من الاعتمادية والمرونة في التعامل مع العمليات المالية المختلفة.

2-2-2 المعمارية القائمة على الخدمات المصغرة (Microservices Architecture):

تُعد المعمارية القائمة على الخدمات المصغرة (Microservices Architecture) أحد أهم القرارات الهندسية التي تم اعتمادها في تصميم النظام المصرفي الإلكتروني، حيث تم بناء النظام على شكل مجموعة من الخدمات المستقلة بدلاً من تطبيق واحد ضخم يحتوي على جميع الوظائف. تعتمد هذه المعمارية على تقسيم النظام إلى وحدات صغيرة ومستقلة، بحيث تكون كل خدمة مسؤولة عن مجال وظيفي محدد وتمتلك منطق الأعمال (Business Logic) والبيانات الخاصة بها.

يختلف هذا الأسلوب عن المعمارية التقليدية أحادية التطبيق (Monolithic Architecture)، والتي يتم فيها بناء جميع وظائف النظام ضمن مشروع واحد مترابط. ففي الأنظمة الأحادية، يؤدي أي تعديل على جزء معين من النظام إلى الحاجة إلى إعادة بناء ونشر التطبيق كاملاً، كما أن زيادة حجم النظام مع مرور الوقت قد تؤدي إلى صعوبة في الصيانة والتطوير. أما في نموذج الخدمات المصغرة، فإن كل خدمة تمثل مكوناً مستقلاً يمكن تطويره واختباره ونشره بشكل منفصل عن باقي الخدمات.

تم اختيار هذه المعمارية في النظام المصرفي نظراً لطبيعة الأنظمة المالية التي تحتاج إلى درجة عالية من المرونة وقابلية التوسع. فعلى سبيل المثال، قد تحتاج خدمة التحويلات المالية (Transfers Service) إلى موارد تشغيلية أكبر نتيجة زيادة عدد العمليات المالية، بينما قد لا تحتاج خدمة الإشعارات (Notifications Service) إلى نفس مستوى الموارد. يسمح تقسيم النظام إلى خدمات مستقلة بتوسيع الخدمة المطلوبة فقط دون الحاجة إلى زيادة موارد النظام بالكامل.

يعتمد كل مكون في النظام على مبدأ امتلاك مسؤولية محددة (Single Responsibility)، حيث تم توزيع وظائف النظام على الخدمات التالية:

خدمة المصادقة وإدارة المستخدمين (Authentication Service)

تتولى هذه الخدمة جميع العمليات المرتبطة بهوية المستخدمين، وتشمل إنشاء الحسابات، تسجيل الدخول، التحقق من البريد الإلكتروني، إصدار الرموز الأمنية (JWT Tokens)، إدارة الرموز طويلة الأجل (Refresh Tokens)، بالإضافة إلى العمليات الإدارية مثل البحث عن المستخدمين وحظر الحسابات.

خدمة إدارة الحسابات البنكية (Accounts Service)

تختص هذه الخدمة بإدارة الحسابات البنكية داخل النظام، وتشمل إنشاء الحسابات الجارية وحسابات التوفير، إدارة الحسابات المشتركة، التحكم في عضويات الحسابات، بالإضافة إلى تنفيذ قواعد التحقق من إمكانية الخصم من الحساب (Can Debit) قبل تنفيذ العمليات المالية.

خدمة التحويلات المالية (Transfers Service)

تعتبر هذه الخدمة المسؤولة عن إدارة دورة حياة التحويلات المالية، حيث تقوم بتنسيق عملية التحويل بين الخدمات الأخرى. قبل تنفيذ أي عملية تحويل، تقوم بالتحقق من حالة الحساب، حالة التحقق من الهوية (KYC)، تطبيق قواعد مكافحة غسل الأموال (AML)، ثم إرسال العملية إلى خدمة سجل المعاملات (Ledger Service) لتنفيذ الحركة المالية وتسجيلها.

خدمة سجل المعاملات المالية (Ledger Service)

تعمل هذه الخدمة كطبقة مسؤولة عن تسجيل جميع العمليات المالية داخل النظام. تقوم بحفظ القیود المالية (Ledger Entries) وحركات الحسابات (Postings)، بالإضافة إلى إدارة أرصدة الحسابات وإنشاء سلسلة مترابطة من العناصر باستخدام آلية الربط التجزئي (Hash Chaining) لضمان سلامة السجل ومنع التلاعب بالبيانات المالية.

خدمة التحقق من الهوية (KYC Service)

تختص هذه الخدمة بإدارة عمليات التحقق من هوية العملاء، حيث تسمح للمستخدمين بتقديم الوثائق المطلوبة، إدارة مراحل طلب التحقق، وتوفير آلية مراجعة للمسؤولين. كما تتكامل مع نظام التخزين (MinIO) لحفظ وثائق التحقق بطريقة آمنة.

خدمة الفواتير والاشتراكات (Billing Service)

مسؤولة عن إدارة العمليات المتعلقة بدفع الفواتير وإنشاء الاشتراكات الدورية. تقوم الخدمة بتنفيذ عمليات الدفع والتحقق من الشروط المطلوبة قبل تنفيذ العملية، بالإضافة إلى تشغيل مهام مجدولة (Scheduled Jobs) لمعالجة الاشتراكات المستحقة بشكل تلقائي.

خدمة البطاقات الافتراضية (Cards Service)

تختص هذه الخدمة بإدارة البطاقات الافتراضية الخاصة بالمستخدمين، وتشمل إنشاء البطاقات، عرض بياناتها، تجديدها أو إعادة تفعيلها، بالإضافة إلى تنفيذ عمليات الدفع المرتبطة بها من خلال إنشاء معاملات مالية داخل النظام.

خدمة الإشعارات (Notifications Service)

تعمل هذه الخدمة على معالجة الرسائل والأحداث القادمة من الخدمات الأخرى، حيث تستقبل طلبات إرسال البريد الإلكتروني عبر نظام الرسائل (Apache Kafka)، ثم تقوم بإرسال الإشعارات باستخدام بروتوكول البريد الإلكتروني (SMTP).

خدمة بوابة النظام (Gateway Service)

تمثل هذه الخدمة نقطة الدخول الرئيسية للنظام، حيث تتعامل مع طلبات المستخدمين الخارجية عبر واجهات (HTTP API)، وتقوم بالتحقق من صحة الرموز الأمنية (JWT)، ثم تحويل الطلبات إلى الخدمات الداخلية باستخدام (gRPC) كما تقوم بتوحيد طريقة التعامل مع الأخطاء وإرجاع استجابات مناسبة للتطبيقات الخارجية.

يعتمد هذا التقسيم على مبدأ فصل المجالات الوظيفية (Domain Separation)، حيث يتم عزل كل مجال من مجالات النظام داخل خدمة مستقلة، مما يؤدي إلى تقليل الاعتماد المباشر بين المكونات وتحسين قدرة النظام على التطوير المستقبلي. فعلى سبيل المثال، يمكن تطوير نظام التحقق من الهوية أو تغيير آلية تخزين الوثائق دون الحاجة إلى تعديل خدمات الحسابات أو التحويلات المالية.

كما توفر الخدمات المصغرة إمكانية النشر المستقل (Independent Deployment)، حيث يمكن تحديث خدمة معينة أو إصلاح مشكلة داخلها وإعادة تشغيلها دون الحاجة إلى إيقاف النظام بالكامل. وهذا الأمر يعتبر مهماً بشكل خاص في الأنظمة المصرفية التي تتطلب استمرارية عالية في تقديم الخدمات.

وبالإضافة إلى ذلك، تسمح هذه المعمارية بتطبيق سياسات أمان ومراقبة مستقلة لكل خدمة حسب طبيعتها، مع المحافظة على وجود آليات موحدة للتحقق من الهوية وإدارة الصلاحيات بين جميع مكونات النظام.

وبذلك تشكل معمارية الخدمات المصغرة الأساس الهندسي للنظام المقترح، حيث توفر بنية مرنة وقابلة للتوسع تتناسب مع متطلبات الأنظمة المصرفية الرقمية الحديثة، وتسمح بإضافة خدمات جديدة مستقبلاً دون الحاجة إلى إعادة بناء النظام من الصفر.

3-2-2 نمط البنية السداسية (Hexagonal Architecture):

تم اعتماد نمط البنية السداسية (Hexagonal Architecture)، والذي يعرف أيضاً باسم نمط المنافذ والمحولات (Ports and Adapters Pattern)، كطريقة تصميم داخلية لكل خدمة من خدمات النظام المصرفي. يهدف هذا النمط إلى تحقيق فصل واضح بين منطق الأعمال الأساسي (Core Business Logic) والمكونات الخارجية التي يتعامل معها النظام، مثل قواعد البيانات، واجهات الاتصال، أنظمة الرسائل، والخدمات الخارجية.

تعتبر هذه البنية مناسبة بشكل خاص للأنظمة المصرفية التي تحتوي على قواعد عمل معقدة (Complex Business Rules)، حيث يجب أن يبقى منطق العمليات المالية مستقلاً عن التقنيات المستخدمة لتنفيذها. فعلى سبيل المثال، يجب ألا يعتمد منطق تنفيذ التحويل المالي على نوع قاعدة البيانات المستخدمة أو على طريقة الاتصال بين الخدمات، وإنما يجب أن يكون مسؤولاً فقط عن تطبيق القواعد المالية والتحقق من صحة العملية.

يعتمد هذا النمط على جعل طبقة المجال (Domain Layer) هي النواة الأساسية للنظام، بحيث تحتوي على الكيانات (Entities)، القيم الثابتة (Enums)، والقواعد الأساسية المرتبطة بمجال الخدمة. ولا تحتوي هذه الطبقة على أي اعتماد مباشر على الأدوات الخارجية، مما يجعلها مستقلة وقابلة للاختبار والتطوير.

يمكن توضيح البنية الداخلية لكل خدمة كما يلي:

طبقة المجال (Domain Layer)

تمثل هذه الطبقة الجزء الأساسي من الخدمة، حيث تحتوي على المفاهيم والبيانات المرتبطة بالمجال الوظيفي الخاص بها. في النظام المصرفي، تختلف محتويات هذه الطبقة حسب مسؤولية كل خدمة.

على سبيل المثال:

- في خدمة الحسابات (Accounts Service) تحتوي على الكيانات المرتبطة بالحسابات البنكية والعضويات .
- في خدمة التحويلات (Transfers Service) تحتوي على كيان التحويل والقواعد المتعلقة بحالته .
- في خدمة سجل المعاملات (Ledger Service) تحتوي على عناصر السجل المالي والقيود المرتبطة بها .

تتميز هذه الطبقة بأنها لا تعرف أي تفاصيل عن طريقة تخزين البيانات أو طريقة استقبال الطلبات، مما يحافظ على استقلالية منطق الأعمال.

طبقة التطبيق (Application Layer)

تحتوي هذه الطبقة على حالات الاستخدام (Use Cases) التي تمثل العمليات التي يمكن تنفيذها داخل الخدمة. تقوم هذه الطبقة بتنسيق خطوات تنفيذ العملية وتطبيق قواعد النظام باستخدام مكونات طبقة المجال.

على سبيل المثال، عند تنفيذ عملية تحويل مالي، تقوم طبقة التطبيق في (Transfers Service) بالخطوات التالية:

1. استقبال طلب التحويل.
2. التحقق من صحة البيانات المدخلة.
3. التأكد من حالة التحقق من الهوية (KYC).
4. التحقق من صلاحية الخصم من الحساب.

5. تطبيق قواعد مكافحة غسل الأموال (AML).

6. إرسال الطلب إلى خدمة سجل المعاملات (Ledger Service).

7. تحديث حالة التحويل بعد نجاح العملية.

وبذلك تبقى خطوات العمل الأساسية داخل طبقة التطبيق دون الاعتماد المباشر على تفاصيل التنفيذ الخارجية.

المنافذ (Ports)

تمثل المنافذ واجهات الاتصال التي تحدد كيفية تفاعل طبقة التطبيق مع العالم الخارجي. وتنقسم إلى نوعين رئيسيين:

المنافذ الداخلية (Inbound Ports)

وهي الواجهات التي تحدد العمليات التي يمكن تنفيذها من قبل المستخدمين أو الخدمات الأخرى.

مثال:

- إنشاء حساب جديد.
- تنفيذ تحويل مالي.
- الحصول على رصيد حساب.
- إرسال طلب تحقق من الهوية.

يتم استدعاء هذه المنافذ من خلال المحولات الواردة مثل وحدات التحكم (Controllers).

المنافذ الخارجية (Outbound Ports)

وهي الواجهات التي تحدد الخدمات الخارجية التي تحتاجها الخدمة لتنفيذ عملياتها.

مثال:

- الوصول إلى قاعدة البيانات.
 - إرسال رسالة عبر Kafka.
 - استدعاء خدمة أخرى باستخدام gRPC.
- لا تحتوي طبقة التطبيق على تفاصيل تنفيذ هذه العمليات، بل تعتمد فقط على تعريف المنفذ المطلوب.

المحولات (Adapters)

تمثل المحولات الطبقة المسؤولة عن تنفيذ عملية الاتصال الفعلية مع المكونات الخارجية. وتنقسم إلى:

المحولات الواردة (Inbound Adapters)

وهي المكونات التي تستقبل الطلبات من خارج الخدمة وتحولها إلى أوامر يمكن لطبقة التطبيق فهمها. في هذا المشروع يتمثل ذلك بشكل أساسي في:

- وحدات التحكم باستخدام (gRPC Controllers).
- حيث تقوم باستقبال طلبات الخدمات الأخرى، استخراج البيانات المطلوبة، ثم استدعاء حالات الاستخدام المناسبة.

المحولات الصادرة (Outbound Adapters)

وهي المكونات التي تنفذ عمليات الاتصال مع الأنظمة الخارجية، وتشمل:

- محولات قواعد البيانات (JPA Repository Adapters).
- محولات الاتصال بالخدمات الأخرى باستخدام (gRPC Clients).
- محولات إرسال الأحداث باستخدام (Kafka Producers).
- محولات إرسال البريد الإلكتروني (SMTP Adapter).
- محولات التخزين باستخدام (MinIO Adapter).

يسمح هذا الفصل بتغيير أي مكون خارجي دون الحاجة إلى تعديل منطق الأعمال. فعلى سبيل المثال، يمكن تغيير نظام تخزين الملفات من (MinIO) إلى نظام آخر دون تغيير آلية معالجة طلبات التحقق من الهوية داخل خدمة (KYC Service).

فوائد استخدام البنية السداسية في النظام

ساهم اعتماد نمط (Hexagonal Architecture) في تحقيق عدة فوائد هندسية مهمة، منها:

1. تقليل الترابط بين المكونات (Low Coupling)

حيث لا تعتمد الخدمات بشكل مباشر على أدوات أو تقنيات محددة، مما يجعل النظام أكثر مرونة.

2. تحسين قابلية الاختبار (Testability)

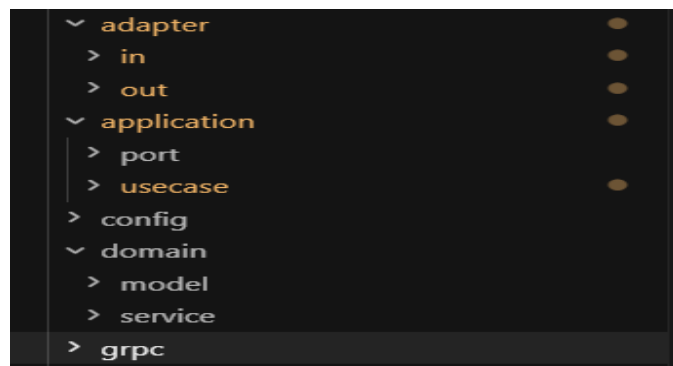
يمكن اختبار منطق الأعمال بشكل مستقل عن قواعد البيانات أو الخدمات الخارجية من خلال استخدام مكونات بديلة (Mock Adapters).

3. سهولة التطوير والصيانة (Maintainability)

يسمح الفصل الواضح بين المسؤوليات بإجراء التعديلات على جزء معين من النظام دون التأثير على باقي المكونات.

4. إمكانية استبدال التقنيات (Technology Independence)

يمكن تغيير قاعدة البيانات أو طريقة الاتصال أو أدوات التخزين دون إعادة تصميم منطق النظام الأساسي.



الشكل (2-2) Hex Architecture

من خلال تطبيق نمط البنية السداسية داخل الخدمات المصغرة، أصبح كل مكون في النظام يمتلك حدوداً واضحة ومسؤوليات محددة، مما ساعد على بناء نظام مصرفي يتميز بالمرونة وقابلية التطوير، ويحافظ في الوقت نفسه على تنظيم الكود البرمجي وسهولة إدارة العمليات المعقدة المرتبطة بالخدمات المالية.

4-2-2 آلية الاتصال بين الخدمات (Inter-Service Communication):

تعتبر آلية الاتصال بين الخدمات من العناصر الأساسية في الأنظمة المبنية على الخدمات المصغرة (Microservices Architecture)، حيث إن تقسيم النظام إلى خدمات مستقلة يتطلب وجود وسيلة فعالة وأمنة لتبادل البيانات وتنفيذ العمليات المشتركة بين هذه الخدمات. في النظام المصرفي المقترح، تم اعتماد نموذج اتصال يعتمد على الجمع بين الاتصال المتزامن (Synchronous Communication) باستخدام بروتوكول (gRPC)، والاتصال غير المتزامن (Asynchronous Communication) باستخدام نظام الرسائل (Apache Kafka) حسب طبيعة العملية المطلوبة.

تم تصميم آلية الاتصال بحيث تحافظ على استقلالية الخدمات، حيث لا تقوم أي خدمة بالوصول المباشر إلى قاعدة بيانات خدمة أخرى، وإنما يتم التفاعل بين الخدمات من خلال واجهات اتصال محددة (Service APIs)، مما يحافظ على مبدأ عزل البيانات (Data Isolation) ويمنع حدوث ترابط مباشر بين مكونات النظام.

أولاً: الاتصال الداخلي باستخدام (gRPC)

تم استخدام بروتوكول (gRPC) كوسيلة الاتصال الرئيسية بين الخدمات الداخلية للنظام. ويعتبر (gRPC) إطاراً حديثاً للاتصال بين التطبيقات يعتمد على استدعاء الإجراءات البعيدة (Remote Procedure Call)، حيث يسمح لخدمة معينة باستدعاء وظائف موجودة في خدمة أخرى كما لو كانت وظائف محلية.

تم اختيار (gRPC) بدلاً من الطرق التقليدية مثل (REST API) للتواصل الداخلي بين الخدمات بسبب عدة أسباب، أهمها:

الأداء العالي (High Performance)

يعتمد (gRPC) على بروتوكول (HTTP/2) ويدعم نقل البيانات بصيغة ثنائية (Binary Serialization) باستخدام (Protocol Buffers)، مما يقلل من حجم البيانات المنقولة ويزيد من سرعة الاتصال مقارنة بالبيانات النصية مثل (JSON).

تعريف عقود الاتصال (Service Contracts)

يتم تعريف واجهات الخدمات باستخدام ملفات (Protocol Buffers)، حيث تحتوي ملفات (proto) على تعريف العمليات والرسائل المتبادلة بين الخدمات.

مثال على ذلك:

- خدمة التحويلات (Transfers Service) تحتاج إلى التواصل مع :

- خدمة الحسابات (Accounts Service) للتحقق من صلاحية الخصم.

- خدمة التحقق من الهوية (KYC Service) لمعرفة حالة المستخدم.

- خدمة سجل المعاملات (Ledger Service) لتنفيذ الحركة المالية.

يتم تعريف هذه العمليات مسبقاً داخل ملفات (Proto)، مما يضمن وجود اتفاق واضح بين الخدمات حول البيانات المطلوبة والاستجابات المتوقعة.

توليد واجهات الاتصال (Generated Stubs)

بعد تعريف ملفات (Protocol Buffers)، يتم توليد الكود الخاص بالاتصال تلقائياً، مما يوفر طبقة تجريد (Abstraction Layer) بين الخدمات ويقلل من احتمالية حدوث أخطاء في عملية تبادل البيانات.

ثانياً: تمرير هوية المستخدم بين الخدمات (Authentication Context Propagation)

بما أن الخدمات الداخلية لا يتم الوصول إليها مباشرة من المستخدم، فقد تم تصميم آلية لتمرير معلومات هوية المستخدم وصلاحياته من بوابة النظام (Gateway Service) إلى الخدمات الداخلية.

تتم العملية بالشكل التالي:

1. يقوم المستخدم بإرسال طلب إلى (Gateway Service).
2. تقوم البوابة بالتحقق من صحة رمز الدخول (JWT).
3. يتم استخراج معلومات المستخدم من الرمز مثل :
 - معرف المستخدم (User ID).
 - الدور الوظيفي (Role).
4. يتم تمرير هذه المعلومات إلى استدعاءات (gRPC) باستخدام البيانات الوصفية للاتصال (gRPC Metadata).

يتم تمرير القيم الأساسية التالية:

- x-user-id
- x-role

حيث تستخدم الخدمات الداخلية هذه البيانات للتحقق من هوية المستخدم وتطبيق قواعد الصلاحيات قبل تنفيذ أي عملية.

ثالثاً: الاتصال غير المتزامن باستخدام (Apache Kafka)

بالإضافة إلى الاتصال المباشر بين الخدمات باستخدام (gRPC)، تم استخدام نظام الرسائل (Apache Kafka) لتنفيذ العمليات التي لا تتطلب استجابة فورية بين الخدمات.

يعتمد هذا الأسلوب على نموذج النشر والاشتراك (Publish/Subscribe Model)، حيث تقوم خدمة معينة بإرسال حدث (Event) إلى قناة محددة (Topic)، ثم تقوم الخدمات المهمة بهذا الحدث باستهلاكه ومعالجته.

تم استخدام (Kafka) في عدة أجزاء من النظام، منها:

إرسال إشعارات البريد الإلكتروني

عند حدوث عملية تتطلب إرسال بريد إلكتروني، مثل:

- تسجيل مستخدم جديد.
- تغيير حالة طلب التحقق من الهوية (KYC).
- إرسال إشعار متعلق بحساب مشترك.

لا تقوم الخدمة الأساسية بإرسال البريد مباشرة، وإنما تقوم بإرسال حدث إلى:

notification-email-requested

ثم تقوم خدمة الإشعارات (Notifications Service) باستهلاك الحدث وإرسال البريد باستخدام (SMTP).

هذا الأسلوب يقلل الاعتماد المباشر بين الخدمات ويحافظ على سرعة تنفيذ العمليات الأساسية.

أحداث التحقق من الهوية (KYC Events)

عند تغيير حالة طلب التحقق من الهوية، تقوم خدمة (KYC Service) بإرسال أحداث إلى:

kyc-events

وتحتوي هذه الأحداث على معلومات مثل:

- معرف المستخدم.
- معرف طلب التحقق.
- الحالة الجديدة للطلب.

- وقت التغيير.

يسمح هذا الأسلوب للخدمات الأخرى بالاستجابة لتغيرات الحالة دون الحاجة إلى اتصال مباشر مع خدمة التحقق من الهوية.

رابعاً: إدارة الأخطاء بين الخدمات (Error Handling)

بسبب اعتماد النظام على عدة خدمات مستقلة، فإن التعامل مع الأخطاء يعتبر جزءاً مهماً من تصميم الاتصال. تم اعتماد آلية موحدة لتحويل أخطاء (gRPC) إلى استجابات مفهومة من خلال (Gateway Service)، حيث تقوم البوابة بتحويل رموز الأخطاء الداخلية إلى رموز HTTP مناسبة.

مثال:

الجدول (1-2) gRPC and HTTP error codes

خطأ gRPC	استجابة HTTP
UNAUTHENTICATED	401 Unauthorized
PERMISSION_DENIED	403 Forbidden
NOT_FOUND	404 Not Found
ALREADY_EXISTS	409 Conflict
UNAVAILABLE	503 Service Unavailable

يسمح ذلك للتطبيقات الخارجية بالتعامل مع الأخطاء بطريقة موحدة دون الحاجة إلى معرفة تفاصيل البنية الداخلية للنظام.

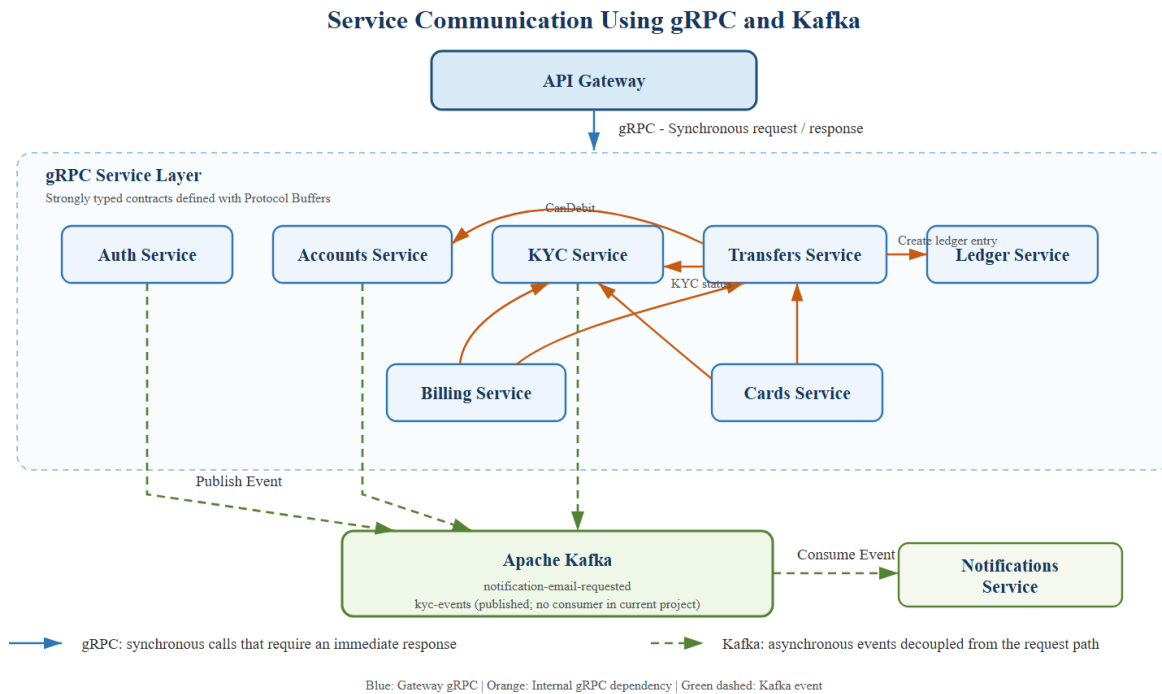
خامساً: مبدأ عدم الوصول المباشر للبيانات (Data Encapsulation)

من أهم مبادئ تصميم الاتصال في النظام هو منع الخدمات من الوصول المباشر إلى قواعد بيانات بعضها البعض. فعلى سبيل المثال:

- لا تقوم خدمة التحويلات (Transfers Service) بقراءة بيانات الحسابات مباشرة من قاعدة بيانات الحسابات.

- لا تقوم خدمة الفواتير (Billing Service) بتعديل بيانات المستخدمين مباشرة.

بدلاً من ذلك، يتم استخدام واجهات الاتصال المعرفة لكل خدمة، مما يحافظ على استقلالية البيانات ويمنع حدوث تعارضات أو اعتماد غير مرغوب بين المكونات.



الشكل (3-2) Service Communication

من خلال الجمع بين الاتصال المتزامن (gRPC) [5] والاتصال غير المتزامن (Kafka) [6]، يحقق النظام توازناً بين سرعة تنفيذ العمليات الحرجة مثل التحويلات المالية، ومرونة معالجة العمليات الثانوية مثل الإشعارات والأحداث الخلفية. كما يساهم هذا التصميم في بناء نظام مصرفي قابل للتوسع وقادر على التعامل مع عدد كبير من العمليات والخدمات بشكل منظم وموثوق.

5-2-2 بوابة النظام (API Gateway):

تعتبر بوابة النظام (API Gateway) نقطة الدخول الرئيسية للنظام المصرفي الإلكتروني، حيث تمثل الطبقة الوسيطة بين المستخدمين أو التطبيقات الخارجية وبين الخدمات الداخلية للنظام. تم تصميم هذه الطبقة بهدف توحيد طريقة الوصول إلى الخدمات المختلفة، وإخفاء تفاصيل البنية الداخلية القائمة على الخدمات المصغرة عن المستخدمين، بالإضافة إلى تطبيق مجموعة من الوظائف المشتركة مثل المصادقة (Authentication)، إدارة الصلاحيات (Authorization)، معالجة الأخطاء، وتحويل الطلبات بين البروتوكولات المختلفة.

في الأنظمة المبنية على الخدمات المصغرة، لا يتم عادةً السماح للتطبيقات الخارجية بالاتصال المباشر مع كل خدمة داخلية، لأن ذلك يؤدي إلى زيادة التعقيد وصعوبة إدارة الأمان. لذلك تم اعتماد نمط بوابة النظام (API Gateway Pattern)، حيث يتعامل العميل مع واجهة واحدة فقط، بينما تتولى البوابة مسؤولية توجيه الطلبات إلى الخدمة المناسبة داخل النظام.

في هذا المشروع تم تنفيذ (Gateway Service) باستخدام إطار العمل (Spring WebFlux)، حيث يوفر بيئة غير متزامنة (Reactive Programming Model) قادرة على التعامل مع عدد كبير من الطلبات المتزامنة بكفاءة عالية. تقوم البوابة باستقبال الطلبات عبر واجهات (HTTP API)، ثم تحويلها إلى استدعاءات داخلية باستخدام بروتوكول (gRPC) للتواصل مع الخدمات المختلفة.

دور بوابة النظام داخل المعمارية

تقوم (Gateway Service) بمجموعة من المسؤوليات الأساسية، وهي:

1. استقبال الطلبات الخارجية (External Request Handling)

تمثل البوابة الواجبة الوحيدة التي يتعامل معها المستخدم، حيث يتم إرسال جميع الطلبات من خلال مسارات (HTTP Endpoints) موحدة.

تشمل هذه الطلبات:

- عمليات المصادقة :

- تسجيل مستخدم جديد.

- تسجيل الدخول.

- تحديث الرموز الأمنية.

- عمليات الحسابات :

- إنشاء حساب.

- عرض الحسابات.

- إدارة الحسابات المشتركة.

- العمليات المالية :

- التحويلات.

- عرض سجل المعاملات.

- دفع الفواتير.

- عمليات التحقق :

- إرسال وثائق KYC.

- متابعة حالة التحقق.

بعد استقبال الطلب، تقوم البوابة بتحديد الخدمة المسؤولة عن معالجة العملية وإرسال الطلب إليها.

2. التحقق من هوية المستخدم (JWT Authentication)

تعتبر المصادقة من أهم وظائف بوابة النظام، حيث يتم التحقق من صحة الرموز الأمنية (JWT Tokens) [9] قبل السماح بالوصول إلى الخدمات الداخلية.

تعتمد العملية على الخطوات التالية:

1. يقوم المستخدم بتسجيل الدخول من خلال خدمة المصادقة (Auth Service).
 2. تقوم الخدمة بإنشاء رمز وصول (Access Token) باستخدام خوارزمية التوقيع (RS256).
 3. يرسل المستخدم هذا الرمز مع الطلبات اللاحقة.
 4. تقوم بوابة النظام بالتحقق من صحة التوقيع باستخدام المفتاح العام (RSA Public Key).
 5. في حال نجاح التحقق، يتم السماح بتمرير الطلب إلى الخدمة المطلوبة.
- يتم تخزين المفتاح العام المستخدم للتحقق من الرموز ضمن إعدادات النظام ويتم تحميله أثناء تشغيل الخدمة.

3. إدارة الصلاحيات (Authorization)

بعد التحقق من هوية المستخدم، تقوم البوابة بتطبيق قواعد الصلاحيات اعتماداً على الدور الموجود داخل الرمز الأمني (JWT Claim).

يدعم النظام نوعين رئيسيين من المستخدمين:

- المستخدم العادي (CUSTOMER).
- المسؤول (ADMIN).

يتم استخراج قيمة الدور من الحقل:

role

داخل رمز (JWT)، ثم تحويلها إلى صلاحية يمكن استخدامها للتحكم في الوصول.
مثال:

• المسارات الإدارية :

/api/users/admin/**

/api/accounts/admin/**

/api/transfers/admin/**

/api/kyc/admin/**

لا يمكن الوصول إليها إلا من قبل المستخدمين الذين يمتلكون صلاحية (ADMIN).

4. تحويل الاتصال من HTTP إلى gRPC

إحدى الوظائف الأساسية للبوابة هي العمل كطبقة تحويل بين بروتوكول الاتصال الخارجي والداخلي للنظام.
فعلى سبيل المثال، عند قيام المستخدم بتنفيذ عملية تحويل مالي:

1. يرسل التطبيق طلب POST /api/transfers.

2. تستقبل البوابة الطلب.

3. تتحقق من هوية المستخدم.

4. تحول الطلب من استدعاء (gRPC) إلى Transfers Service.

5. تستقبل النتيجة.

6. تعيد الاستجابة بصيغة HTTP مناسبة.

5. تمرير بيانات المستخدم إلى الخدمات الداخلية (gRPC Metadata Propagation)

بعد نجاح عملية المصادقة، تحتاج الخدمات الداخلية إلى معرفة هوية المستخدم الذي قام بالطلب. لذلك تقوم البوابة باستخراج البيانات من (JWT) وتميرها ضمن بيانات الاتصال الخاصة بـ (gRPC).

يتم تمرير المعلومات التالية:

- معرف المستخدم x-user-id

- دور المستخدم x-role

وتقوم الخدمات الداخلية باستخدام هذه المعلومات لتطبيق قواعد الوصول الخاصة بها.

مثال:

عند طلب مستخدم تنفيذ تحويل مالي، تقوم (Transfers Service) بالتحقق من معرف المستخدم المرسل ضمن الاتصال قبل السماح بتنفيذ العملية.

معالجة الأخطاء وتوحيد الاستجابات (Error Handling)

نظراً لأن النظام يحتوي على مجموعة من الخدمات المستقلة، فقد تنتج الأخطاء بصيغ مختلفة حسب الخدمة المسؤولة. لذلك تقوم بوابة النظام بتوحيد هذه الأخطاء وتحويلها إلى استجابات HTTP مفهومة للتطبيقات الخارجية.

تم استخدام معيار (RFC7807 Problem Details) لإرجاع معلومات الأخطاء، مما يوفر صيغة موحدة تحتوي على:

- نوع الخطأ.

- الحالة.

- وصف المشكلة.

- تفاصيل إضافية عند الحاجة.

مثال:

في حال محاولة الوصول إلى مورد غير موجود داخل إحدى الخدمات:

- تقوم الخدمة الداخلية بإرجاع :

NOT_FOUND

- تقوم البوابة بتحويلها إلى :

404 Not Found

فصل واجهات المستخدم عن الخدمات الداخلية (Decoupling)

من الفوائد المهمة لوجود بوابة النظام هو فصل التطبيقات الخارجية عن تفاصيل تنفيذ الخدمات الداخلية. حيث لا يحتاج التطبيق إلى معرفة:

- أسماء الخدمات الداخلية.
- منافذ الاتصال (Ports).
- بروتوكول gRPC.
- طريقة تخزين البيانات.

بل يتعامل فقط مع واجهات HTTP مستقرة توفرها البوابة.

هذا التصميم يسمح بتغيير البنية الداخلية للنظام أو إعادة توزيع الخدمات دون التأثير على التطبيقات التي تستخدم النظام.

فوائد استخدام API Gateway في النظام

ساهم اعتماد بوابة النظام في تحقيق عدة فوائد، منها:

1. تحسين الأمان (Security Enhancement)

منع الوصول المباشر إلى الخدمات الداخلية وتطبيق سياسات المصادقة في نقطة واحدة.

2. تبسيط الاتصال (Simplified Client Communication)

توفير واجهة موحدة للتعامل مع جميع وظائف النظام.

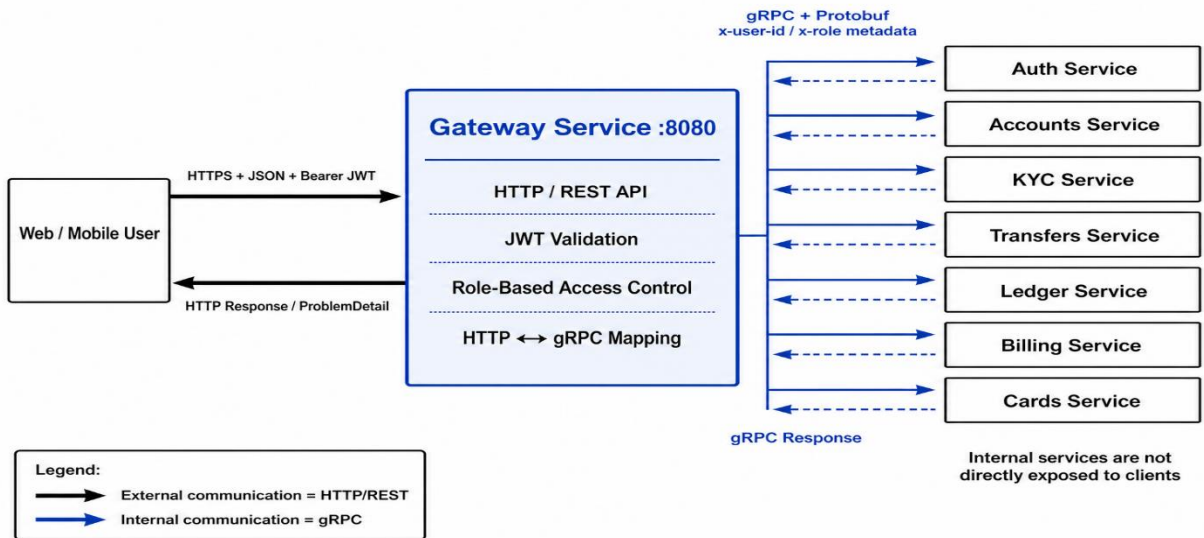
3. تحسين قابلية الصيانة (Maintainability)

إمكانية تعديل الخدمات الداخلية دون الحاجة إلى تغيير التطبيقات الخارجية.

4. التحكم المركزي بالطلبات (Centralized Request Management)

إدارة عمليات التحقق، تسجيل الطلبات، ومعالجة الأخطاء من خلال طبقة واحدة.

من خلال اعتماد (API Gateway)، أصبح النظام يمتلك نقطة دخول منظمة وآمنة تفصل بين العالم الخارجي والبنية الداخلية المعقدة للخدمات المصغرة، مما يساهم في تحسين الأمان، تقليل التعقيد، ورفع قابلية التوسع والإدارة في النظام المصرفي الإلكتروني



الشكل (4-2) Gateway Service

6-2-2 إدارة البيانات وقواعد البيانات (Data Management Architecture):

تعتبر إدارة البيانات من أهم الجوانب في تصميم الأنظمة المصرفية الحديثة، وذلك بسبب الطبيعة الحساسة للبيانات المالية وضرورة ضمان سلامتها، توفرها، واستقلالية التعامل معها. في الأنظمة التقليدية غالباً ما يتم الاعتماد على قاعدة بيانات مركزية مشتركة بين جميع مكونات النظام، إلا أن هذا الأسلوب قد يؤدي إلى زيادة الترابط بين المكونات وصعوبة تطوير النظام مع توسع حجمه.

بسبب اعتماد النظام الحالي على بنية الخدمات المصغرة (Microservices Architecture)، تم تصميم طبقة إدارة البيانات وفق مبدأ قاعدة بيانات مستقلة لكل خدمة (Database per Service Pattern)، بحيث تمتلك كل خدمة قاعدة بيانات خاصة بها ومسؤولة بشكل كامل عن تخزين وإدارة البيانات التابعة لها.

يسمح هذا التصميم بتحقيق استقلالية الخدمات، حيث لا تحتاج أي خدمة إلى الوصول المباشر إلى بيانات خدمة أخرى، وإنما يتم تبادل المعلومات من خلال واجهات الاتصال المعرفة بين الخدمات باستخدام بروتوكول (gRPC) أو الأحداث غير المتزامنة (Event Driven Communication) عبر (Apache Kafka).

قاعدة بيانات لكل خدمة (Database per Service)

تم اعتماد نمط (Database per Service) كجزء أساسي من تصميم النظام، حيث تمتلك كل خدمة مصغرة قاعدة بيانات منفصلة خاصة بها.

يعني ذلك أن:

- خدمة المصادقة (Auth Service) تدير بيانات المستخدمين والرموز الأمنية الخاصة بها.
- خدمة الحسابات (Accounts Service) تدير بيانات الحسابات البنكية والعضويات.
- خدمة السجل المالي (Ledger Service) تدير بيانات المعاملات والأرصدة.
- خدمة التحويلات (Transfers Service) تدير سجلات عمليات التحويل.

- خدمة التحقق من الهوية (KYC Service) تدير طلبات التحقق والوثائق.
 - خدمة الفواتير (Billing Service) تدير الفواتير والاشتراكات.
 - خدمة البطاقات (Cards Service) تدير بيانات البطاقات الافتراضية.
- بهذا الأسلوب تصبح كل خدمة مسؤولة عن دورة حياة بياناتها بالكامل، بدءاً من إنشاء الجداول، مروراً بعمليات القراءة والتعديل، وصولاً إلى عمليات الحذف أو التحديث.

أسباب استخدام Database per Service

1. تحقيق استقلالية الخدمات (Service Independence)

من أهم أهداف الخدمات المصغرة هو أن تكون كل خدمة مستقلة قدر الإمكان عن باقي الخدمات. في حال وجود قاعدة بيانات مشتركة، فإن أي تعديل على بنية البيانات قد يؤثر على عدة خدمات في نفس الوقت.

مثلاً:

إذا كانت جميع الخدمات تستخدم جدول المستخدمين نفسه، فإن تعديل هذا الجدول قد يتطلب تعديل:

- خدمة المصادقة.
 - خدمة الحسابات.
 - خدمة التحويلات.
 - خدمة الإشعارات.
- أما عند استخدام قاعدة بيانات مستقلة، فإن تعديل جدول داخل (Auth Service) لا يؤثر على باقي الخدمات.

2. تقليل الترابط بين الخدمات (Reducing Coupling)

اعتماد قاعدة بيانات مشتركة يؤدي إلى إنشاء ترابط قوي بين الخدمات، حيث تصبح الخدمات مرتبطة بطريقة تخزين البيانات وليس فقط بالوظائف التي تقدمها. في هذا النظام تم منع الوصول المباشر إلى قواعد بيانات الخدمات الأخرى. مثلاً:

لا تقوم خدمة التحويلات (Transfers Service) بقراءة جدول الحسابات مباشرة للتحقق من الرصيد، وإنما تقوم باستدعاء واجهة خدمة الحسابات. وبذلك تبقى مسؤولية إدارة بيانات الحسابات داخل خدمة الحسابات فقط.

3. تحسين قابلية التوسع (Scalability)

امتلاك كل خدمة لقاعدة بيانات خاصة بها يسمح بتوسيع الموارد حسب احتياجات كل خدمة. مثلاً:

- خدمة السجل المالي (Ledger Service) قد تحتاج إلى مساحة تخزين أكبر بسبب كثرة العمليات المالية.
- خدمة المصادقة (Auth Service) قد تحتاج إلى تحسين سرعة عمليات البحث عن المستخدمين. بدلاً من توسيع قاعدة بيانات واحدة ضخمة، يمكن تحسين كل قاعدة بيانات بشكل مستقل.

4. تحسين الأمان (Security)

في الأنظمة المصرفية تعتبر حماية البيانات أمراً أساسياً، لذلك فإن فصل قواعد البيانات يساعد على تقليل نطاق الوصول إلى المعلومات الحساسة.

مثلاً:

- بيانات كلمات المرور والرموز الأمنية تبقى داخل (auth_db).
 - وثائق التحقق الشخصية تبقى مرتبطة بخدمة (KYC Service).
 - بيانات العمليات المالية تبقى داخل (ledger_db).
- ولا تستطيع أي خدمة الوصول إلى هذه البيانات إلا من خلال الواجهات المخصصة لها.

نظام إدارة قواعد البيانات PostgreSQL

تم استخدام نظام إدارة قواعد البيانات العلائقية (PostgreSQL) [6] كقاعدة البيانات الأساسية للنظام. يعتبر (PostgreSQL) من أنظمة قواعد البيانات مفتوحة المصدر (Open Source Relational Database Management System)، ويتميز بالاعتمادية العالية ودعم العمليات المعقدة، مما يجعله مناسباً للأنظمة التي تتعامل مع بيانات مالية تحتاج إلى مستوى عالٍ من الدقة والموثوقية.

أسباب استخدام PostgreSQL

1. دعم المعاملات المالية (ACID Transactions)

تحتاج الأنظمة المصرفية إلى ضمان تنفيذ العمليات المالية بشكل صحيح، لذلك تم الاعتماد على خصائص (ACID):

- **Atomicity الذرية:** تنفيذ العملية بالكامل أو إلغاؤها بالكامل.
- **Consistency الاتساق:** المحافظة على صحة البيانات بعد كل عملية.

- **Isolation العزل:**

منع تعارض العمليات المتزامنة.

- **Durability الاستمرارية:**

حفظ البيانات بعد نجاح العملية حتى في حال حدوث مشاكل.

هذه الخصائص مهمة بشكل خاص في عمليات مثل التحويلات المالية، حيث يجب منع فقدان الأموال أو إنشاء أرصدة غير صحيحة.

2. دعم العلاقات والقيود (Constraints)

تم استخدام خصائص PostgreSQL لضمان صحة البيانات من خلال:

- المفاتيح الأساسية (Primary Keys).

- القيود الفريدة (Unique Constraints).

- العلاقات بين الجداول.

- الفهارس (Indexes).

مثال:

في خدمة المصادقة تم استخدام قيود لمنع وجود أكثر من جلسة فعالة للمستخدم نفسه من خلال:

`refresh_tokens_one_active_per_user`

3. الاعتمادية والأداء

يوفر PostgreSQL دعماً جيداً للأنظمة التي تحتوي على حجم كبير من العمليات، بالإضافة إلى إمكانية تحسين الأداء باستخدام:

- الفهارس.

- تحسين الاستعلامات.

- النسخ المتماثل (Replication).

وقد تم تجهيز النظام ليحتوي على قاعدة بيانات رئيسية (Primary Database) ونسخة احتياطية (Replica Database) ضمن بيئة التشغيل.

توزيع قواعد البيانات داخل النظام

تم إنشاء قاعدة بيانات مستقلة لكل خدمة من خدمات النظام، كما هو موضح:

الجدول (2-2) Services and DBs

الخدمة	قاعدة البيانات
Auth Service	auth_db
Accounts Service	accounts_db
Ledger Service	ledger_db
Transfers Service	transfers_db
KYC Service	kyc_db
Billing Service	billing_db
Cards Service	cards_db

كل قاعدة بيانات تحتوي فقط على الجداول الخاصة بالخدمة المالكة لها.

مثال:

auth_db

تحتوي على بيانات:

- المستخدمين.
- كلمات المرور المشفرة.
- رموز تحديث الجلسات (Refresh Tokens).
- رموز التحقق من البريد الإلكتروني.
- رموز إعادة تعيين كلمة المرور.

accounts_db

تحتوي على:

- الحسابات البنكية.
- أنواع الحسابات.
- عضويات الحسابات المشتركة.
- دعوات المشاركة.

ledger_db

تحتوي على:

- القیود المالية (Ledger Entries).
- التسجيلات المحاسبية (Ledger Postings).

- عناصر السجل المترابط (Account Ledger Items).
- الأرصدة.

transfers_db

تحتوي على:

- عمليات التحويل.
- حالة العملية.
- أسباب الرفض.
- معلومات الرسوم المالية.

kyc_db

تحتوي على:

- طلبات التحقق.
- حالات KYC.
- معلومات الوثائق.
- بيانات الملفات المخزنة.

billing_db

تحتوي على:

- الفواتير.

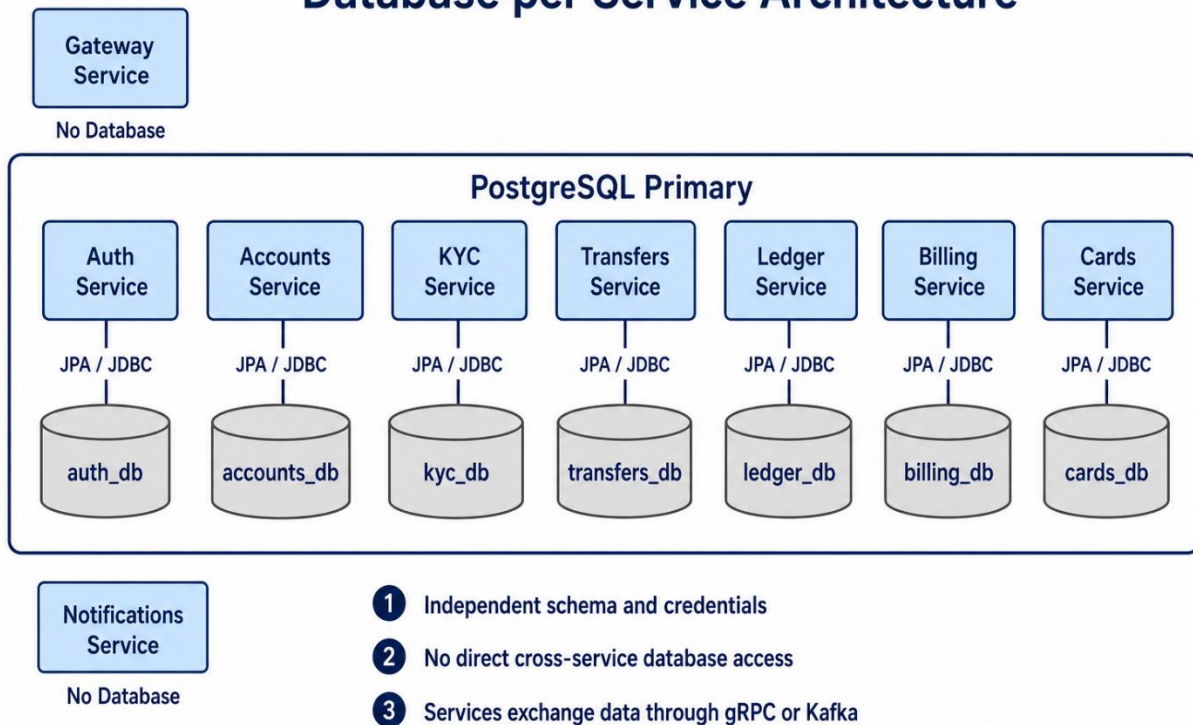
- الاشتراكات.
- عمليات الدفع.

cards_db

تحتوي على:

- البطاقات الافتراضية.
- عمليات الدفع باستخدام البطاقات.

Database per Service Architecture



الشكل (5-2) Database Architecture

إدارة تغييرات قاعدة البيانات باستخدام Flyway

لإدارة إنشاء وتحديث قواعد البيانات تم استخدام أداة [11] (Flyway Database Migration)، وهي أداة متخصصة في التحكم بإصدارات قواعد البيانات وتتبع التغييرات التي تحدث على المخطط البنوي (Database Schema).

بدلاً من إنشاء الجداول يدوياً أو تعديلها بشكل مباشر، يتم تعريف كل تغيير ضمن ملف Migration مستقل يحتوي على أوامر (SQL).

آلية عمل Flyway

عند تشغيل أي خدمة:

1. تقوم الخدمة بالاتصال بقاعدة البيانات الخاصة بها.
2. يتحقق (Flyway) من ملفات التحديث الموجودة.
3. يقارن الإصدارات المنفذة مع الإصدارات الجديدة.
4. يقوم بتنفيذ التعديلات غير المطبقة.
5. يسجل عملية التنفيذ داخل جدول خاص بالتتبع.

مثال على إصدارات قاعدة البيانات

داخل كل خدمة توجد ملفات مشابهة:

db/migration/

V1__init.sql

V2__account_scoped.sql

V3__user_blocked.sql

حيث يمثل:

- V1 الإصدار الأول لإنشاء الجداول الأساسية.
 - V2 إضافة تغييرات جديدة.
 - V3 تحديث إضافي على البنية.
- يضمن هذا الأسلوب أن جميع بيانات التشغيل تحتوي على نفس إصدار قاعدة البيانات.

فوائد استخدام Flyway

1. التحكم بإصدارات قاعدة البيانات

يسمح بمعرفة:

- ما هي التعديلات التي تمت.
- متى تم تطبيقها.
- ترتيب تنفيذها.

2. سهولة النشر (Deployment)

عند تشغيل النظام على خادم جديد، لا يحتاج المطور إلى إنشاء قواعد البيانات يدوياً، حيث تقوم الخدمات بتطبيق جميع التعديلات تلقائياً.

3. تقليل أخطاء التطوير

يساعد على منع اختلاف بنية قاعدة البيانات بين بيئات:

- التطوير (Development).
- الاختبار (Testing).
- الإنتاج (Production).

4. الحفاظ على استقرار البيانات

يتم تنفيذ التغييرات بطريقة منظمة دون فقدان البيانات الموجودة مسبقاً.

من خلال اعتماد (Database per Service) مع استخدام (PostgreSQL) و (Flyway Migration)، أصبح النظام يمتلك بنية بيانات مرنة وآمنة تتوافق مع مبادئ الخدمات المصغرة، حيث تحافظ كل خدمة على استقلالية بياناتها مع ضمان تنظيم عمليات التطوير والنشر والمحافظة على سلامة المعلومات المالية.

2-3 أدوات البنية التحتية (Infrastructure Tools):

إلى جانب تصميم النظام وتطوير خدماته البرمجية، كان من الضروري توفير بيئة تشغيل متكاملة تضمن سهولة إدارة الخدمات، واستقرار تشغيلها، وإمكانية نشرها على الخوادم بطريقة منظمة. ولهذا الغرض تم الاعتماد على مجموعة من أدوات البنية التحتية الحديثة التي تسهل تشغيل الخدمات المصغرة، وإدارة قواعد البيانات، وربط مكونات النظام المختلفة، بالإضافة إلى إدارة الإعدادات والملفات السرية الخاصة بالنظام. يوضح هذا القسم أهم الأدوات المستخدمة في تشغيل النظام وآلية توظيفها ضمن بيئة العمل.

2-3-1 الحاويات (Containers)

تم استخدام تقنية (Docker) لتشغيل جميع مكونات النظام داخل حاويات مستقلة (Containers)، حيث تمثل كل خدمة من خدمات النظام وحدة مستقلة تحتوي على جميع المتطلبات اللازمة لتشغيلها، بما في ذلك بيئة التشغيل والمكتبات والإعدادات الخاصة بها.

يساعد استخدام Docker على ضمان تشغيل الخدمات بنفس الطريقة في جميع البيئات، سواء أثناء التطوير أو الاختبار أو بعد نشر النظام على الخادم، مما يقلل من المشاكل الناتجة عن اختلاف بيئات التشغيل.

وفي هذا المشروع تم تشغيل كل خدمة من الخدمات المصغرة داخل حاوية مستقلة.

2-3-2 إدارة وتشغيل الخدمات (Docker Compose)

تم استخدام أداة (Docker Compose) لإدارة وتشغيل جميع مكونات النظام من خلال ملفات إعداد موحدة، مما يسهل تشغيل البيئة الكاملة للنظام باستخدام أمر واحد فقط.

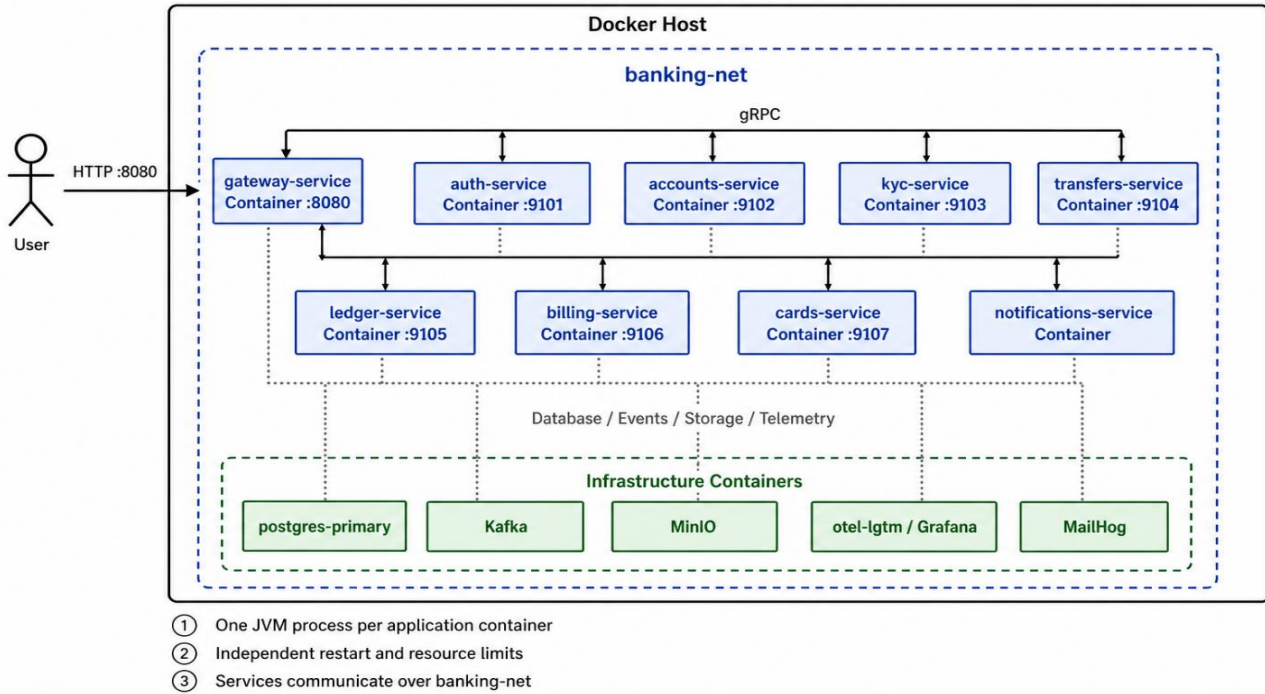
وقد تم تقسيم ملفات Compose إلى ملفين رئيسيين:

أولاً: ملف البنية التحتية (Infrastructure Compose)

يختص هذا الملف بتشغيل المكونات المشتركة التي تعتمد عليها جميع الخدمات، وتشمل:

- قاعدة البيانات (PostgreSQL).
- نظام الرسائل (Apache Kafka).
- نظام تخزين الملفات (MinIO) [12].
- أدوات المراقبة (OpenTelemetry LGTM Stack).

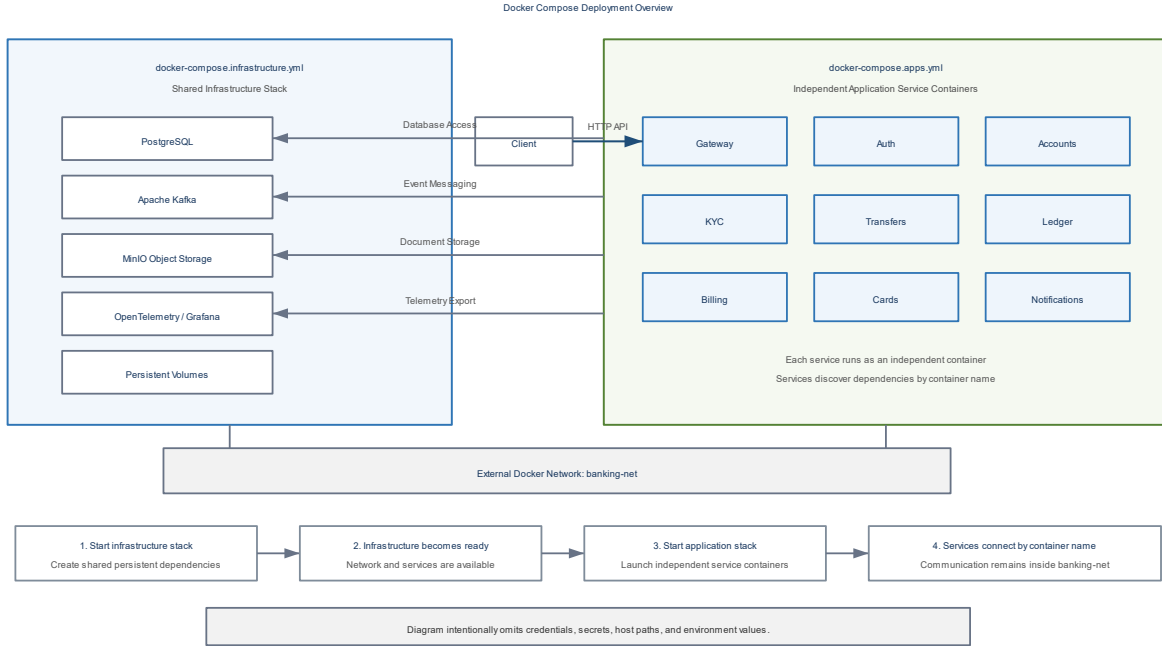
Docker Container Deployment



الشكل (6-2) Docker Container Deployment

ثانياً: ملف التطبيقات (Applications Compose)

يتولى هذا الملف تشغيل جميع الخدمات المصغرة الخاصة بالنظام، وربطها مع مكونات البنية التحتية ضمن شبكة داخلية واحدة، بالإضافة إلى تمرير متغيرات البيئة اللازمة لكل خدمة أثناء التشغيل. ساهم هذا التنظيم في الفصل بين البنية التحتية والخدمات التطبيقية، مما يجعل إدارة النظام أكثر مرونة وسهولة.



الشكل (7-2) Docker Compose Deployment Overview

3-3-2 إدارة الإعدادات والأسرار (Configuration and Secrets Management)

يعتمد النظام على آلية مرنة لإدارة الإعدادات من خلال متغيرات البيئة (Environment Variables) وملفات الإعدادات الخاصة بـ (Spring Boot) [4]، مما يسمح بتغيير إعدادات التشغيل دون الحاجة إلى تعديل الشيفرة المصدرية.

كما يتم تخزين البيانات الحساسة، مثل مفاتيح تشفير (RSA) المستخدمة لتوقيع والتحقق من رموز (JWT)، داخل مجلد مخصص للأسرار (Secrets) [8]، ويتم ربطه بالحاويات أثناء التشغيل باستخدام خاصية (Secret Mounting)، بحيث لا يتم تضمين هذه المفاتيح داخل الشيفرة البرمجية أو مستودع المشروع.

يساهم هذا الأسلوب في تحسين مستوى الأمان، كما يسهّل نقل النظام بين بيئات التشغيل المختلفة مع الاحتفاظ بنفس آلية الإعداد.

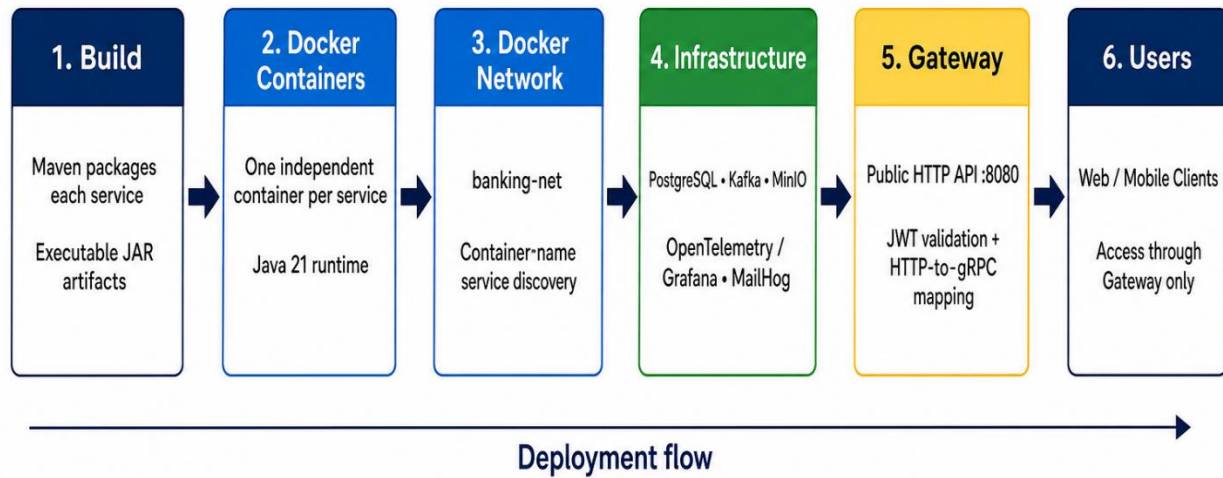
2-3-4 نشر النظام (Deployment)

تم تصميم النظام بحيث يمكن نشره بسهولة على أي خادم يدعم Docker [10]، حيث يتم أولاً بناء كل خدمة على شكل ملف تنفيذي (JAR) باستخدام أدوات بناء مشاريع Java، ثم يتم تشغيل هذه الملفات داخل الحاويات.

بعد ذلك يتم تشغيل مكونات البنية التحتية، مثل قواعد البيانات، ونظام الرسائل، وتخزين الملفات، ثم تشغيل جميع الخدمات المصغرة وربطها عبر شبكة Docker داخلية، مما يسمح لها بالتواصل فيما بينها باستخدام بروتوكول (gRPC)، بينما يتم استقبال طلبات المستخدمين من خلال خدمة (Gateway Service) التي تمثل نقطة الدخول الوحيدة للنظام.

يساعد هذا الأسلوب في تبسيط عملية النشر، وإعادة تشغيل النظام، وإجراء التحديثات المستقبلية مع الحفاظ على استقلالية كل خدمة

Deployment Stages



Credentials, secrets, host paths, and environment values are intentionally omitted.

الشكل (8-2) Deployment Stages

الفصل الثالث

(آليات الأمان والخوارزميات المستخدمة في النظام)

1-3 تمهيد:

يُعد الأمان من أهم المتطلبات الأساسية في تصميم الأنظمة المصرفية الرقمية، وذلك بسبب حساسية البيانات المالية التي يتم التعامل معها، إضافة إلى المخاطر المرتبطة بعمليات الاحتيال أو التلاعب بالمعاملات. لذلك، تم تصميم النظام المصرفي الإلكتروني في هذا المشروع اعتماداً على مجموعة متكاملة من الخوارزميات والآليات الأمنية التي تعمل معاً لضمان حماية البيانات وتحقيق الموثوقية.

يرتكز النظام على بنية (Microservices Architecture)، حيث يتم توزيع الوظائف على خدمات مستقلة، ويتم التواصل بينها باستخدام بروتوكول (gRPC)، بينما يتم التعامل مع المستخدمين عبر واجهة (HTTP API Gateway) في هذا السياق، يتم تمرير هوية المستخدم بين الخدمات باستخدام (Metadata) مثل (x-user-id) و (x-role)، مما يضمن تطبيق سياسات الأمان بشكل موحد على جميع مكونات النظام.

يتناول هذا الفصل شرح أهم الخوارزميات المستخدمة لضمان سلامة البيانات، مثل خوارزمية ربط التجزئة (Hash Chaining) المعتمدة في سجل المعاملات، بالإضافة إلى خوارزمية (SHA-256) المستخدمة في توليد القيم التجزئية. كما سيتم شرح آليات حماية كلمات المرور باستخدام (BCrypt)، وآلية إدارة الجلسات باستخدام (JWT) الموقعة بخوارزمية (RS256).

إضافة إلى ذلك، سيتم التطرق إلى الأنظمة التنظيمية والأمنية المطبقة داخل النظام مثل قواعد مكافحة غسل الأموال (AML)، وآلية التحقق من هوية المستخدمين (KYC)، والتي تمنع تنفيذ العمليات المالية قبل التأكد من هوية المستخدم. كما سيتم شرح مفاهيم مهمة مثل المعاملات الذرية (Atomic Transactions) التي تضمن تنفيذ العمليات بشكل كامل أو عدم تنفيذها، وآلية (Idempotency) التي تمنع تكرار العمليات المالية.

وأخيراً، سيتم استعراض مجموعة من الآليات الأمنية الإضافية مثل إدارة الصلاحيات (RBAC)، وتجميد الحسابات، ومنع الوصول غير المصرح به، وذلك بهدف بناء نظام مصرفي رقمي يتمتع بدرجة عالية من الأمان والاستقرار.

3-2 خوارزمية ربط التجزئة (Hash Chaining):

تُعتبر خوارزمية ربط التجزئة (Hash Chaining) من أهم التقنيات المستخدمة في هذا النظام لضمان سلامة البيانات المالية ومنع أي محاولة تلاعب في سجل المعاملات. تم تطبيق هذه الآلية داخل خدمة سجل المعاملات (Ledger Service)، والتي تُعد المكوّن المسؤول عن تسجيل جميع العمليات المالية بشكل دقيق وموثوق.

مفهوم خوارزمية Hash Chaining

تعتمد هذه الخوارزمية على إنشاء سلسلة مترابطة من العمليات المالية لكل حساب بنكي، بحيث يكون كل عنصر في هذه السلسلة مرتبطاً بالعنصر الذي يسبقه من خلال قيمة تجزئة (Hash) يتكوّن كل عنصر من عنصرين أساسيين:

- قيمة التجزئة الخاصة بالعنصر السابق (prevHash)

- قيمة التجزئة الخاصة بالعنصر الحالي (itemHash)

وبذلك، فإن كل عملية تعتمد بشكل مباشر على العملية السابقة، مما يؤدي إلى تكوين سلسلة مترابطة يصعب تعديل أي جزء منها دون التأثير على باقي السلسلة.

آلية التطبيق في النظام

في النظام الحالي، يتم إنشاء سلسلة مستقلة لكل حساب بنكي (Account)، حيث يتم تسجيل العمليات المالية بشكل تسلسلي باستخدام رقم ترتيب (Sequence Number) عند تنفيذ عملية مالية جديدة، يتم:

1. قراءة قيمة التجزئة الخاصة بآخر عنصر في السلسلة (Chain Head)

2. استخدام هذه القيمة كـ (prevHash) للعملية الجديدة.

3. إنشاء تمثيل موحد (Canonical Representation) يحتوي على جميع بيانات العملية مثل:

- معرف الحساب (accountId).
- رقم التسلسل (seq).
- معرف العملية (entryId).
- وقت التنفيذ (timestamp).
- قيمة المبلغ (signedAmountCents).
- الحساب المقابل (counterparty).
- معرف العملية (idempotencyKey).
- وصف العملية (description).

4. حساب قيمة التجزئة الجديدة (itemHash) باستخدام خوارزمية (SHA-256).

5. تخزين العنصر الجديد وربطه بالعنصر السابق.

التحقق من سلامة السلسلة

يوفر النظام آلية للتحقق من صحة السلسلة (Chain Verification)، حيث يتم إعادة حساب قيم التجزئة لجميع العناصر بشكل تسلسلي، والتأكد من أن:

- قيمة (prevHash) لكل عنصر تطابق قيمة (itemHash) للعنصر السابق.
- لا يوجد أي انقطاع أو تعديل في التسلسل.

في حال وجود أي اختلاف، يتم اعتبار السلسلة غير سليمة، مما يدل على وجود محاولة تلاعب بالبيانات.

أهمية هذه الخوارزمية

تكمُن أهمية (Hash Chaining) في تحقيق الخصائص التالية:

- سلامة البيانات (Data Integrity): أي تعديل في البيانات يؤدي إلى تغيير في قيمة التجزئة.

- قابلية التدقيق (Auditability): يمكن تتبع جميع العمليات والتحقق منها بسهولة.
- مقاومة التلاعب (Tamper Resistance): يصعب تعديل عملية واحدة دون التأثير على كامل السلسلة.

العلاقة مع تقنيات Blockchain

تتشابه هذه الآلية مع المفهوم المستخدم في (Blockchain)، حيث يتم ربط الكتل باستخدام قيم تجزئة. ومع ذلك، فإن النظام الحالي لا يعتمد على شبكة لامركزية أو آليات مثل (Proof of Work)، وإنما يستخدم هذا المفهوم كحل داخلي لتحسين موثوقية سجل المعاملات.

وبذلك، توفر خوارزمية (Hash Chaining) طبقة قوية من الحماية لسجل العمليات المالية، مما يعزز ثقة المستخدمين بالنظام ويضمن سلامة البيانات على المدى الطويل.

3-3 خوارزمية SHA-256:

تُستخدم خوارزمية (SHA-256) في النظام كأحد المكونات الأساسية المستخدمة لضمان سلامة البيانات، وبشكل خاص ضمن خدمة سجل المعاملات (Ledger Service) وتُعد (SHA-256) إحدى خوارزميات التجزئة (Cryptographic Hash Functions) التابعة لعائلة (SHA-2)، حيث تقوم بتحويل مجموعة من البيانات ذات أي حجم إلى قيمة تجزئة ثابتة الطول تبلغ (256-bit).

تختلف خوارزمية التجزئة عن عملية التشفير التقليدية (Encryption)، إذ لا تهدف إلى إخفاء البيانات بحيث يمكن استعادتها لاحقاً باستخدام مفتاح، وإنما تهدف إلى إنتاج بصمة رقمية (Digital Fingerprint) للبيانات. فإذا تم تغيير أي جزء من البيانات الأصلية، حتى ولو كان تغييراً بسيطاً جداً، فإن قيمة التجزئة الناتجة ستتغير بشكل كبير.

آلية استخدام SHA-256 في النظام

يتم استخدام (SHA-256) ضمن آلية ربط التجزئة (Hash Chaining) في خدمة (Ledger Service) قبل حساب قيمة التجزئة للعملية، يتم إنشاء تمثيل موحد للبيانات (Canonical String)، بحيث يتم ترتيب وتجميع المعلومات المهمة المتعلقة بعنصر السجل.

تشمل هذه البيانات مجموعة من المعلومات مثل:

- معرف الحساب (Account ID).
- رقم التسلسل (Sequence Number).
- معرف عنصر السجل (Item ID).
- معرف العملية المالية (Entry ID).
- وقت إنشاء العملية (Created At).
- قيمة المبلغ واتجاهه (Signed Amount).
- الحساب المقابل للعملية (Counterparty).
- مفتاح منع التكرار (Idempotency Key).
- معلومات الحساب المرسل والمستقبل.
- قيمة التحويل.
- وصف العملية المالية.

بعد تجهيز هذه البيانات، يتم تمريرها إلى خوارزمية (SHA-256) للحصول على قيمة (itemHash)، ويتم تخزين هذه القيمة مع عنصر السجل في قاعدة البيانات.

كما يتم تضمين قيمة التجزئة السابقة (prevHash) ضمن البيانات التي تدخل في عملية إنشاء التجزئة الجديدة، مما يؤدي إلى ربط العناصر مع بعضها وتكوين سلسلة متتابعة من القيم التجزئية.

حساسية التجزئة تجاه التغييرات

من الخصائص المهمة لخوارزمية (SHA-256) ما يُعرف بتأثير الانتشار (Avalanche Effect)، حيث إن تغيير قيمة واحدة فقط في البيانات المدخلة يمكن أن يؤدي إلى تغيير كبير في قيمة التجزئة الناتجة.

على سبيل المثال، إذا تم تغيير مبلغ معاملة مالية أو وقت تنفيذها أو الحساب المرتبط بها، فإن قيمة (itemHash) الأصلية لن تعود متطابقة مع القيمة التي يمكن حسابها من البيانات الحالية. وبالتالي يستطيع النظام اكتشاف الاختلاف أثناء عملية التحقق من سلامة السجل.

استخدامها في التحقق من سجل المعاملات

يوفر النظام وظيفة للتحقق من سلامة سلسلة الحساب (Verify Account Chain)، حيث تقوم الخدمة بقراءة عناصر السجل بالترتيب وإعادة حساب قيمة (SHA-256) لكل عنصر، ثم مقارنتها بالقيمة المخزنة في قاعدة البيانات.

بالإضافة إلى ذلك، يتم التأكد من أن قيمة (prevHash) الموجودة في كل عنصر تطابق قيمة (itemHash) الخاصة بالعنصر السابق. وبذلك يتم التحقق من جانبين أساسيين:

1. صحة محتوى كل عنصر في السجل.

2. صحة الترابط بين عناصر السلسلة.

في حال تم تعديل أي بيانات بشكل مباشر داخل قاعدة البيانات، فإن عملية إعادة الحساب ستنتج قيمة مختلفة، وبالتالي يمكن اكتشاف عدم سلامة السجل.

أهمية SHA-256 في النظام المصرفي

يساهم استخدام (SHA-256) في تعزيز خاصية سلامة البيانات (Data Integrity)، وهي من الخصائص الأساسية في الأنظمة المالية التي يجب أن تحافظ على دقة العمليات وسجلها التاريخي.

ومن خلال دمج (SHA-256) مع (Hash Chaining)، لا يعتمد النظام على قيمة تجزئة منفردة لكل عملية فقط، وإنما يجعل كل عملية مرتبطة بالعملية السابقة. لذلك فإن محاولة تعديل عملية قديمة لا تؤثر على تلك العملية فقط، بل تؤدي إلى عدم تطابق السلسلة ابتداءً من ذلك العنصر.

ومن المهم توضيح أن استخدام (SHA-256) في هذا المشروع لا يعني أن البيانات المالية نفسها أصبحت مشفرة (Encrypted). فالخوارزمية مستخدمة هنا بهدف إنشاء بصمات للبيانات والتحقق من سلامتها، وليس بهدف تشفير البيانات وإخفائها عن المستخدمين المصرح لهم.

وبذلك توفر خوارزمية (SHA-256)، إلى جانب آلية (Hash Chaining)، وسيلة فعالة للتحقق من سلامة سجل المعاملات واكتشاف أي تغيير غير مصرح به في البيانات المالية.

4-3 خوارزمية BCrypt:

تُستخدم خوارزمية BCrypt في النظام كأحد أهم مكونات الأمان المرتبطة بحماية بيانات المصادقة، حيث يتم الاعتماد عليها لتخزين كلمات مرور المستخدمين والقيم السرية بطريقة آمنة داخل Authentication Service. تقوم هذه الخوارزمية على مبدأ التجزئة أحادية الاتجاه (One-Way Hashing)، مما يعني أن القيمة الأصلية (مثل كلمة المرور) لا يمكن استرجاعها بعد تحويلها إلى قيمة مجزأة، وهو ما يمنع كشف البيانات الحساسة حتى في حال الوصول إلى قاعدة البيانات.

عند قيام المستخدم بعملية التسجيل (Register)، يتم إرسال كلمة المرور إلى خدمة المصادقة، حيث يتم تمريرها إلى مكون يستخدم BCryptPasswordEncoder ضمن بيئة Spring Security تقوم الخوارزمية بإنتاج قيمة مجزأة يتم تخزينها في قاعدة البيانات الخاصة بالخدمة (auth_db)، دون الاحتفاظ بأي نسخة من كلمة المرور الأصلية. وهذا يضمن أن النظام لا يحتوي في أي مرحلة على بيانات حساسة بصيغتها الصريحة.

عند تسجيل الدخول (Login)، لا يتم فك التجزئة أو استرجاع كلمة المرور، بل يتم استخدام آلية مقارنة (Matching) حيث يتم تطبيق BCrypt على كلمة المرور المدخلة ومقارنتها مع القيمة المخزنة. في حال التطابق، يتم اعتبار بيانات المستخدم صحيحة، ويتم بعدها إصدار رموز المصادقة مثل JWT Access Token و Refresh Token.

من الخصائص المهمة في BCrypt أنها تعتمد على إضافة قيمة عشوائية تُعرف باسم Salt داخل عملية التجزئة. يؤدي ذلك إلى إنتاج قيمة مختلفة حتى في حال استخدام نفس كلمة المرور، مما يمنع استخدام هجمات مثل (Rainbow Table Attacks). بالإضافة إلى ذلك، تم تصميم BCrypt لتكون خوارزمية

بطيئة نسبياً من حيث الحسابات، وهو ما يجعل هجمات التخمين (Brute Force Attacks) أكثر صعوبة وتكلفة من الناحية الزمنية.

ولا يقتصر استخدام BCrypt في النظام على كلمات المرور فقط، بل يتم استخدامها أيضاً لحماية القيم الحساسة الأخرى داخل auth-service، مثل:

- أسرار Refresh Tokens

- رموز التحقق من البريد الإلكتروني (Email Verification Codes)

- رموز إعادة تعيين كلمة المرور (Password Reset Tokens)

في هذه الحالات، يتم تخزين القيم بعد تجزئتها فقط، بينما يتم استخدام القيم الأصلية مرة واحدة عند إنشائها. كما يعتمد النظام في بعض هذه الحالات على نموذج تقسيم الرمز إلى جزأين (id.secret)، حيث يتم تخزين المعرف بشكل مباشر، بينما يتم تخزين تجزئة الجزء السري فقط، مما يزيد من مستوى الحماية.

وتتكامل خوارزمية BCrypt مع بقية مكونات الأمان في النظام. فبعد التحقق من كلمة المرور باستخدام BCrypt، يتم إصدار رمز JWT موقع باستخدام RS256، والذي يُستخدم لاحقاً لإثبات هوية المستخدم عند التواصل مع باقي الخدمات عبر API Gateway وبهذا تكون BCrypt مسؤولة عن حماية بيانات الاعتماد، بينما تتولى JWT إدارة هوية المستخدم وصلاحياته أثناء تنفيذ العمليات.

وبالتالي، فإن استخدام BCrypt يوفر طبقة قوية من الحماية لبيانات المستخدمين، ويمنع تخزين المعلومات الحساسة بصيغتها الأصلية، كما يساهم في مقاومة العديد من الهجمات الشائعة على أنظمة المصادقة. ويُعد هذا المكون جزءاً أساسياً من منظومة الأمان في النظام، خاصة عند دمجها مع آليات أخرى مثل JWT و(RS256)، و (RBAC) ضمن بيئة تعتمد على Microservices Architecture.

3-5 توقيع JWT باستخدام RS256:

يعتمد النظام على استخدام رموز (JSON Web Tokens) JWT كآلية أساسية لإدارة هوية المستخدمين بعد نجاح عملية تسجيل الدخول، حيث يتم إصدار هذه الرموز من خلال Authentication Service واستخدامها

للوصول إلى باقي الخدمات عبر API Gateway يتم تأمين هذه الرموز باستخدام خوارزمية توقيع رقمية من نوع RS256، والتي تعتمد على التشفير غير المتماثل (Asymmetric Cryptography).

تعتمد آلية RS256 على استخدام زوج من المفاتيح: مفتاح خاص (Private Key) يتم استخدامه لتوقيع الرمز داخل خدمة المصادقة، ومفتاح عام (Public Key) يتم استخدامه من قبل Gateway Service للتحقق من صحة التوقيع. يتم تخزين هذه المفاتيح ضمن النظام باستخدام آلية Secrets Management، حيث يتم تحميلها من ملفات خارجية عند تشغيل النظام، مما يمنع تضمينها داخل الكود مباشرة.

عند تسجيل الدخول بنجاح، تقوم خدمة المصادقة بإنشاء رمز JWT يحتوي على مجموعة من المعلومات (Claims) المهمة، مثل:

- معرف المستخدم (sub) وهو UUID
- دور المستخدم (role) مثل USER أو ADMIN
- وقت الإصدار (iat)
- وقت انتهاء الصلاحية (exp)

بعد إنشاء هذه البيانات، يتم توقيع الرمز باستخدام المفتاح الخاص وخوارزمية RS256، مما يضمن عدم إمكانية تعديل محتوى الرمز دون إبطال التوقيع. يتم إرسال هذا الرمز إلى المستخدم ليقوم بإرساله مع كل طلب إلى النظام.

عند وصول الطلب إلى API Gateway، يتم التحقق أولاً من صحة الرمز باستخدام المفتاح العام. في حال كان التوقيع صحيحاً والرمز لم تنتهِ صلاحيته، يقوم النظام باستخراج معلومات المستخدم وإضافتها إلى الطلب على شكل gRPC Metadata Headers مثل:

- x-user-id
- x-role

ثم يتم تمرير الطلب إلى الخدمات الداخلية عبر بروتوكول gRPC مع الحفاظ على هوية المستخدم. أما في حال فشل التحقق، يتم رفض الطلب مباشرة دون الوصول إلى الخدمات الداخلية.

ومن الجوانب المهمة في هذا التصميم أن رموز JWT قصيرة العمر (حوالي 15 دقيقة)، مما يقلل من خطر استخدامها في حال تم تسريبها. ولتجديد الجلسة، يعتمد النظام على Refresh Tokens، والتي يتم تخزينها داخل قاعدة البيانات ويمكن إلغاؤها (Revocation) عند تسجيل الخروج أو عند تغيير كلمة المرور، مما يوفر طبقة إضافية من التحكم والأمان.

توفر هذه الآلية عدة مزايا، منها:

- عدم الحاجة لتخزين جلسات المستخدمين على الخادم (Stateless Authentication)
 - إمكانية التحقق من الرمز في أي خدمة باستخدام المفتاح العام فقط
 - دعم التوسع ضمن بنية Microservices Architecture
 - فصل واضح بين المصادقة (Authentication) والتفويض (Authorization)
- وبالتالي، فإن استخدام JWT مع RS256 يشكّل العمود الفقري لنظام إدارة الهوية في المشروع، حيث يضمن سلامة البيانات داخل الرمز، ويمنع التلاعب بها، ويوفر طريقة فعالة وآمنة لنقل هوية المستخدم بين الخدمات المختلفة داخل النظام.

3-6 قواعد مكافحة غسل الأموال (AML):

يعتمد النظام على مجموعة من القواعد البسيطة لمكافحة غسل الأموال (AML – Anti-Money Laundering) ضمن Transfers Service، حيث يتم تطبيق هذه القواعد قبل تنفيذ أي عملية تحويل مالي. الهدف من هذه الآلية هو منع تنفيذ العمليات المشبوهة أو غير المنطقية، وتعزيز موثوقية النظام من خلال فرض قيود على سلوك المستخدمين أثناء إجراء التحويلات.

تعتمد آلية AML في المشروع على نموذج Rule-Based System، أي مجموعة من القواعد المحددة مسبقاً التي يتم التحقق منها عند كل عملية تحويل. ومن أهم هذه القواعد:

- التحقق من أن قيمة التحويل أكبر من الصفر، حيث يتم رفض أي عملية بقيمة غير صحيحة.

- تحديد حد أعلى لقيمة التحويل الواحد (Max Single Transfer Amount)، بحيث يتم منع العمليات التي تتجاوز هذا الحد.

- التحكم في عدد العمليات خلال فترة زمنية محددة (Velocity Control) ، حيث يتم حساب عدد التحويلات التي قام بها المستخدم خلال نافذة زمنية معينة (مثلاً 10 دقائق)، ومنع تنفيذ العملية في حال تجاوز العدد المسموح.

عند تنفيذ عملية تحويل، تقوم خدمة التحويلات باستدعاء وحدة AML للتحقق من هذه القواعد. في حال اكتشاف أي مخالفة، لا يتم تنفيذ العملية مباشرة، بل يتم تسجيلها داخل النظام بحالة BLOCKED مع توضيح سبب الرفض. هذا التصميم يسمح بتتبع العمليات المشبوهة وتحليلها لاحقاً بدلاً من تجاهلها بشكل كامل.

تتكامل آلية AML مع باقي مكونات النظام، حيث يتم تطبيقها بعد التحقق من شروط أخرى مثل:

- حالة التحقق من الهوية (KYC Status)

- صلاحية الحساب من خلال Accounts Service

- توفر الرصيد الكافي

وبذلك تكون AML جزءاً من سلسلة من الفحوصات التي تضمن أن العملية المالية سليمة قبل تنفيذها فعلياً في Ledger Service.

من المهم الإشارة إلى أن هذه الآلية تُعتبر نموذجاً مبسطاً لأغراض أكاديمية، حيث لا تتضمن تكاملات متقدمة مثل قوائم العقوبات (Sanctions Lists) أو تحليل سلوك المستخدم باستخدام تقنيات الذكاء الاصطناعي. ومع ذلك، فإنها توضح المفهوم الأساسي لكيفية تطبيق قواعد رقابية على العمليات المالية داخل نظام مصرفي رقمي.

وبالتالي، تساهم قواعد AML في تعزيز أمان النظام من خلال تقليل احتمالية إساءة استخدامه في عمليات غير قانونية، كما تضيف طبقة إضافية من التحقق قبل تنفيذ المعاملات، مما يدعم موثوقية النظام وسلامة عملياته المالية.

3-7 آلية اعرف عميلك (KYC):

تُعد آلية اعرف عميلك (KYC – Know Your Customer) من الركائز الأساسية في الأنظمة المصرفية الحديثة، حيث تهدف إلى التحقق من هوية المستخدمين قبل السماح لهم باستخدام الخدمات المالية الحساسة. في هذا المشروع، تم تنفيذ نظام KYC ضمن KYC Service كخدمة مستقلة ضمن بنية Microservices Architecture، ويتم استخدامها كشرط أساسي قبل تنفيذ العمليات المالية مثل التحويلات أو إنشاء البطاقات أو الاشتراكات.

تمر عملية KYC بعدة مراحل تبدأ بقيام المستخدم برفع الوثائق المطلوبة من خلال واجهة النظام، مثل صورة الهوية وصورة شخصية. يتم إنشاء طلب تحقق (KYC Application) خاص بالمستخدم، ويتم ربطه بمجموعة من الوثائق (Documents) التي يتم تخزينها خارج قاعدة البيانات باستخدام نظام تخزين كائنات (Object Storage) وهو MinIO يتم رفع الملفات باستخدام روابط مؤقتة (Presigned URLs) تتيح للمستخدم رفع الملف مباشرة إلى التخزين دون المرور عبر الخادم، مما يحسن الأداء ويزيد من أمان العملية. يتم تخزين معلومات الوثائق داخل قاعدة بيانات خدمة KYC، بالإضافة إلى بصمة رقمية (Hash) مثل SHA-256 يتم توليدها من الملف لضمان سلامته. تتيح هذه البصمة التحقق من أن الملف لم يتم تعديله لاحقاً، مما يضيف طبقة إضافية من الأمان لبيانات المستخدم الحساسة.

بعد رفع الوثائق، يقوم المستخدم بإرسال الطلب (Submit KYC)، ليتم تغيير حالة الطلب إلى PENDING في هذه المرحلة، يقوم المسؤول (ADMIN) بمراجعة الطلب من خلال واجهة الإدارة، ويمكنه قبول الطلب أو رفضه. في حال القبول، يتم تحديث الحالة إلى APPROVED، وفي حال الرفض يتم تعيين الحالة إلى REJECTED مع إمكانية إضافة سبب الرفض.

تُستخدم حالة KYC كشرط أساسي في عدة خدمات داخل النظام. على سبيل المثال:

- لا يمكن تنفيذ عمليات التحويل في Transfers Service إلا إذا كانت الحالة APPROVED

- لا يمكن إنشاء بطاقات في Cards Service

• لا يمكن تنفيذ عمليات الدفع أو الاشتراكات في Billing Service

في حال لم يتم اعتماد المستخدم، يتم منع العملية وتسجيلها بحالة مناسبة مثل BLOCKED مع توضيح السبب.

كما تعتمد الخدمة على نظام الأحداث (Event-Driven Architecture) باستخدام Apache Kafka، حيث يتم إرسال أحداث عند تغيير حالة KYC مثل APPROVED أو REJECTED، ويتم استخدامها لإرسال إشعارات للمستخدم عبر Notifications Service.

وبذلك توفر آلية KYC طبقة تحقق أساسية تضمن أن المستخدمين داخل النظام معروفون وموثقون، مما يقلل من مخاطر الاحتيال ويعزز الامتثال للمعايير التنظيمية. كما تتكامل هذه الآلية مع باقي مكونات النظام مثل AML و JWT لتوفير بيئة مصرفية آمنة وموثوقة.

3-8 المعاملات الذرية (Atomic Transactions):

تُعتبر المعاملات الذرية (Atomic Transactions) من أهم المبادئ في الأنظمة المالية، حيث تضمن أن تنفيذ العملية يتم بشكل كامل أو لا يتم إطلاقاً، دون وجود حالة وسطية قد تؤدي إلى عدم اتساق البيانات. في هذا المشروع، تم تطبيق هذا المفهوم بشكل أساسي داخل Ledger Service لضمان سلامة العمليات المالية. عند تنفيذ أي عملية تحويل، لا يتم تعديل رصيد حساب واحد فقط، بل يتم تسجيل العملية وفق مبدأ القيد المزدوج (Double-Entry Accounting)، بحيث يتم إنشاء:

• قيد خصم (Debit) من حساب المرسل

• قيد إضافة (Credit) إلى حساب المستلم

يتم تنفيذ هذه العمليات ضمن معاملة واحدة داخل قاعدة البيانات باستخدام آلية @Transactional في Spring Boot هذا يعني أن جميع العمليات المرتبطة بالتحويل (إنشاء السجل، إضافة القيود، تحديث الأرصدة، تحديث سلسلة السجل) يتم تنفيذها كوحدة واحدة. في حال حدوث أي خطأ أثناء التنفيذ، يتم التراجع (Rollback) عن جميع التغييرات، بحيث لا يتم تطبيق أي جزء من العملية.

ومن الخصائص المهمة في هذا التصميم أن مجموع القيود (Postings) يجب أن يساوي صفراً، أي أن:

مجموع القيم المخصومة = مجموع القيم المضافة

وهذا يضمن الحفاظ على التوازن المالي داخل النظام ومنع إنشاء أو فقدان أموال بشكل غير منطقي.

تتكامل المعاملات الذرية أيضاً مع Transfers Service، حيث تقوم هذه الخدمة أولاً بتنفيذ جميع عمليات التحقق) مثل KYC و AML والتحقق من الرصيد (، ثم تقوم باستدعاء Ledger Service لتنفيذ العملية المالية الفعلية بشكل ذري. بذلك يتم الفصل بين منطق الأعمال (Business Logic) وتنفيذ العمليات المالية الحساسة.

كما أن استخدام قاعدة بيانات مستقلة لكل خدمة (Database per Service) لا يؤثر على ذرية العملية داخل الخدمة نفسها، حيث يتم ضمان الاتساق على مستوى كل خدمة بشكل مستقل، بينما يتم التنسيق بين الخدمات من خلال تصميم يعتمد على التسلسل الصحيح للعمليات.

وبالتالي، فإن استخدام Atomic Transactions يضمن عدم حدوث حالات غير متسقة مثل خصم المبلغ دون إضافته، أو تسجيل جزء من العملية فقط. وهذا يُعد ضرورياً في الأنظمة المصرفية لضمان دقة الأرصدة وموثوقية العمليات المالية، ويشكل أساساً قوياً لبناء نظام آمن ومستقر.

9-3 منع تكرار العمليات (Idempotency):

تُستخدم آلية منع تكرار العمليات (Idempotency) في النظام لضمان عدم تنفيذ العملية المالية أو العملية الحساسة أكثر من مرة عند إرسال الطلب نفسه عدة مرات. وتُعد هذه الآلية مهمة بشكل خاص في الأنظمة المصرفية، لأن إعادة إرسال طلب التحويل نتيجة انقطاع الشبكة أو تأخر الاستجابة يجب ألا تؤدي إلى خصم المبلغ مرتين.

يعتمد النظام على Idempotency Key يتم إرساله مع العمليات التي قد يؤدي تكرارها إلى نتائج غير مرغوبة. يتم استخدام هذا المفتاح لتمييز الطلب الأصلي عن عمليات إعادة الإرسال لنفس الطلب. ومن

العمليات التي تعتمد هذه الآلية إنشاء الحسابات، والتحويلات المالية، ودفع الفواتير، وإنشاء الاشتراكات، وعمليات شحن البطاقات، إضافة إلى بعض العمليات الإدارية مثل Mint.

عند وصول طلب جديد، تقوم الخدمة بالتحقق من وجود عملية سابقة مرتبطة بالمستخدم وبنفس Idempotency Key فإذا لم تكن هناك عملية سابقة، يتم تنفيذ الطلب وتخزين المفتاح مع النتيجة الخاصة بالعملية. أما إذا تم إرسال الطلب نفسه مرة أخرى، فيتم اكتشاف العملية السابقة وإرجاع نتائجها بدلاً من إنشاء عملية جديدة.

يتم دعم هذه الآلية أيضاً على مستوى قاعدة البيانات من خلال استخدام Unique Constraints أو عمليات البحث عن المفتاح المرتبط بالمستخدم. فعلى سبيل المثال، في عمليات التحويل يتم ربط Idempotency Key بالمستخدم الذي بدأ العملية، بحيث لا يمكن لنفس المستخدم إنشاء تحويلين متطابقين باستخدام المفتاح نفسه.

وتظهر أهمية هذه الآلية بشكل واضح في عمليات التحويل المالي. فعلى سبيل المثال، قد يرسل المستخدم طلب تحويل ثم يحدث انقطاع مؤقت في الاتصال قبل وصول الاستجابة إليه، فيقوم بإرسال الطلب مرة أخرى معتقداً أن العملية لم تنفذ. بدون Idempotency يمكن أن يتم تنفيذ التحويل مرتين، بينما يقوم النظام في هذه الحالة بالتعرف على المفتاح المستخدم وإعادة نتيجة العملية الأصلية فقط.

كما تتكامل Idempotency مع المعاملات الذرية (Atomic Transactions) داخل Ledger Service حيث تمنع Idempotency إنشاء العملية المالية نفسها أكثر من مرة، بينما تضمن Atomic Transactions أن العملية التي يتم تنفيذها تُطبق بالكامل أو يتم التراجع عنها بالكامل عند حدوث خطأ.

وبذلك تساهم آلية Idempotency في حماية النظام من تكرار العمليات الناتج عن إعادة إرسال الطلبات، وأخطاء الشبكة، وإعادة المحاولة (Retries)، كما توفر مستوى إضافياً من الموثوقية في العمليات المالية الحساسة. ويُعد الجمع بين Idempotency و (Atomic Transactions) و (Double-Entry Ledger) من العناصر المهمة لضمان دقة العمليات والأرصدة داخل النظام المصرفي.

الفصل الرابع

(تصميم وتنفيذ الواجهة الامامية لنظام المصرف الإلكتروني)

4-1 تمهيد:

تمثل الواجهة الأمامية الجزء من النظام الذي يتفاعل معه المستخدم بشكل مباشر، حيث توفر مجموعة من الصفحات والمكونات التي تتيح للمستخدم الوصول إلى الخدمات المصرفية المختلفة وتنفيذ العمليات المطلوبة بطريقة منظمة وسهلة الاستخدام.

تم تطوير الواجهة الأمامية لنظام المصرف الإلكتروني باستخدام مكتبة React، مع اعتماد أسلوب مكونات الواجهة القابلة لإعادة الاستخدام، وفصل منطق الاتصال بالخدمات الخلفية عن منطق عرض الصفحات. وتتولى الواجهة الأمامية إرسال الطلبات إلى النظام الخلفي من خلال Gateway Service، والذي يمثل نقطة الدخول الموحدة للتطبيقات الخارجية إلى الخدمات المصغرة للنظام.

تغطي الواجهة الأمامية الوظائف الأساسية للنظام المصرفي، ومن أهمها تسجيل المستخدمين وتسجيل الدخول، إدارة الحسابات المصرفية، تنفيذ التحويلات المالية، إدارة البطاقات الافتراضية، إدارة الفواتير والاشتراكات، تقديم طلبات التحقق من الهوية، بالإضافة إلى توفير واجهة خاصة بالمسؤول لإدارة ومراقبة أجزاء مختلفة من النظام.

يهدف هذا الفصل إلى توضيح آلية تصميم وتنفيذ الواجهة الأمامية للنظام، وشرح التقنيات المستخدمة، وبنية المشروع، وآلية إدارة المسارات والمصادقة، وطريقة التواصل مع الخدمات الخلفية، بالإضافة إلى شرح الصفحات والمكونات البرمجية الأساسية والعلاقة بينها وبين الخدمات التي يوفرها النظام الخلفي.

2-4 التقنيات والأدوات المستخدمة في تطوير الواجهة الأمامية :

1-2-4 مكتبة React:

تم تطوير الواجهة الأمامية لنظام المصرف الإلكتروني باستخدام مكتبة React [13]، وهي مكتبة تعتمد على بناء واجهات المستخدم من خلال مجموعة من المكونات البرمجية (Components)، يسمح هذا الأسلوب بتقسيم الواجهة إلى أجزاء مستقلة، بحيث يكون لكل مكون وظيفة محددة، مع إمكانية إعادة استخدام المكونات المشتركة في أكثر من صفحة.

في المشروع، تم الاعتماد على هذا المفهوم في بناء الصفحات والمكونات المختلفة، حيث تم فصل الصفحات الرئيسية عن المكونات المشتركة ومكونات الواجهة الرسومية. فعلى سبيل المثال، تم تخصيص مجموعة من المكونات لعمليات المصادقة، ومكونات أخرى لتخطيط التطبيق والنوافذ المنبثقة وعناصر واجهة المستخدم، بينما تم وضع الصفحات الرئيسية ضمن مجلد مستقل.

يساعد هذا التنظيم على تقليل تكرار الكود وتحسين قابلية الصيانة، كما يسمح بتعديل أحد المكونات المشتركة دون الحاجة إلى تعديل جميع الصفحات التي تستخدمه.

2-2-4 JavaScript & JSX:

تم استخدام لغة JavaScript لتنفيذ منطق الواجهة الأمامية وإدارة العمليات البرمجية المختلفة، بينما تم استخدام صيغة JSX لكتابة بنية عناصر واجهة المستخدم داخل مكونات React.

يسمح JSX بكتابة عناصر الواجهة بطريقة قريبة من بنية HTML، مع إمكانية دمج JavaScript داخلها لتنفيذ العمليات الديناميكية. وقد تم استخدامه في المشروع لبناء الصفحات والمكونات والتعامل مع بيانات المستخدم وحالات التحميل والأخطاء وتنفيذ الأحداث الناتجة عن تفاعل المستخدم مع الواجهة. كما تعتمد ملفات المشروع التي تحتوي على المكونات والصفحات على امتداد JSX، مما يشير إلى استخدامها لبناء مكونات React باستخدام JSX.

:Vite 3-2-4

تم استخدام Vite [14] كأداة لبناء وتشغيل مشروع الواجهة الأمامية. يوفر Vite بيئة تطوير سريعة لتطبيقات الويب، كما يتولى عملية تشغيل خادم التطوير وتجهيز المشروع للإنتاج. يتيح استخدام Vite تشغيل الواجهة الأمامية أثناء مرحلة التطوير من خلال أوامر المشروع المعرفة في ملف package.json، كما يوفر آلية للتعامل مع متغيرات البيئة والإعدادات الخاصة بالتطبيق، ومن بينها عنوان الواجهة الخلفية التي تتصل بها الواجهة الأمامية.

:React Router 4-2-4

تم استخدام React Router لإدارة عملية التنقل بين صفحات التطبيق. يسمح ذلك بربط كل عنوان URL بصفحة أو مكون معين دون الحاجة إلى إعادة تحميل التطبيق بالكامل عند الانتقال بين الصفحات. تم تعريف مجموعة من المسارات الخاصة بالوظائف المختلفة للنظام، ومن بينها صفحة تسجيل الدخول، لوحة التحكم، الحسابات، التحويلات، البطاقات، الفواتير والاشتراكات، التحقق من الهوية، بالإضافة إلى صفحة الإدارة.

كما تم استخدام آلية لحماية بعض المسارات، بحيث لا يستطيع المستخدم غير المصادق عليه الوصول إلى الصفحات المصرفية المحمية، بينما يتم تخصيص بعض المسارات للمستخدمين الإداريين فقط.

:Axios 5-2-4

تم استخدام مكتبة Axios [15] لتنفيذ طلبات HTTP بين الواجهة الأمامية والواجهة الخلفية للنظام. وقد تم تنظيم عملية الاتصال من خلال إنشاء عميل HTTP مركزي داخل الملف api/client.js بدلاً من إنشاء اتصال مستقل داخل كل صفحة.

يساعد هذا الأسلوب على توحيد طريقة إرسال الطلبات واستقبال الاستجابات، كما يسمح بإضافة الوظائف المشتركة المتعلقة بالمصادقة والتعامل مع الأخطاء وانتهاء صلاحية رموز الوصول في مكان واحد.

وبذلك تصبح الصفحات والخدمات البرمجية قادرة على إرسال الطلبات إلى الواجهة الخلفية دون الحاجة إلى تكرار إعدادات الاتصال في كل ملف.

4-2-6: Tailwind CSS

تم استخدام Tailwind CSS [16] لتنسيق وتصميم عناصر الواجهة الأمامية. ويعتمد Tailwind على مجموعة من الأصناف الجاهزة (Utility Classes) التي يمكن تطبيقها مباشرة على عناصر JSX لتحديد خصائص مثل الأبعاد، والمسافات، والمحاذاة، وحجم الخطوط، وأسلوب العرض وغيرها من خصائص التصميم.

ساهم استخدام Tailwind CSS في توحيد أسلوب تصميم صفحات التطبيق، كما سهّل تطبيق التنسيقات المختلفة على المكونات وإعادة استخدام نفس المكونات ضمن عدة صفحات.

4-2-7: إدارة الحالة باستخدام Context

تم استخدام آلية Context في React لمشاركة بعض البيانات والحالات بين أجزاء مختلفة من التطبيق دون الحاجة إلى تمريرها يدويًا عبر جميع المكونات الوسيطة.

ويظهر ذلك في المشروع من خلال وجود AuthContext المسؤول عن إدارة حالة المصادقة وبيانات المستخدم، بالإضافة إلى ToastContext المسؤول عن إدارة رسائل التنبيه والإشعارات التي تظهر للمستخدم.

يساعد هذا الأسلوب على توفير البيانات المشتركة للمكونات التي تحتاج إليها، ويجعل بنية التطبيق أكثر تنظيمًا ويقلل من تمرير الخصائص (Props) عبر مستويات متعددة من المكونات.

4-2-8: الاتصال بالواجهة الخلفية:

تعتمد الواجهة الأمامية على طلبات HTTP للتواصل مع الخدمات التي يوفرها النظام الخلفي. ولا تتصل الواجهة الأمامية مباشرة بالخدمات المصغرة الداخلية، وإنما تمر الطلبات من خلال Gateway Service الذي يمثل نقطة الدخول الخارجية للنظام.

يتم تنظيم هذه الاتصالات ضمن مجموعة من الخدمات البرمجية في الواجهة الأمامية، بحيث يوجد ملف مخصص للتعامل مع كل مجموعة من الوظائف، مثل خدمة المصادقة، وخدمة الحسابات، وخدمة التحويلات، وخدمة البطاقات، وخدمة الفواتير، وخدمة التحقق من الهوية، وخدمة سجل المعاملات.

يسمح هذا الفصل بين واجهة المستخدم وعمليات الاتصال بالواجهة الخلفية بالحفاظ على تنظيم المشروع وتقليل الاعتماد المباشر بين الصفحات والخدمات الخلفية.

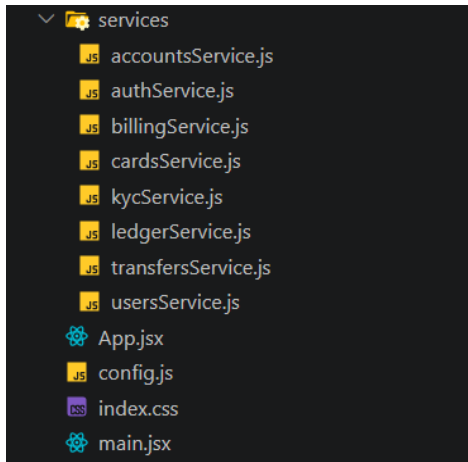
3-4 البنية التنظيمية لمشروع الواجهة الأمامية :

1-3-4 التنظيمية لمشروع الواجهة الأمامية:

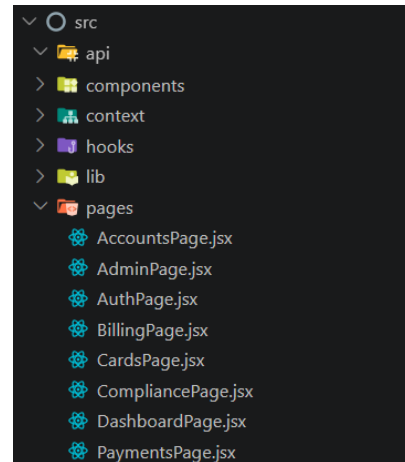
تم تنظيم مشروع الواجهة الأمامية وفق بنية تهدف إلى فصل المسؤوليات البرمجية بين مختلف أجزاء التطبيق، بحيث يتم فصل الصفحات عن المكونات القابلة لإعادة الاستخدام، وعن الخدمات المسؤولة عن التواصل مع الواجهة الخلفية، بالإضافة إلى فصل منطق المصادقة وإدارة الحالة والوظائف المساعدة. يساعد هذا التنظيم على جعل المشروع أكثر وضوحًا وقابلية للصيانة والتطوير، حيث يمكن تعديل جزء معين من النظام دون التأثير بشكل مباشر على بقية الأجزاء. كما يسمح بإعادة استخدام المكونات والخدمات البرمجية في أكثر من صفحة، مما يقلل من تكرار الكود ويحافظ على تنظيم المشروع مع ازدياد عدد الوظائف والصفحات.

تتكون البنية الرئيسية لمشروع الواجهة الأمامية من مجموعة من المجلدات والملفات، ويتركز الجزء البرمجي الأساسي داخل مجلد src، الذي يحتوي على الصفحات والمكونات والخدمات وإدارة الحالة والوظائف المساعدة، إضافة إلى الملفات المسؤولة عن تشغيل التطبيق وإعداداته.

4-3-2 البنية العامة للمجلدات:



الشكل (1-4) Folders



الشكل (2-4) Pages

4-3-3 مجلد الصفحات Pages:

يحتوي مجلد pages على الصفحات الرئيسية التي يتعامل معها المستخدم داخل النظام. تم فصل الصفحات عن بقية مكونات المشروع بحيث تكون كل صفحة مسؤولة عن تجميع الوظائف والمكونات التي تحتاج إليها لتقديم وظيفة محددة للمستخدم.

يحتوي المجلد على مجموعة من الصفحات التي تغطي الوظائف الأساسية للنظام، وهي:

- AuthPage.jsx صفحة المصادقة وتسجيل الدخول والتسجيل.
- DashboardPage.jsx الصفحة الرئيسية للمستخدم بعد تسجيل الدخول.
- AccountsPage.jsx إدارة الحسابات البنكية.
- PaymentsPage.jsx تنفيذ وإدارة التحويلات المالية.

- CardsPage.jsx إدارة البطاقات الافتراضية.
 - BillingPage.jsx إدارة الفواتير والاشتراكات.
 - CompliancePage.jsx إدارة طلبات التحقق من الهوية (KYC).
 - AdminPage.jsx الواجهة الخاصة بالوظائف الإدارية.
- ويتم ربط هذه الصفحات بالمسارات المناسبة داخل التطبيق، بحيث يؤدي كل مسار إلى الصفحة التي تمثل الوظيفة المطلوبة.

4-3-4 مجلد المكونات components:

يحتوي مجلد components على المكونات البرمجية القابلة لإعادة الاستخدام داخل صفحات التطبيق. ويعتمد تصميم الواجهة على تقسيم العناصر الكبيرة إلى مكونات أصغر، بحيث يؤدي كل مكون وظيفة محددة ويمكن استخدامه في أكثر من مكان عند الحاجة.

تم تقسيم المكونات إلى مجموعات فرعية حسب وظيفتها، ومن أبرزها:

أولاً: مكونات المصادقة auth:

تحتوي على المكونات المستخدمة في صفحات تسجيل الدخول وإنشاء الحساب، مثل حقول الإدخال ونموذج تسجيل الدخول ونموذج التسجيل والغلاف العام لواجهة المصادقة.

ثانياً: مكونات التخطيط layout:

تحتوي على المكونات المسؤولة عن الهيكل العام للصفحات الداخلية للتطبيق، ومن بينها AppLayout.jsx.

ثالثاً: مكونات سجل المعاملات ledger:

تحتوي على المكون المسؤول عن عرض سلسلة سجل المعاملات بصورة مرئية، وهو
LedgerChainVisualizer.jsx.

رابعًا: النوافذ المنبثقة **modals**:

تحتوي على النوافذ التي تستخدم لتنفيذ عمليات معينة دون الانتقال إلى صفحة جديدة، مثل إنشاء حساب جديد وتنفيذ تحويل مالي.

خامسًا: مكونات واجهة المستخدم **ui**:

تحتوي على العناصر العامة والقابلة لإعادة الاستخدام في الواجهة، مثل الأزرار وحقول الإدخال والبطاقات والنوافذ والتنبيهات وغيرها من عناصر واجهة المستخدم.

4-3-5 مجلد الخدمات **services**:

يحتوي مجلد **services** على الملفات المسؤولة عن تنفيذ عمليات الاتصال بالواجهة الخلفية المتعلقة بالوظائف المختلفة للنظام.

تم تقسيم الخدمات حسب المجال الوظيفي، بحيث يوجد ملف مستقل لكل مجموعة من العمليات المرتبطة بوظيفة معينة. ويؤدي هذا الفصل إلى منع وضع جميع طلبات الاتصال بالواجهة الخلفية داخل الصفحات، مما يحافظ على فصل واضح بين منطق عرض الواجهة ومنطق الاتصال بالخدمات الخلفية.

تشمل الخدمات الموجودة في المشروع:

authService.js عمليات المصادقة وإدارة المستخدم.

accountsService.js العمليات المتعلقة بالحسابات.

transfersService.js عمليات التحويلات المالية.

cardsService.js إدارة البطاقات الافتراضية.

billingService.js الفواتير والاشتراكات.

kycService.js عمليات التحقق من الهوية والوثائق.

ledgerService.js الوصول إلى بيانات سجل المعاملات.

userService.js العمليات المتعلقة بالمستخدمين.

وبذلك يمكن تمثيل العلاقة بشكل مبسط كما يلي:

Page → Service → API Client → Gateway → Backend Service

حيث تقوم الصفحة باستدعاء الوظيفة المناسبة من الخدمة، ثم تقوم الخدمة بإرسال الطلب من خلال عميل الاتصال المركزي إلى الـ Gateway ، الذي يتولى توجيه الطلب إلى الخدمة الخلفية المناسبة.

4-3-6 مجلد الاتصال بالواجهة البرمجية api:

يحتوي مجلد api على الملف المسؤول عن إعداد عميل الاتصال بالواجهة الخلفية، وهو client.js.

يمثل هذا الملف طبقة مشتركة تستخدمها الخدمات البرمجية المختلفة لإرسال طلبات HTTP ويساعد وجود عميل مركزي على توحيد إعدادات الاتصال والتعامل مع المصادقة والاستجابات والأخطاء بدل تكرار هذه العمليات داخل كل خدمة أو صفحة.

وسيتم شرح آلية عمل client.js بالتفصيل في قسم الاتصال بالواجهة الخلفية.

4-3-7 مجلد إدارة الحالة context:

يحتوي مجلد context على الحالات العامة التي تحتاج إليها أجزاء متعددة من التطبيق.

يتضمن المجلد:

- `AuthContext.jsx` إدارة حالة المستخدم والمصادقة والجلسة.
 - `ToastContext.jsx` إدارة رسائل التنبيه والإشعارات التي تظهر للمستخدم.
- يسمح استخدام Context بمشاركة هذه البيانات بين المكونات المختلفة دون الحاجة إلى تمريرها يدويًا عبر جميع المستويات الوسيطة في شجرة المكونات.

4-3-8 مجلد Hooks:

يحتوي مجلد Hooks على الوظائف المخصصة التي تعتمد على React Hooks لإعادة استخدام منطق معين بين أكثر من مكون أو صفحة.

يحتوي المشروع على `useAccounts.js`، وهو Hook مخصص للتعامل مع البيانات والعمليات المرتبطة بالحسابات، مما يسمح للصفحات التي تحتاج إلى هذه البيانات باستخدام نفس المنطق بدل تكراره داخل كل صفحة.

4-3-9 مجلد الوظائف المساعدة lib:

يحتوي مجلد lib على مجموعة من الوظائف والأدوات المساعدة التي تستخدمها أجزاء مختلفة من التطبيق. يتضمن المجلد ملفات متخصصة في مجالات مختلفة، منها:

- `accountNames.js` وظائف مرتبطة بأسماء الحسابات.
- `crypto.js` وظائف مرتبطة بالعمليات التشفيرية وإنشاء بعض المعرفات والتحقق من الملفات.
- `format.js` وظائف تنسيق البيانات مثل المبالغ والتواريخ والمعرفات.
- `tokenStore.js` إدارة رموز المصادقة الخاصة بالمستخدم.

يساعد فصل هذه الوظائف في مجلد مستقل على تجنب تكرارها داخل الصفحات والمكونات، كما يجعل الوصول إليها وإعادة استخدامها أكثر سهولة.

4-3-10 الملفات الرئيسية داخل مجلد src:

بالإضافة إلى المجلدات السابقة، يحتوي مجلد src على مجموعة من الملفات الرئيسية التي تشكل نقطة تشغيل وتنظيم التطبيق.

Main.jsx يمثل نقطة بدء تشغيل تطبيق React، حيث يتم من خلاله تحميل المكون الرئيسي للتطبيق وربطه بعنصر الصفحة الأساسي.

App.jsx يمثل المكون الرئيسي للتطبيق، ويحتوي على البنية العامة للتطبيق وآلية تعريف المسارات وربط الصفحات وإدارة بعض الطبقات العامة مثل المصادقة والتنبيهات.

Config.js يحتوي على إعدادات عامة تستخدمها الواجهة الأمامية، مثل الإعدادات المتعلقة بالاتصال بالخدمات الخلفية وبعض القيم الخاصة بالتطبيق.

Index.css يحتوي على قواعد CSS العامة المستخدمة ضمن التطبيق، بالإضافة إلى الإعدادات المتعلقة بتنسيق الواجهة.

وبالتالي يمكن اعتبار main.jsx نقطة بدء التطبيق، بينما يمثل App.jsx نقطة التجميع الرئيسية لبقية أجزاء الواجهة.

4-3-11 العلاقة بين مكونات الواجهة الأمامية:

تعتمد الواجهة الأمامية على مجموعة من الطبقات البرمجية المترابطة، حيث تبدأ عملية التفاعل من الصفحات التي تمثل واجهة المستخدم، ثم تستدعي المكونات والخدمات المناسبة لتنفيذ العمليات المطلوبة.

عند تنفيذ عملية مصرفية، تبدأ العملية من صفحة المستخدم، ثم يتم استخدام المكون المناسب لاستقبال البيانات، وبعد ذلك يتم استدعاء الخدمة البرمجية المرتبطة بالعملية. تقوم الخدمة باستخدام عميل الاتصال المركزي لإرسال الطلب إلى الـ Gateway، الذي يقوم بدوره بالتحقق من الطلب وتوجيهه إلى الخدمة الخلفية المختصة.

يتوافق هذا التصميم مع البنية العامة للنظام الخلفي، حيث تم اعتماد Gateway Service كنقطة دخول موحدة للتطبيقات الخارجية، بينما يتم التواصل بين الخدمات الداخلية باستخدام آليات الاتصال المحددة في بنية النظام.

4-4 نقطة تشغيل التطبيق وبنية الرئيسية:

يمثل الملف `main.jsx` نقطة الدخول الرئيسية لتطبيق الواجهة الأمامية، حيث تبدأ منه عملية تشغيل تطبيق React وربطه بعنصر الصفحة الأساسي، ثم يتم تحميل المكون الرئيسي `App.jsx` الذي يتولى تنظيم بقية أجزاء التطبيق.

يمثل `App.jsx` المكون الجذري للواجهة الأمامية، وتتم من خلاله إدارة البنية العامة للتطبيق، وربط المسارات بالصفحات المناسبة، بالإضافة إلى توفير آليات المصادقة والتنبيهات وحماية الصفحات التي تتطلب تسجيل الدخول.

1-4-4 إدارة المسارات:

يستخدم التطبيق نظام Routing للتنقل بين الصفحات المختلفة، حيث يرتبط كل مسار بالصفحة التي تمثل الوظيفة المطلوبة، مثل لوحة التحكم والحسابات والتحويلات والبطاقات والفواتير والتحقق من الهوية و صفحة الإدارة.

4-4-2 حماية الصفحات:

نظرًا لأن النظام يتعامل مع بيانات مصرفية حساسة، يتم تقييد الوصول إلى الصفحات التي تتطلب تسجيل الدخول. كما يتم التمييز بين المستخدم العادي والمسؤول، بحيث تكون الوظائف الإدارية متاحة للمستخدم الذي يمتلك صلاحية ADMIN.

ولا تعتمد الحماية في الواجهة الأمامية وحدها، إذ يقوم ال Gateway في الواجهة الخلفية بالتحقق من JWT والصلاحيات قبل تمرير الطلبات إلى الخدمات الداخلية. ويستخرج ال Gateway معلومات المستخدم والدور من ال JWT ويمررها للخدمات الداخلية ضمن بيانات اتصال gRPC.

4-4-3 العلاقة مع الواجهة الخلفية:

عند تنفيذ عملية مصرفية، تنتقل العملية بصورة عامة من صفحة المستخدم إلى الخدمة المناسبة في الواجهة الأمامية، ثم إلى ال API Client، وبعدها إلى ال Gateway الذي يقوم بتوجيه الطلب إلى الخدمة الخلفية المختصة.

يساهم هذا التصميم في فصل الواجهة الأمامية عن تفاصيل الخدمات الداخلية، فلا تحتاج الواجهة إلى معرفة أسماء الخدمات الداخلية أو منافذها أو استخدام بروتوكول gRPC مباشرة.

4-5 نظام المصادقة وإدارة جلسة المستخدم:

4-5-1 مقدمة:

يُعد نظام المصادقة من الأجزاء الأساسية في الواجهة الأمامية لنظام المصرف الإلكتروني، نظرًا لأن معظم الوظائف المصرفية تتطلب التأكد من هوية المستخدم قبل السماح له بالوصول إليها. وتتكامل الواجهة الأمامية في هذه العملية مع خدمة المصادقة الموجودة في الواجهة الخلفية، والتي تتولى إنشاء الحسابات وتسجيل الدخول والتحقق من البريد الإلكتروني وإصدار رموز JWT وإدارة رموز تحديث الجلسة.

يعتمد النظام على آلية المصادقة باستخدام JWT، حيث يحصل المستخدم بعد نجاح تسجيل الدخول على رمز وصول يتم استخدامه مع الطلبات اللاحقة. تقوم بوابة النظام بالتحقق من صحة الرمز قبل السماح بتمرير الطلب إلى الخدمة المطلوبة.

4-5-2 عملية تسجيل الدخول:

يبدأ المستخدم عملية تسجيل الدخول من خلال إدخال بياناته في نموذج المصادقة. تقوم الواجهة الأمامية بإرسال هذه البيانات إلى خدمة المصادقة في الواجهة الخلفية، والتي تتحقق من صحة بيانات المستخدم، ثم تصدر رمز الوصول JWT عند نجاح عملية المصادقة.

بعد الحصول على الرمز، تستخدم الواجهة الأمامية في الطلبات اللاحقة للوصول إلى الخدمات التي تتطلب المصادقة. وتقوم بوابة النظام بالتحقق من صحة توقيع JWT باستخدام المفتاح العام قبل السماح بتمرير الطلب إلى الخدمة المطلوبة.

4-5-3 إدارة الصلاحيات:

بعد التحقق من هوية المستخدم، يتم تحديد صلاحياته اعتمادًا على الدور الموجود داخل رمز JWT يدعم النظام دورين رئيسيين هما CUSTOMER و ADMIN.

يسمح ذلك للواجهة الأمامية بتحديد الصفحات والوظائف التي يمكن للمستخدم الوصول إليها، بينما يتم تطبيق التحقق الفعلي من الصلاحيات في الواجهة الخلفية. فالمسارات الإدارية لا يمكن الوصول إليها إلا من قبل المستخدمين الذين يمتلكون صلاحية ADMIN.

4-5-4 إدارة الجلسة:

يتم تنظيم حالة المصادقة داخل الواجهة الأمامية من خلال طبقة مخصصة لإدارة جلسة المستخدم. وتوفر هذه الطبقة المعلومات والوظائف التي تحتاج إليها الصفحات والمكونات لمعرفة حالة المستخدم وتنفيذ العمليات المرتبطة بالمصادقة.

يساعد هذا الفصل على منع تكرار منطق المصادقة داخل الصفحات المختلفة، كما يسمح باستخدام حالة المستخدم في أجزاء متعددة من التطبيق، مثل حماية المسارات وعرض معلومات المستخدم وتنفيذ عملية تسجيل الخروج.

4-5-5 تسجيل الخروج:

عند اختيار المستخدم تسجيل الخروج، يتم إنهاء جلسة المستخدم على مستوى الواجهة الأمامية وإزالة بيانات المصادقة المحلية، ثم يتم توجيه المستخدم إلى واجهة المصادقة.

4-5-6 إدارة رموز المصادقة وتجديد الجلسة:

تعتمد الواجهة الأمامية على تخزين رموز المصادقة اللازمة لاستمرار جلسة المستخدم، ويتم تنظيم عملية التعامل مع هذه الرموز من خلال الملف `tokenStore.js`. يوفر هذا الملف مجموعة من الوظائف المسؤولة عن قراءة رمز الوصول ورمز تحديث الجلسة، بالإضافة إلى تخزين هذه الرموز وإزالتها عند انتهاء الجلسة أو تسجيل الخروج.

يتم الاحتفاظ برموز الوصول `Access Token` ورمز تحديث الجلسة `Refresh Token` في التخزين المحلي للمتصفح `localStorage`، مما يسمح للواجهة الأمامية باستعادة حالة الجلسة عند إعادة تحميل التطبيق. وعند نجاح عملية تسجيل الدخول، تقوم الواجهة الأمامية بتخزين الرموز التي تعيدها خدمة المصادقة، ثم تستخدم رموز الوصول في الطلبات اللاحقة المرسل إلى `Gateway` للوصول إلى الخدمات التي تتطلب المصادقة.

كما يحتوي الملف `tokenStore.js` على آلية لإشعار `AuthContext` عند حدوث تغيير في حالة رموز المصادقة. فعند إزالة الرموز نتيجة تسجيل خروج المستخدم أو فشل عملية تجديد الجلسة، يتم تحديث حالة المصادقة داخل التطبيق وإعادة حالة المستخدم إلى الحالة غير المصادق عليها، مما يؤدي إلى منعه من الوصول إلى الصفحات المحمية وإعادة توجيهه إلى واجهة تسجيل الدخول.

4-5-7 تجديد رمز الوصول:

نظرًا لأن رمز الوصول له مدة صلاحية محددة، فقد تم تنفيذ آلية تسمح بتجديد جلسة المستخدم تلقائيًا عند انتهاء صلاحية الرمز أثناء استخدام التطبيق، وذلك دون الحاجة إلى تسجيل الدخول مرة أخرى.

يتم تنفيذ هذه الآلية من خلال الملف `api/client.js`، حيث يحتوي عميل `Axios` على آلية لاعتراض الاستجابات القادمة من الواجهة الخلفية. فعند استقبال استجابة تحمل رمز الحالة 401، والذي يشير إلى عدم صلاحية المصادقة أو انتهاء صلاحية رمز الوصول، تقوم الواجهة الأمامية بالتحقق من وجود `Refresh Token` صالح.

في حال توفر رمز التحديث، يتم إرسال طلب إلى نقطة النهاية `/auth/refresh` للحصول على رمز وصول جديد. وبعد نجاح عملية التجديد، يتم تخزين الرموز الجديدة، ثم إعادة إرسال الطلب الأصلي باستخدام رمز الوصول الجديد، وبذلك يستطيع المستخدم متابعة استخدام النظام دون انقطاع في الجلسة.

كما تم التعامل مع حالة إرسال عدة طلبات في الوقت نفسه، والتي قد تحصل جميعها على استجابة 401 بسبب انتهاء صلاحية رمز الوصول. ولمنع تنفيذ عدة عمليات لتجديد الجلسة بشكل متزامن، يتم استخدام متغير مشترك باسم `refreshPromise`، بحيث يتم تنفيذ عملية تجديد واحدة فقط، بينما تنتظر الطلبات الأخرى نتائجها ثم تستخدم رمز الوصول الجديد بعد نجاح عملية التجديد.

أما في حال فشل عملية تجديد الجلسة، سواء بسبب عدم صلاحية رمز التحديث أو انتهاء الجلسة، فتقوم الواجهة الأمامية بحذف رموز المصادقة المحلية، وتحديث حالة المصادقة داخل `AuthContext` إلى الحالة غير المصادق عليها، ثم يتم توجيه المستخدم إلى واجهة تسجيل الدخول ليتمكن من إنشاء جلسة جديدة.

4-5-8 التسجيل والتحقق من البريد الإلكتروني:

توفر الواجهة الأمامية نموذجًا مخصصًا لإنشاء حساب مستخدم جديد من خلال `RegisterForm.jsx`. يبدأ المستخدم بإدخال البريد الإلكتروني وكلمة المرور، ويتم إجراء بعض عمليات التحقق الأولية على البيانات قبل إرسالها إلى الواجهة الخلفية.

تتشرط الواجهة الأمامية أن يكون البريد الإلكتروني بالصيغة المناسبة، وأن تحتوي كلمة المرور على ثمانية أحرف على الأقل. بعد ذلك يتم إرسال البيانات إلى نقطة التسجيل من خلال `authService.js`.

بعد إنشاء الحساب، يتم الانتقال إلى مرحلة التحقق من البريد الإلكتروني، حيث يعرض النظام للمستخدم حقلاً لإدخال رمز التحقق المرسل إلى بريده. وتقوم الواجهة الأمامية بإرسال البريد الإلكتروني ورمز التحقق إلى خدمة المصادقة من خلال نقطة `/auth/verify-email`.

وفي حال نجاح عملية التحقق، يستطيع المستخدم الانتقال إلى تسجيل الدخول، كما يمكن للواجهة الأمامية تنفيذ تسجيل الدخول مباشرة عندما تكون كلمة المرور التي استخدمت أثناء عملية التسجيل ما تزال موجودة في حالة النموذج.

كما تم توفير إمكانية إعادة إرسال رمز التحقق من خلال الوظيفة `resendVerification`، مما يسمح للمستخدم بطلب رمز جديد عند انتهاء صلاحية الرمز السابق أو عدم وصوله.

4-5-9 استعادة كلمة المرور:

تتضمن واجهة تسجيل الدخول آلية مخصصة لاستعادة كلمة المرور في حال نسيانها. تبدأ العملية من خلال إدخال المستخدم لعنوان البريد الإلكتروني، ثم يتم استدعاء الوظيفة `forgotPassword` لإرسال طلب استعادة كلمة المرور إلى الواجهة الخلفية.

بعد إرسال الطلب، تنتقل الواجهة الأمامية إلى مرحلة إدخال رمز إعادة التعيين وكلمة المرور الجديدة. ويتم التحقق من كلمة المرور الجديدة قبل إرسالها، بحيث يجب ألا يقل طولها عن ثمانية أحرف.

بعد ذلك يتم استدعاء الوظيفة `resetPassword` مع رمز إعادة التعيين وكلمة المرور الجديدة، ثم يتم إرسال الطلب إلى خدمة المصادقة في الواجهة الخلفية. وعند نجاح العملية، يتم إرجاع المستخدم إلى شاشة تسجيل الدخول، ليتمكن من تسجيل الدخول باستخدام كلمة المرور الجديدة.

يساعد تقسيم عملية استعادة كلمة المرور إلى مرحلتين على فصل عملية طلب رمز إعادة التعيين عن عملية تغيير كلمة المرور، كما يتيح للواجهة الأمامية عرض رسائل واضحة للمستخدم توضح حالة العملية في كل مرحلة.

6-4 اتصال بالواجهة الخلفية:

1-6-4 عميل الاتصال المركزي API Client:

تم تنفيذ عملية الاتصال بين الواجهة الأمامية والواجهة الخلفية من خلال الملف `src/api/client.js`، والذي يمثل طبقة الاتصال المركزية التي تعتمد عليها الخدمات البرمجية المختلفة في المشروع لإرسال طلبات HTTP.

تم إنشاء عميل `Axios` بالاعتماد على عنوان الواجهة الخلفية المحدد في الملف `config.js`، وبذلك يمكن لجميع الخدمات استخدام إعدادات اتصال موحدة، دون الحاجة إلى إنشاء عميل اتصال مستقل داخل كل خدمة أو صفحة.

كما يتولى عميل الاتصال المركزي تنفيذ مجموعة من الوظائف المشتركة، ومن أهمها إضافة رمز المصادقة إلى الطلبات، ومعالجة الأخطاء الناتجة عن الاستجابات، وتجديد رمز الوصول عند انتهاء صلاحيته، بالإضافة إلى تحويل الاستجابات غير الناجحة إلى أخطاء موحدة يمكن للصفحات والمكونات التعامل معها بطريقة منظمة.

وقد تم أيضًا تحديد مجموعة من المسارات باعتبارها مسارات عامة، مثل تسجيل الدخول، وإنشاء الحساب، والتحقق من البريد الإلكتروني، وتحديث الجلسة. ولا يتم إرسال رمز المصادقة `Bearer Token` مع هذه الطلبات، بينما تتم إضافة رمز الوصول تلقائيًا إلى بقية الطلبات التي تتطلب مصادقة عند توفره.

يساهم هذا التصميم في توحيد آلية الاتصال بين الواجهة الأمامية والواجهة الخلفية، كما يقلل من تكرار الكود، ويجعل إدارة المصادقة والأخطاء وتجديد الجلسة تتم من خلال نقطة مركزية واحدة.

2-6-4 إضافة رمز المصادقة إلى الطلبات:

يستخدم عميل `Axios` آلية `request interceptor` لإضافة رمز الوصول إلى الطلبات التي تتطلب مصادقة. ويتم تنفيذ هذه العملية قبل إرسال الطلب إلى الواجهة الخلفية، مما يسمح بتطبيق آلية موحدة لإضافة بيانات المصادقة على مختلف الطلبات.

في البداية، يتم التحقق من المسار المطلوب للتأكد من أنه لا ينتمي إلى قائمة المسارات العامة التي لا تتطلب مصادقة. وفي حال كان الطلب يتطلب مصادقة، يتم الحصول على رمز الوصول من الملف tokenStore.js، ثم إضافته إلى ترويسة الطلب باستخدام الحقل Authorization بالشكل التالي:

Authorization: Bearer <Access Token>

وبذلك يتم إرسال رمز الوصول تلقائيًا مع الطلبات المحمية، دون الحاجة إلى كتابة آلية إضافة رمز المصادقة بشكل منفصل داخل كل Service أو Page.

يساهم هذا الأسلوب في تقليل تكرار الكود وتوحيد طريقة التعامل مع المصادقة، كما يجعل الخدمات والصفحات أكثر بساطة، بحيث تركز على تنفيذ الوظائف المطلوبة دون الحاجة إلى التعامل بشكل مباشر مع تفاصيل إضافة رمز المصادقة إلى كل طلب.

4-6-3 معالجة أخطاء الاتصال:

تم إنشاء الصنف ApiError لتوحيد طريقة التعامل مع الأخطاء القادمة من الواجهة الخلفية، بحيث يتم تحويل الأخطاء المختلفة إلى صيغة موحدة يمكن للصفحات والمكونات التعامل معها. ويمكن أن يتضمن الخطأ رمز حال HTTP، ورمزًا خاصًا بالخدمة، بالإضافة إلى رسالة توضح سبب فشل العملية.

تتعامل الواجهة الأمامية مع مجموعة من حالات HTTP الشائعة، حيث يتم تفسير كل حالة وفقًا لطبيعة الخطأ، كما يلي:

- 400: عند وجود طلب غير صالح أو وجود بيانات غير صحيحة في الطلب.
- 401: عند انتهاء جلسة المستخدم أو عدم صحة بيانات المصادقة.
- 403: عند محاولة المستخدم تنفيذ عملية لا يمتلك الصلاحيات المطلوبة لتنفيذها.
- 404: عند عدم العثور على المورد المطلوب.
- 409: عند حدوث تعارض بين العملية المطلوبة والحالة الحالية للبيانات.
- 429: عند تجاوز عدد الطلبات المسموح بها خلال فترة زمنية محددة.
- 503: عند عدم توفر إحدى الخدمات الخلفية أو عدم قدرتها على معالجة الطلب.
- 504: عند انتهاء المهلة المحددة لانتظار استجابة الخدمة.

كما يتم التعامل مع حالات عدم القدرة على الوصول إلى Gateway، فعند عدم الحصول على استجابة من البوابة يتم تحويل الخطأ إلى رسالة مناسبة للمستخدم توضح أن خدمة النظام غير متاحة حالياً، بدلاً من عرض رسالة تقنية ناتجة عن مكتبة Axios.

يساعد هذا الأسلوب على توحيد معالجة الأخطاء في مختلف أجزاء الواجهة الأمامية، كما يوفر للمستخدم رسائل أوضح حول سبب فشل العملية، ويمنع ظهور التفاصيل التقنية الداخلية التي لا يحتاج المستخدم إلى معرفتها.

4-6-4 استخراج بيانات الاستجابة:

يحتوي الملف client.js على وظيفة unwrap المسؤولة عن استخراج البيانات الفعلية من استجابة Axios، وذلك من خلال الوصول إلى الخاصية data الموجودة ضمن الاستجابة.

يسمح هذا الأسلوب للخدمات البرمجية بالتعامل مباشرة مع البيانات التي يعيدها API، دون الحاجة إلى التعامل مع بنية استجابة Axios في كل عملية اتصال.

وبذلك تصبح طريقة كتابة الخدمات أكثر وضوحاً وتنظيماً، حيث تقوم كل Service باستدعاء الدالة المناسبة من عميل الاتصال، مثل api.get أو api.post أو غيرها، ثم تحصل على البيانات المطلوبة وتعيدها إلى الصفحة أو المكون الذي قام باستدعاء الخدمة.

يساهم استخدام unwrap في تقليل تكرار الكود داخل الخدمات، كما يحافظ على فصل مسؤولية التعامل مع بنية استجابة Axios داخل client.js، بينما تركز الخدمات على تنفيذ العمليات الخاصة بوظائف النظام.

4-7 الخدمات البرمجية في الواجهة الأمامية:

تم تقسيم عمليات الاتصال بالواجهة الخلفية إلى مجموعة من الملفات الموجودة داخل مجلد services. ويعتمد هذا التقسيم على المجال الوظيفي لكل خدمة، بحيث يتم تخصيص ملف مستقل للعمليات المرتبطة بكل وظيفة، بدلاً من وضع جميع طلبات API داخل ملف واحد.

يساعد هذا الأسلوب على فصل مسؤوليات الاتصال بالواجهة الخلفية عن الصفحات والمكونات، كما يجعل الوصول إلى العمليات المرتبطة بكل وظيفة أكثر وضوحًا، ويسهل عملية صيانة المشروع وتطويره مستقبلاً.

4-7-1 خدمة المصادقة Auth Service:

يحتوي الملف `authService.js` على مجموعة من الوظائف المرتبطة بعمليات المصادقة وإدارة جلسة المستخدم. وتوفر هذه الخدمة الواجهة البرمجية اللازمة للتعامل مع إنشاء الحسابات وتسجيل الدخول والتحقق من البريد الإلكتروني وإدارة الجلسة.

ومن أهم الوظائف الموجودة في الخدمة:

- `register`: تستخدم لإنشاء مستخدم جديد.
- `verifyEmail`: تستخدم للتحقق من البريد الإلكتروني من خلال رمز التحقق.
- `resendVerification`: تستخدم لإعادة إرسال رمز التحقق إلى البريد الإلكتروني.
- `login`: تستخدم لتنفيذ عملية تسجيل الدخول.
- `refresh`: تستخدم لتحديث جلسة المستخدم والحصول على رموز مصادقة جديدة.
- `logout`: تستخدم لإنهاء جلسة المستخدم.
- `forgotPassword`: تستخدم لطلب استعادة كلمة المرور.
- `resetPassword`: تستخدم لتغيير كلمة المرور بعد التحقق من رمز إعادة التعيين.
- `getCurrentUser`: تستخدم للحصول على بيانات المستخدم الحالي والتحقق من حالة الجلسة.

تعتمد هذه الوظائف على عميل الاتصال المركزي الموجود في `client.js` لإرسال الطلبات إلى الواجهة الخلفية، ولذلك لا تحتوي `authService.js` على تفاصيل إنشاء اتصال مستقل أو إعدادات Axios الخاصة بالطلبات.

4-7-2 خدمة الحسابات Accounts Service:

يحتوي الملف `accountsService.js` على مجموعة من العمليات المرتبطة بإدارة الحسابات البنكية الخاصة بالمستخدم. وتوفر هذه الخدمة الوظائف اللازمة للوصول إلى بيانات الحسابات وتنفيذ العمليات المتعلقة بها من خلال الواجهة الخلفية.

تشمل هذه العمليات الحصول على قائمة الحسابات الخاصة بالمستخدم، وإنشاء حساب بنكي جديد، وتعديل اسم الحساب، بالإضافة إلى الحصول على بيانات حساب محدد وإدارة أعضاء الحسابات المشتركة.

كما توفر الخدمة مجموعة من الوظائف الخاصة بدعوات الحسابات المشتركة، حيث يمكن إرسال دعوة إلى مستخدم آخر، وعرض الدعوات الموجهة إلى المستخدم الحالي، بالإضافة إلى إمكانية قبول الدعوة أو رفضها أو إلغائها حسب الحالة.

وتتضمن الخدمة كذلك مجموعة من العمليات الإدارية الخاصة بالحسابات، مثل الحصول على حسابات مستخدم معين، واستعراض الحسابات وفق نوع محدد، بالإضافة إلى تجميد الحساب أو إلغاء تجميده.

يساعد فصل هذه العمليات داخل `accountsService.js` على إبقاء الصفحات والمكونات بعيدة عن تفاصيل الاتصال بالواجهة الخلفية، بحيث تستدعي الصفحة الوظيفة المناسبة من الخدمة وتتعامل مع البيانات التي تعيدها فقط.

4-7-3 خدمة التحويلات Transfers Service:

يختص الملف `transfersService.js` بتنفيذ العمليات المتعلقة بالتحويلات المالية واستعراض بيانات التحويلات التي تم تنفيذها.

تستخدم الوظيفة `createTransfer` لإرسال بيانات التحويل إلى الواجهة الخلفية، وتشمل هذه البيانات الحساب المصدر، والحساب المستلم، والمبلغ المطلوب تحويله، بالإضافة إلى وصف العملية ومفتاح `Idempotency` المستخدم لتمييز العملية ومنع تكرار تنفيذها عند إعادة إرسال الطلب.

كما توفر الخدمة وظائف للحصول على بيانات تحويل محدد، واستعراض تحويلات المستخدم، مع إمكانية تصفية النتائج وفق حالة التحويل أو الحساب المرتبط بالعملية.

وتتضمن الخدمة أيضًا بعض الوظائف الخاصة بالعمليات الإدارية، مثل إنشاء الأموال في حساب معين واستعراض التحويلات من خلال الوظائف المخصصة للمسؤول.

وبذلك توفر `Transfers Service` طبقة مستقلة للتعامل مع التحويلات، مما يسمح لصفحات التحويلات باستدعاء العمليات المطلوبة دون التعامل مباشرة مع تفاصيل طلبات `HTTP`.

4-7-4 خدمة البطاقات Cards Service:

يتولى الملف `cardsService.js` تنفيذ العمليات المتعلقة بالبطاقات الافتراضية الخاصة بالمستخدم.

تشمل هذه العمليات عرض البطاقات الموجودة، والحصول على بيانات بطاقة محددة، وإنشاء بطاقة جديدة وربطها بحساب التمويل المطلوب.

كما توفر الخدمة وظائف للتحكم في حالة البطاقة، مثل تجميد البطاقة وإلغاء تجميدها، بالإضافة إلى تنفيذ عمليات الدفع باستخدام البطاقة.

وتوفر الخدمة كذلك إمكانية الحصول على العمليات المرتبطة ببطاقة معينة، مما يسمح للواجهة الأمامية بعرض سجل العمليات التي تمت باستخدام البطاقة.

يساعد وجود هذه العمليات ضمن خدمة مستقلة على فصل منطق إدارة البطاقات عن الصفحة التي تعرض بياناتها، كما يسهل إعادة استخدام هذه العمليات في أجزاء أخرى من التطبيق عند الحاجة.

4-7-5 خدمة الفواتير والاشتراكات Billing Service:

يحتوي الملف `billingService.js` على العمليات المرتبطة بدفع الفواتير وإدارة الاشتراكات الدورية.

تتيح الخدمة للمستخدم تنفيذ عملية دفع فاتورة من حساب معين إلى حساب التاجر، مع تحديد المبلغ المطلوب دفعه وإضافة وصف للعملية.

كما تسمح بإنشاء اشتراك دوري من خلال تحديد الحساب المصدر وحساب التاجر، بالإضافة إلى قيمة الاشتراك ووحدة التكرار وعدد الوحدات المطلوبة. ويمكن أن تكون وحدة التكرار مثل اليوم أو الأسبوع أو الشهر وفق الخيارات التي يوفرها النظام.

وتوفر الخدمة أيضًا وظائف لاستعراض الدفعات السابقة والاشتراكات الموجودة، والحصول على بيانات اشتراك محدد، بالإضافة إلى إلغاء الاشتراك عند الحاجة.

وبذلك تجمع Billing Service العمليات المرتبطة بالفواتير والاشتراكات ضمن طبقة واحدة، بينما تظل تفاصيل الاتصال بالواجهة الخلفية ومعالجة الطلبات ضمن طبقة API Client.

4-7-6 خدمة التحقق من الهوية KYC Service:

يتعامل الملف kycService.js مع العمليات المتعلقة بالتحقق من هوية المستخدم وإدارة الوثائق المطلوبة ضمن عملية KYC.

تدعم عملية التحقق عدة أنواع من الوثائق، وتشمل الوجه الأمامي لبطاقة الهوية، والوجه الخلفي لبطاقة الهوية، وإثبات العنوان، بالإضافة إلى الصورة الشخصية مع بطاقة الهوية.

تتضمن الخدمة وظائف للحصول على طلب التحقق الحالي، واستعراض الوثائق المرتبطة بالطلب، وإنشاء مساحة مخصصة لرفع الوثيقة، والحصول على رابط تنزيل الوثيقة، بالإضافة إلى إرسال طلب التحقق بعد استكمال البيانات والوثائق المطلوبة.

كما تحتوي الخدمة على مجموعة من الوظائف المخصصة للمستخدم الإداري، والتي تسمح بعرض طلبات التحقق التي تحتاج إلى مراجعة، واستعراض الوثائق المرتبطة بطلب معين، بالإضافة إلى الموافقة على طلب التحقق أو رفضه.

يسمح هذا التنظيم بفصل عمليات التحقق من الهوية والوثائق عن صفحة CompliancePage، بحيث تتولى الصفحة عرض الواجهة والتعامل مع إدخال المستخدم، بينما تتولى KYC Service تنفيذ عمليات الاتصال الخاصة بهذه الوظائف.

4-7-7 خدمة سجل المعاملات Ledger Service:

يختص الملف ledgerService.js بالوصول إلى بيانات دفتر الحسابات وسجل العمليات المالية المرتبطة بالحسابات.

توفر الخدمة إمكانية الحصول على رصيد حساب معين، والحصول على قيد محدد من سجل المعاملات، بالإضافة إلى استعراض القيود الخاصة بالحساب والحصول على رأس سلسلة السجل.

كما توفر الخدمة الوظيفة verifyChain التي تستخدم لطلب التحقق من سلسلة سجل الحساب من الواجهة الخلفية، وإظهار موضع أول خلل في السلسلة عند وجوده.

وتوفر الخدمة كذلك الوظيفة `getBalances` التي تسمح بالحصول على أرصدة مجموعة من الحسابات في عملية واحدة. ويتم استخدام `Promise.allSettled` عند التعامل مع هذه الطلبات، بحيث لا يؤدي فشل الحصول على رصيد أحد الحسابات إلى إلغاء جميع الطلبات الأخرى.

يساعد هذا الأسلوب على التعامل مع بيانات الأرصدة بصورة أكثر مرونة، حيث يمكن للواجهة الأمامية معالجة النتائج الناجحة والاستمرار في عرض البيانات المتاحة حتى في حال حدوث خطأ في أحد الطلبات.

4-7-8 خدمة المستخدمين `Users Service`:

يحتوي الملف `userService.js` على الوظائف المرتبطة بالبحث عن المستخدمين والحصول على بعض بياناتهم.

تستخدم عملية البحث عن المستخدمين بشكل خاص عند إدارة أعضاء الحسابات المشتركة، حيث يمكن للمستخدم البحث عن مستخدم آخر باستخدام البيانات المتاحة، ثم اختيار المستخدم المطلوب لإرسال دعوة للانضمام إلى الحساب المشترك.

كما يحتوي الملف على بعض الوظائف الإدارية المرتبطة بالمستخدمين، مثل الحصول على بيانات مستخدم معين وتغيير حالة حظر المستخدم.

يساعد فصل هذه العمليات في خدمة مستقلة على منع تكرار منطق البحث وإدارة المستخدمين داخل الصفحات المختلفة، كما يحافظ على تنظيم طبقة الخدمات وتقسيمها وفق المجالات الوظيفية للنظام.

4-8 صفحات الواجهة الأمامية:

تم تقسيم الوظائف الرئيسية للنظام إلى مجموعة من الصفحات، بحيث تمثل كل صفحة مجاًلاً وظيفياً واضحاً داخل النظام المصرفي.

4-8-1 صفحة المصادقة `AuthPage`:

تمثل `AuthPage.jsx` نقطة الدخول للمستخدم غير المصادق عليه.

تجمع الصفحة بين LoginForm و RegisterForm، وتسمح بالتبديل بين تسجيل الدخول وإنشاء الحساب دون الحاجة إلى الانتقال إلى صفحة مستقلة.

كما تستخدم الصفحة AnimatePresence لإضافة انتقالات حركية عند تغيير النموذج الظاهر.

تتعامل الصفحة أيضًا مع حالة المستخدم الذي حاول تسجيل الدخول قبل إكمال التحقق من بريده الإلكتروني، حيث يتم تمرير البريد الإلكتروني إلى نموذج التسجيل ليتمكن المستخدم من إدخال رمز التحقق مباشرة.

4-8-2 صفحة لوحة التحكم Dashboard:

تمثل DashboardPage.jsx الصفحة الرئيسية للمستخدم بعد تسجيل الدخول.

تعرض الصفحة إجمالي الرصيد المتاح عبر الحسابات، بالإضافة إلى توزيع الرصيد بين الحسابات الجارية وحسابات التوفير.

كما تعرض الحسابات البنكية الخاصة بالمستخدم مع نوع الحساب والرصيد ورقم IBAN بشكل مختصر.

وتعرض الصفحة كذلك آخر التحويلات، والدعوات المتعلقة بالحسابات المشتركة، وحالة التحقق من الهوية.

وتتضمن الصفحة إجراءات مباشرة مثل فتح حساب جديد وإرسال الأموال، ويتم تنفيذ هذه العمليات من خلال النوافذ المنبثقة CreateAccountModal و TransferModal.

عند تحميل الصفحة، يتم استخدام Promise.allSettled لجلب عدة مجموعات من البيانات في الوقت نفسه، مثل التحويلات والدعوات وحالة KYC، مما يسمح بعرض كل جزء من البيانات عند نجاح طلبه دون أن يؤدي فشل أحد الطلبات إلى إيقاف بقية الصفحة.

4-8-3 صفحة الحسابات Accounts:

تمثل AccountsPage.jsx الصفحة المخصصة لإدارة الحسابات البنكية.

يمكن للمستخدم من خلالها استعراض حساباته، وإنشاء حساب جديد، وتعديل اسم الحساب، واستعراض أعضاء الحسابات المشتركة وإدارتهم.

كما تسمح الصفحة بإرسال دعوات للمستخدمين للانضمام إلى الحساب، مع تحديد الدور المطلوب للعضو. وتوفر الصفحة كذلك عرض سجل العمليات المرتبط بالحساب، مع إمكانية استخدام بيانات سجل المعاملات وتصديرها بصيغة CSV.

كما يتم عرض تفاصيل الحساب وتحديد الحساب المطلوب من خلال معاملات عنوان الصفحة، مما يسمح بالانتقال مباشرة إلى حساب محدد باستخدام معرف الحساب الموجود في عنوان URL.

4-8-4 صفحة التحويلات Payments:

تمثل PaymentsPage.jsx الواجهة الخاصة بتنفيذ التحويلات المالية وعرض التحويلات السابقة.

تسمح الصفحة باختيار الحساب الذي ستخرج منه الأموال، وإدخال معرف الحساب المستلم، وتحديد المبلغ والوصف.

قبل إرسال التحويل، يتم تنفيذ مجموعة من عمليات التحقق على مستوى الواجهة، مثل التأكد من اختيار الحساب المصدر، والتحقق من صحة UUID الخاص بالحساب المستلم، ومنع التحويل إلى الحساب نفسه، والتأكد من أن المبلغ أكبر من صفر.

كما يتم حساب رسوم التحويل وإضافتها إلى إجمالي المبلغ المطلوب من الحساب المصدر قبل تنفيذ العملية.

بعد نجاح التحويل يتم تحديث البيانات وإظهار رسالة نجاح للمستخدم، بينما يتم عرض رسالة خطأ مناسبة عند فشل العملية.

كما تحتوي الصفحة على سجل للتحويلات مع إمكانية التصفية حسب حالة التحويل والحساب.

4-8-5 صفحة البطاقات Cards:

تمثل CardsPage.jsx الواجهة الخاصة بإدارة البطاقات الافتراضية.

تعرض الصفحة البطاقات الموجودة للمستخدم على شكل بطاقات رسومية تحتوي على الاسم المختصر وآخر أربعة أرقام وحالة البطاقة.

يمكن للمستخدم إنشاء بطاقة جديدة عن طريق تحديد حساب التمويل وإدخال الاسم الاختياري للبطاقة.

كما يمكن تحديد بطاقة معينة لعرض تفاصيلها وتنفيذ عمليات مثل التجميد وإلغاء التجميد، بالإضافة إلى تنفيذ عملية دفع باستخدام البطاقة.

وتعرض الصفحة أيضًا سجل العمليات المرتبطة بالبطاقة المحددة.

4-8-6 صفحة الفواتير والاشتراكات Billing:

تمثل BillingPage.jsx الواجهة الخاصة بدفع الفواتير وإدارة الاشتراكات.

تحتوي الصفحة على نموذج مستقل لدفع فاتورة، حيث يحدد المستخدم الحساب الذي سيتم الخصم منه وحساب التاجر والمبلغ والوصف.

كما تحتوي على نموذج آخر لإنشاء اشتراك دوري، يمكن من خلاله تحديد الحساب المصدر وحساب التاجر والمبلغ ووحدة التكرار وعدد مرات التكرار والوصف.

بعد تنفيذ العمليات يتم تحديث قائمة الدفعات والاشتراكات، كما تعرض الصفحة حالة الاشتراك وتوفر إمكانية إلغاؤه عند الحاجة.

4-8-7 صفحة التحقق من الهوية Compliance:

تمثل CompliancePage.jsx الواجهة الخاصة بعملية KYC.

تتيح الصفحة للمستخدم إدخال بياناته الأساسية، مثل الاسم الكامل والرقم الوطني والعنوان، بالإضافة إلى رفع الوثائق المطلوبة.

قبل رفع الوثيقة، يتم حساب قيمة SHA-256 للملف باستخدام Web Crypto API، ثم يتم إرسال معلومات الوثيقة إلى الواجهة الخلفية لإنشاء مساحة رفع.

بعد الحصول على رابط الرفع، يتم إرسال الملف إلى التخزين الآمن MinIO من خلال طلب PUT.

تقرض الواجهة حدًا أقصى لحجم الملف يبلغ 10 ميغابايت، كما تعرض للمستخدم حالة عملية الرفع، مثل حساب البصمة، وحجز مساحة الوثيقة، ورفع الملف.

بعد تجهيز الوثائق يتم إرسال طلب KYC إلى الواجهة الخلفية، ثم تعرض الصفحة حالة الطلب، سواء كان قيد الانتظار أو مقبولا أو مرفوضا.

4-8-8 صفحة الإدارة Admin:

تمثل AdminPage.jsx الواجهة الخاصة بالمستخدم الإداري.

تم تقسيم الصفحة إلى عدة تبويبات وظيفية، وهي:

- Overview.
- Treasury.
- KYC Review.
- Users.
- Accounts.
- Transfers.

يسمح تبويب Overview بعرض المعلومات العامة والعمليات التي يحتاج المسؤول إلى متابعتها.

أما Treasury فيرتبط بالعمليات المالية الخاصة بحساب الخزينة، بينما يتيح قسم KYC مراجعة طلبات التحقق من الهوية والوثائق المرتبطة بها.

كما يتيح قسم Users البحث عن المستخدمين وإدارة بعض حالاتهم، بينما يسمح قسم Accounts باستعراض الحسابات وتنفيذ العمليات الإدارية الخاصة بها.

ويتيح قسم Transfers استعراض التحويلات المالية ومتابعتها.

وتستخدم الصفحة مكونات وخدمات مستقلة لكل مجال، مما يمنع تجمع جميع العمليات الإدارية داخل وظيفة واحدة.

4-9 المكونات القابلة لإعادة الاستخدام:

تم تصميم الواجهة الأمامية بالاعتماد على مجموعة من المكونات البرمجية القابلة لإعادة الاستخدام، بحيث يتم تقسيم عناصر الواجهة إلى مكونات مستقلة يمكن استخدامها في أكثر من صفحة أو جزء من التطبيق. ويساعد هذا الأسلوب على تقليل تكرار الكود، وتوحيد شكل الواجهة وسلوك عناصرها، بالإضافة إلى تسهيل عملية صيانة المشروع وتطويره.

4-9-1 مكونات واجهة المستخدم UI:

تم إنشاء مجموعة من المكونات المشتركة داخل الملف `components/ui/index.jsx`، وذلك بهدف توحيد شكل عناصر واجهة المستخدم وتقليل تكرار الكود بين الصفحات المختلفة.

تشمل هذه المكونات `Button` المستخدم لإنشاء الأزرار، و `Card` لعرض المحتوى ضمن بطاقات، و `TextField` لحقول إدخال البيانات، و `MoneyField` لإدخال القيم المالية، و `SelectField` للقوائم المنسدلة، بالإضافة إلى مجموعة من المكونات الخاصة بعرض الحالات والتنبيهات والنوافذ المنبثقة.

يسمح استخدام هذه المكونات بتطبيق نفس التصميم والسلوك على عناصر الواجهة في صفحات متعددة، دون الحاجة إلى إعادة كتابة بنية `HTML` وقواعد `CSS` الخاصة بكل عنصر في كل صفحة.

كما يساعد هذا الأسلوب على إجراء التعديلات المتعلقة بتصميم أحد العناصر من مكان واحد، بحيث ينعكس التعديل على جميع الصفحات التي تستخدم المكون نفسه.

4-9-2 مكونات المصادقة:

تم تقسيم واجهة المصادقة إلى مجموعة من المكونات المستقلة، ومن أبرزها `AuthField` و `AuthShell` و `LoginForm` و `RegisterForm`.

يستخدم `AuthField` لتوفير حقول الإدخال المستخدمة ضمن نماذج المصادقة، بينما يتولى `AuthShell` توفير الهيكل العام لواجهة المصادقة.

أما LoginForm و RegisterForm فيتوليان التعامل مع نماذج تسجيل الدخول وإنشاء الحساب، بما في ذلك استقبال بيانات المستخدم وتنفيذ عمليات التحقق الأولية وإرسال البيانات إلى خدمة المصادقة.

يساعد هذا التقسيم على فصل الهيكل العام لواجهة المصادقة عن تفاصيل النماذج، كما يسمح بإعادة استخدام الحقول والمكونات المشتركة دون الحاجة إلى تكرارها في أكثر من مكان.

4-9-3 مكون تخطيط التطبيق AppLayout:

يستخدم المكون AppLayout.jsx لتوفير الهيكل العام للصفحات الداخلية في التطبيق، بحيث تشترك الصفحات المصرفية المختلفة في نفس التخطيط العام وعناصر التنقل.

يتضمن المكون القائمة الجانبية، واسم النظام، وبيانات المستخدم، وروابط التنقل بين الصفحات، بالإضافة إلى زر تسجيل الخروج.

كما يتعامل AppLayout مع عرض القائمة الجانبية على الشاشات الصغيرة، حيث يمكن فتح القائمة وإغلاقها بما يتناسب مع حجم الشاشة، مما يساعد على توفير واجهة متجاوبة مع مختلف الأجهزة.

ويتم تحديد روابط التنقل للظاهرة للمستخدم وفقاً لنوع المستخدم وصلاحياته، بحيث تظهر للمستخدم العادي روابط الخدمات المصرفية المتاحة له، بينما يتم توفير الروابط الخاصة بالواجهة الإدارية للمستخدم الذي يمتلك الصلاحيات الإدارية.

4-9-4 النوافذ المنبثقة Modals:

يحتوي المشروع على مجموعة من النوافذ المنبثقة التي تستخدم لتنفيذ بعض العمليات دون الحاجة إلى الانتقال إلى صفحة جديدة.

ومن أبرز هذه النوافذ CreateAccountModal، والذي يستخدم لإنشاء حساب بنكي جديد، بالإضافة إلى TransferModal المستخدم لتنفيذ عمليات التحويل المالي.

تسمح هذه النوافذ بعرض نموذج العملية المطلوبة فوق الصفحة الحالية، ثم تنفيذ العملية وإغلاق النافذة بعد الانتهاء منها. ويساعد ذلك على إبقاء المستخدم ضمن الصفحة التي يعمل عليها، وتقليل عمليات التنقل بين الصفحات عند تنفيذ العمليات السريعة والمتكررة.

4-9-5 مكون AccountSelect:

يستخدم المكون AccountSelect.jsx لعرض الحسابات البنكية المتاحة للمستخدم وتمكينه من اختيار الحساب المطلوب لتنفيذ العملية.

يتم استخدام هذا المكون في أكثر من جزء من النظام، مثل صفحات التحويلات والبطاقات والفواتير، حيث تحتاج هذه العمليات إلى تحديد الحساب الذي سيتم استخدامه كمصدر للأموال.

يساهم استخدام AccountSelect كمكون مستقل في توحيد طريقة عرض الحسابات واختيارها في مختلف الصفحات، كما يمنع تكرار الكود الخاص بتحميل الحسابات وعرضها داخل كل صفحة.

4-9-6 مكون LedgerChainVisualizer:

يستخدم المكون LedgerChainVisualizer.jsx لعرض سلسلة سجل المعاملات بصورة مرئية تساعد المستخدم على متابعة تسلسل القيود والتحقق من سلامة السلسلة.

يقوم المكون بتحميل قيود الحساب وترتيبها حسب التسلسل، ثم يقوم بفحص العلاقة بين القيود من خلال مقارنة قيمة prevHash في القيد الحالي مع قيمة itemHash في القيد السابق.

بعد الانتهاء من عملية الفحص المحلي، يتم استدعاء الوظيفة verifyChain الموجودة في خدمة Ledger لإجراء عملية التحقق من السلسلة من خلال الواجهة الخلفية.

يسمح هذا الأسلوب بعرض حالة سلسلة سجل المعاملات بصورة واضحة للمستخدم، كما يمكن تحديد موضع الخلل في حال اكتشاف عدم تطابق بين القيود، مما يوفر وسيلة مرئية لمتابعة سلامة تسلسل سجل المعاملات.

10-4 إدارة الحالة والوظائف المساعدة:

تحتوي الواجهة الأمامية على مجموعة من الأدوات البرمجية التي تساعد في إدارة الحالات المشتركة بين مكونات التطبيق، بالإضافة إلى مجموعة من الوظائف المساعدة التي تستخدم في معالجة البيانات وتنفيذ بعض العمليات المتكررة. ويساهم فصل هذه الوظائف عن الصفحات والمكونات في تقليل تكرار الكود وتحسين تنظيم المشروع.

:AuthContext 1-10-4

يستخدم الملف `AuthContext.jsx` لإدارة حالة المستخدم والمصادقة على مستوى التطبيق، بحيث يمكن للمكونات والصفحات المختلفة الوصول إلى بيانات المستخدم وحالة الجلسة دون الحاجة إلى تمريرها بشكل متكرر من خلال الخصائص.

يوفر `AuthContext` معلومات تتعلق ببيانات المستخدم وحالة المصادقة، بالإضافة إلى الوظائف المرتبطة بإدارة الجلسة، مثل تسجيل الدخول وتسجيل الخروج وتحديث بيانات المستخدم.

عند تشغيل التطبيق، يتم التحقق من وجود `Access Token`. وفي حال وجود الرمز، تحاول الواجهة الأمامية استعادة جلسة المستخدم من خلال استدعاء نقطة `/auth/me` للحصول على بيانات المستخدم الحالي والتحقق من صلاحية الجلسة.

كما يتم الربط بين `AuthContext` و `tokenStore`، بحيث يتم إشعار `AuthContext` عند حدوث تغيير في رموز المصادقة. فعند حذف الرموز نتيجة تسجيل الخروج أو انتهاء الجلسة أو فشل عملية تجديد رمز الوصول، يتم تحديث حالة المستخدم داخل التطبيق وإعادتها إلى الحالة غير المصادق عليها.

يساعد هذا الأسلوب على جعل حالة المصادقة مركزية ومتاحة لمختلف أجزاء التطبيق، كما يسمح باستخدامها في حماية المسارات وعرض بيانات المستخدم وتنفيذ العمليات المرتبطة بجلسة المستخدم.

:ToastContext 2-10-4

يستخدم الملف `ToastContext.jsx` لإدارة رسائل التنبيه والإشعارات التي تظهر للمستخدم أثناء استخدام النظام.

يوفر هذا السياق آلية مشتركة يمكن للصفحات والمكونات من خلالها عرض رسائل مختلفة، مثل رسائل النجاح والمعلومات والأخطاء، دون الحاجة إلى إنشاء مكون خاص لعرض التنبيه داخل كل صفحة.

فعلى سبيل المثال، بعد نجاح عملية مصرفية يمكن للصفحة استدعاء وظيفة عرض رسالة نجاح، بينما يمكن استخدامها أيضًا لعرض رسالة توضح سبب فشل العملية في حال حدوث خطأ.

يساعد استخدام ToastContext على توحيد طريقة عرض الرسائل والإشعارات في مختلف أجزاء التطبيق، كما يجعل التعامل مع نتائج العمليات المصرفية أكثر وضوحًا للمستخدم.

3-10-4 useAccounts:

يحتوي الملف useAccounts.js على Hook مخصص لإدارة بيانات الحسابات البنكية وأرصدها.

يقوم هذا ال Hook بتحميل حسابات المستخدم من خلال accountsService، ثم يقوم بالحصول على أرصدة هذه الحسابات من خلال ledgerService. وبعد الحصول على البيانات، يتم تجميعها وربط كل حساب بالرصيد الخاص به.

يوفر ال Hook مجموعة من البيانات التي تحتاج إليها الصفحات، مثل قائمة الحسابات، والأرصدة، وإجمالي الرصيد، بالإضافة إلى حالة التحميل والأخطاء التي قد تحدث أثناء جلب البيانات.

كما يوفر وظيفة reload التي تسمح بإعادة تحميل الحسابات والأرصدة عند الحاجة، مثلًا بعد إنشاء حساب جديد أو تنفيذ عملية تؤثر على الرصيد.

يساعد استخدام useAccounts على منع تكرار منطق تحميل الحسابات والأرصدة داخل أكثر من صفحة، بحيث يمكن إعادة استخدام نفس المنطق في الصفحات التي تحتاج إلى بيانات الحسابات.

4-10-4 الوظائف المساعدة Format:

يحتوي الملف format.js على مجموعة من الوظائف المساعدة التي تستخدم لتنسيق البيانات ومعالجتها قبل عرضها في واجهة المستخدم.

تشمل هذه الوظائف تنسيق المبالغ المالية والتواريخ، وإخفاء جزء من رقم IBAN، والتحقق من صحة UUID، وتحويل المبالغ المالية من الدولار إلى السنتات، بالإضافة إلى تحديد نوع الحساب وحالة KYC ودور عضو الحساب.

يسمح لجميع هذه الوظائف في ملف مستقل باستخدام نفس طريقة تنسيق البيانات في مختلف أجزاء التطبيق، كما يمنع تكرار عمليات التنسيق داخل الصفحات والمكونات.

ويؤدي ذلك إلى جعل الكود أكثر تنظيماً، بحيث تركز الصفحات والمكونات على عرض البيانات والتفاعل معها، بينما تتولى الوظائف المساعدة عمليات التنسيق والتحويل المطلوبة.

4-10-5 توليد مفاتيح Idempotency:

يحتوي الملف crypto.js على الوظيفة newIdempotencyKey، والتي تستخدم لإنشاء معرف فريد للعمليات التي تؤدي إلى إنشاء أو تعديل البيانات.

تستخدم هذه المفاتيح في عدد من العمليات الحساسة، مثل إنشاء الحسابات، وتنفيذ التحويلات، وإنشاء البطاقات، ودفع الفواتير، وإنشاء الاشتراكات.

يتم إرسال مفتاح Idempotency مع الطلب إلى الواجهة الخلفية، بحيث يمكن استخدامه لتمييز العملية الأصلية عن محاولات إعادة إرسال الطلب نفسه.

يساعد ذلك على تقليل احتمالية تنفيذ العملية أكثر من مرة في الحالات التي يتم فيها إعادة إرسال الطلب بسبب مشكلة في الاتصال أو إعادة المحاولة، خصوصاً في العمليات المالية التي قد يؤدي تكرارها إلى نتائج غير مرغوبة.

4-10-6 التحقق من الملفات:

يحتوي الملف crypto.js كذلك على الوظيفة sha256Hex، والتي تستخدم لحساب قيمة SHA-256 للملفات باستخدام واجهة Web Crypto API من خلال crypto.subtle.

تستخدم هذه الوظيفة ضمن عملية رفع وثائق التحقق من الهوية KYC، حيث يتم حساب البصمة الرقمية للملف قبل رفعه إلى التخزين.

بعد ذلك يتم إرسال قيمة البصمة إلى خدمة التحقق من الهوية ضمن بيانات الوثيقة، مما يسمح للواجهة الخلفية باستخدامها للتحقق من تطابق الملف المرفوع مع الملف الذي تم تسجيل بصمته أثناء إنشاء مساحة الوثيقة.

يساهم استخدام البصمة الرقمية في توفير آلية إضافية للتحقق من سلامة الوثائق أثناء عملية الرفع والتخزين، كما يساعد على ربط الملف المرفوع بالبيانات التي تم تسجيلها عنه في النظام.

11-4 آلية التحكم في الوصول إلى الصفحات:

تم تنفيذ آلية التحكم في الوصول إلى الصفحات داخل الملف App.jsx من خلال مجموعة من المكونات المخصصة لحماية المسارات، بحيث يتم تحديد الصفحات التي يمكن للمستخدم الوصول إليها بناءً على حالة المصادقة والدور الخاص به.

يستخدم المكون RequireAuth للتحقق من حالة مصادقة المستخدم قبل السماح له بالوصول إلى الصفحات المحمية. فإذا لم تكن هناك جلسة مصادقة صالحة، يتم توجيه المستخدم إلى المسار /login، وبالتالي لا يستطيع المستخدم غير المصادق عليه الوصول مباشرة إلى الصفحات المصرفية الداخلية.

أما المكون RequireAdmin فيستخدم للتحقق من امتلاك المستخدم للدور الإداري ADMIN. وفي حال كان المستخدم مسجلاً للدخول ولكنه لا يمتلك هذا الدور، يتم منعه من الوصول إلى الواجهة الإدارية وتوجيهه إلى الصفحة الرئيسية الخاصة بالمستخدم.

كما تم استخدام المكون RequireCustomer للتحكم في وصول المستخدمين إلى صفحات الخدمات المصرفية الخاصة بالعميل. فإذا كان المستخدم يمتلك الدور الإداري، يتم توجيهه تلقائياً إلى المسار /admin بدلاً من السماح له بالدخول إلى صفحات العميل.

وبذلك لا تعتمد آلية التحكم في الوصول على التحقق من تسجيل الدخول فقط، وإنما تأخذ أيضاً في الاعتبار نوع المستخدم والدور الذي يمتلكه، مما يسمح بفصل واجهة المستخدم العادية عن الواجهة الإدارية.

ومن المهم الإشارة إلى أن آلية الحماية الموجودة في الواجهة الأمامية تستخدم بشكل أساسي للتحكم في التنقل وتحسين تجربة المستخدم، ولا تعتبر بديلاً عن التحقق الأمني في الواجهة الخلفية. حيث يقوم Gateway والخدمات الخلفية بالتحقق من صحة رمز JWT والصلاحيات المطلوبة قبل تنفيذ العمليات الحساسة، مما يمنع تجاوز الصلاحيات من خلال إرسال الطلبات مباشرة إلى الواجهة الخلفية.

4-12 التعامل مع العمليات الحساسة:

نظراً لطبيعة النظام المصرفية وحساسية العمليات التي يتم تنفيذها من خلال الواجهة الأمامية، تم تضمين مجموعة من الآليات التي تساعد على تقليل الأخطاء أثناء تنفيذ العمليات والتعامل مع البيانات الحساسة.

فعلى سبيل المثال، تقوم صفحة التحويلات بالتحقق من البيانات الأساسية للعملية قبل إرسالها إلى الواجهة الخلفية، مثل تحديد الحساب المصدر والحساب المستلم، والتحقق من صحة المبلغ، بالإضافة إلى التأكد من توفر الرصيد المطلوب لتنفيذ العملية.

كما تستخدم العمليات التي تؤدي إلى إنشاء أو تعديل البيانات المالية مفاتيح Idempotency، حيث يتم إنشاء مفتاح فريد لكل عملية وإرساله مع الطلب إلى الواجهة الخلفية. ويساعد ذلك على منع تنفيذ العملية نفسها أكثر من مرة في حال إعادة إرسال الطلب نتيجة مشكلة في الاتصال أو إعادة المحاولة.

أما عمليات رفع وثائق التحقق من الهوية KYC، فتستخدم آلية حساب قيمة SHA-256 للملف قبل رفعه، حيث يتم إنشاء بصمة رقمية للوثيقة وإرسالها إلى خدمة التحقق من الهوية. كما يتم رفع الملفات من خلال روابط مخصصة يتم الحصول عليها من الواجهة الخلفية، بدلاً من إرسال محتوى الملف مباشرة ضمن طلب API الرئيسي.

كما تم فصل الوظائف الإدارية عن واجهة المستخدم العادية، بحيث يتم التحكم في ظهور الصفحات والوظائف الإدارية بناءً على دور المستخدم. ولا تظهر الوظائف المخصصة للمسؤول للمستخدمين الذين لا يمتلكون الدور المطلوب.

تساهم هذه الآليات في تحسين موثوقية الواجهة الأمامية وتقليل الأخطاء الناتجة عن إدخال البيانات أو إعادة إرسال العمليات، مع الإبقاء على مسؤولية التحقق الأمني النهائي وتنفيذ قواعد العمل لدى الواجهة الخلفية والGateway.

4-13 تصميم الواجهة وتجربة المستخدم:

تم الاعتماد على Tailwind CSS في تصميم المظهر العام للواجهة الأمامية، مع إنشاء مجموعة من المكونات المشتركة التي تساعد على توحيد أسلوب التصميم بين الصفحات المختلفة والحفاظ على تناسق عناصر الواجهة.

وتستخدم الواجهة مجموعة من المكونات الموحدة لعرض البيانات وتنفيذ العمليات، مثل البطاقات التي تستخدم لتنظيم وعرض المحتوى، والأزرار التي تختلف أنماطها حسب طبيعة العملية، بالإضافة إلى مكونات عرض حالات العمليات مثل StatusBadge وBadge، والتي تساعد على توضيح حالة العملية للمستخدم بصورة مباشرة.

كما تم توفير مجموعة من المكونات التي تتعامل مع الحالات المختلفة أثناء استخدام التطبيق، حيث يستخدم Spinner لعرض حالة التحميل أثناء انتظار استجابة الواجهة الخلفية، بينما يستخدم EmptyState عند عدم توفر بيانات لعرضها، ويستخدم ErrorNotice عند حدوث خطأ أثناء تحميل البيانات أو تنفيذ إحدى العمليات.

يساعد هذا الأسلوب على جعل الواجهة أكثر وضوحًا وتنظيمًا، خصوصًا في العمليات التي تحتاج إلى وقت للحصول على استجابة من الخدمات الخلفية، حيث يستطيع المستخدم معرفة حالة العملية بدلًا من ظهور صفحة فارغة أو عدم وجود مؤشر على أن النظام يقوم بتنفيذ الطلب.

كما تم الاهتمام بتوافق الواجهة مع أحجام الشاشات المختلفة من خلال استخدام تخطيطات متجاوبة، بحيث تتكيف عناصر الصفحة مع حجم الشاشة المتاح. وتم كذلك توفير قائمة تنقل مناسبة للشاشات الصغيرة ضمن المكون AppLayout، مما يسمح للمستخدم بالوصول إلى الصفحات المختلفة من الأجهزة ذات الشاشات الصغيرة بصورة أكثر سهولة.

14-4 خلاصة تنفيذ الواجهة الأمامية:

تم تصميم وتنفيذ الواجهة الأمامية لنظام المصرف الإلكتروني باستخدام مكتبة React، مع الاعتماد على مجموعة من المكونات البرمجية القابلة لإعادة الاستخدام، وفصل واضح بين الصفحات والمكونات والخدمات وإدارة الحالة والوظائف المساعدة. وقد ساهم هذا التنظيم في جعل بنية المشروع أكثر وضوحًا وقابلية للصيانة والتطوير.

كما ساعد فصل مسؤوليات طبقات الواجهة الأمامية على تقليل الاعتماد المباشر بين الصفحات والخدمات الخلفية، حيث يتم تنفيذ عمليات الاتصال من خلال API Client مركزي، بينما يتم توزيع طلبات النظام على مجموعة من Services وفقًا للمجال الوظيفي لكل خدمة.

تم كذلك تنفيذ نظام متكامل للمصادقة وإدارة جلسة المستخدم يعتمد على JWT و Refresh Token، مع توفير آلية لحماية المسارات، وتجديد رمز الوصول عند انتهاء صلاحيته، والتعامل مع انتهاء الجلسة وإعادة المستخدم إلى واجهة تسجيل الدخول عند الحاجة.

وتغطي الواجهة الأمامية مختلف الوظائف الرئيسية لنظام المصرف الإلكتروني، بدءًا من إنشاء الحساب وتسجيل الدخول والتحقق من البريد الإلكتروني، مرورًا بإدارة الحسابات والتحويلات المالية والبطاقات الافتراضية والفواتير والاشتراكات، ووصولًا إلى عمليات التحقق من الهوية والوظائف المخصصة للمستخدم الإداري.

كما تم الاهتمام بتحسين تجربة المستخدم من خلال توفير مكونات مشتركة لعرض البيانات والحالات المختلفة، مثل حالات التحميل وعدم وجود البيانات والأخطاء، بالإضافة إلى التحقق من المدخلات قبل

إرسال الطلبات، وإعادة استخدام المكونات في الصفحات المختلفة، وتطبيق التصميم المتجاوب مع أحجام الشاشات المختلفة.

ومن الجوانب المهمة أيضًا التعامل مع العمليات الحساسة، حيث يتم استخدام مفاتيح Idempotency للعمليات التي تؤدي إلى إنشاء أو تعديل البيانات، بالإضافة إلى استخدام SHA-256 للتحقق من وثائق KYC قبل رفعها، مع تطبيق آليات مناسبة للتعامل مع أخطاء الاتصال والاستجابات المختلفة من الواجهة الخلفية.

وبذلك تمثل الواجهة الأمامية طبقة التفاعل الرئيسية بين المستخدم وبقية مكونات نظام المصرف الإلكتروني، بينما يعمل Gateway كحلقة وصل موحدة بينها وبين الخدمات المصغرة الداخلية. ويتولى النظام الخلفي بدوره تنفيذ عمليات التحقق والمعالجة وتطبيق قواعد العمل والصلاحيات والتعامل مع البيانات، مما يحقق فصلًا واضحًا بين مسؤوليات الواجهة الأمامية وبقية طبقات النظام.

References

- [1] UML@Classroom, Martina Seidl, Marion Scholz (ehem. Brandsteidl), Christian Huemer, Gerti Kappel, [uml.ac.at/en/](http://www.uml.ac.at/en/), 2015. Available: <http://www.uml.ac.at/en/>
- [2] Microservices Patterns, Chris Richardson, Manning Publications, 2018. Available: <https://microservices.io/>
- [3] Designing Data-Intensive Applications, Martin Kleppmann, O'Reilly Media, 2017. Available: <https://dataintensive.net/>
- [4] Spring Boot Reference Documentation, Pivotal Software, 2024. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [5] gRPC Documentation, Google Developers, 2024. Available: <https://grpc.io/docs/>
- [6] PostgreSQL Documentation, PostgreSQL Global Development Group, 2024. Available: <https://www.postgresql.org/docs/>
- [7] Apache Kafka Documentation, The Apache Software Foundation, 2024. Available: <https://kafka.apache.org/documentation/>
- [8] OWASP Top Ten Web Application Security Risks, OWASP Foundation, 2021. Available: <https://owasp.org/www-project-top-ten/>
- [9] JSON Web Token (JWT), RFC 7519, IETF, 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [10] Docker Documentation, Docker Inc., 2024. Available: <https://docs.docker.com/>
- [11] Flyway Database Migration Documentation, Redgate, 2024. Available: <https://flywaydb.org/documentation/>
- [12] MinIO Object Storage Documentation, MinIO Inc., 2024. Available: <https://min.io/docs/>
- [13] React Documentation, React – The library for web and native user interfaces, Meta Platforms, Inc., 2024. Available: <https://react.dev/>
- [14] Vite Documentation, Vite – Next Generation Frontend Tooling, Vite Team, 2024. Available: <https://vite.dev/guide/>
- [15] Axios Documentation, Axios – Promise based HTTP client for the browser and Node.js, Axios Contributors, 2024. Available: <https://axios-http.com/docs/intro>
- [16] Tailwind CSS Documentation, Tailwind CSS – Rapidly build modern websites without ever leaving your HTML, Tailwind Labs, 2024. Available: <https://tailwindcss.com/docs/>